408 详解 · 2021 年选择题 · 从零版

只含选择题第 1–40 题，不含综合应用题。

每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。

说明：凡题干问"不能得到 / 不能精确表示 / 不属于 / 错误的是 / 不必完成 / 不可能是"的题（第 2、14、18、19、20、22、30、35 题），第 4 节说明正确选项（即那个错述 / 不可能项）为什么成立，第 5 节说明其余选项为什么是对的（配例子），这样 6 节结构全卷统一。

默认读者是零基础跨考学生：不默认任何 408 前置知识，每题自成体系，不依赖翻王道书、不依赖其他题的解析；题干与选项里的每个术语都讲清，推理和计算的每一步都写出，不用"显然""易得""同理"这类跳步词；错误选项的例子选最直观的小数字、小例子、生活类比。

本版每题含完整题干与选项，可脱离真题册独立使用。

答案以原卷解析首页答案表为准；题干与选项为还原版写法（关键数字、条件准确，非原卷逐字照录），解析均为自己的话。第 6、7、8、28、37 题的图上数据已用原卷材料校准落定；第 34 题第 3—8 码元的波形细节仍待终审核对（已在相应步骤中标注），见文末核对结果。

升级版说明：每题在原有 6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）；原有题干、选项、答案与 6 节内容未作改动。

第 1 题 · 单循环链表删除首元素（正确答案：D）

题干与选项：

头指针 h 指向一个带头结点的非空单循环链表，结点结构为数据域 data 加指针域 next，

next 指向直接后继结点；p 是尾指针，指向最后一个数据结点；q 是临时指针。

删除第一个数据元素，正确的语句序列是（ ）。

- A. h->next = h->next->next; q = h->next; free(q);
- B. q = h->next; h->next = h->next->next; free(q);
- C. q = h->next; h->next = q->next; if (p != q) p = h; free(q);
- D. q = h->next; h->next = q->next; if (p == q) p = h; free(q);

入手点： 删结点要分四步走：先把要删的结点地址存下来，再把链表绕过它接好，

再处理只剩一个结点时的尾指针，最后才释放存下来的那个结点。

顺序一乱，不是删错结点，就是留下悬空指针。

题干专有名词从零解释：

- 带头结点：链表最前面有一个不存数据的头结点，头指针 h 指向它，删除首元素时头结点本身不动。
- 单循环链表：每个结点只有一个 next 指针，最后一个数据结点的 next 指回头结点，形成一个环。
- 数据域 data：结点里存数据的地方；指针域 next：结点里存下一个结点地址的地方。
- 尾指针 p：指向最后一个数据结点的指针，删除操作后它必须仍然指向真正的最后一个结点。
- 临时指针 q：临时保存要删除结点地址的指针，防止链表接好之后再也找不到这个结点。
- free(q)：把 q 指向的结点占用的空间释放掉，释放后这个结点就不能再用了。

考查知识点： 单循环链表删除首元素的操作顺序，以及只有一个数据结点时的尾指针处理。

正确选项为什么对： D 的四步各有作用。第一步 q = h->next，把第一个数据结点地址存进 q，

因为第二步一改指针，这个地址就再也找不回来了。第二步 h->next = q->next，

让头结点直接指向第二个数据结点，第一个结点就从链表里脱开了；

如果本来只有一个数据结点，q->next 就是 h，这一步会让 h->next 指回 h，正好表示空表。

第三步 if (p == q) p = h，只在“要删的结点恰好也是尾结点”，也就是只有一个数据结点时，

才把尾指针改到头结点；有多个数据结点时 p 本来就指向最后一个结点，不用动。

第四步 free(q)，释放的正是第一步存下来的第一个结点，不多不少。

错误选项为什么错＋举例：

- A：先改 h->next，再让 q = h->next，此时 q 拿到的是第二个结点，free(q) 删错了结点。

举例：链表是头结点、结点甲、结点乙、结点丙，甲是首元素；A 改完指针后 q 指向乙，释放乙，

甲却从链表里脱开又没被释放，成了找不到也收不回的垃圾结点。

- B：前两步和 D 一样，缺了单结点时的尾指针处理。举例：链表只有一个数据结点甲，

p 本来指向甲；B 删掉甲之后 p 还指向甲的旧地址，p 成了悬空指针，以后顺着 p 找尾结点会出错。

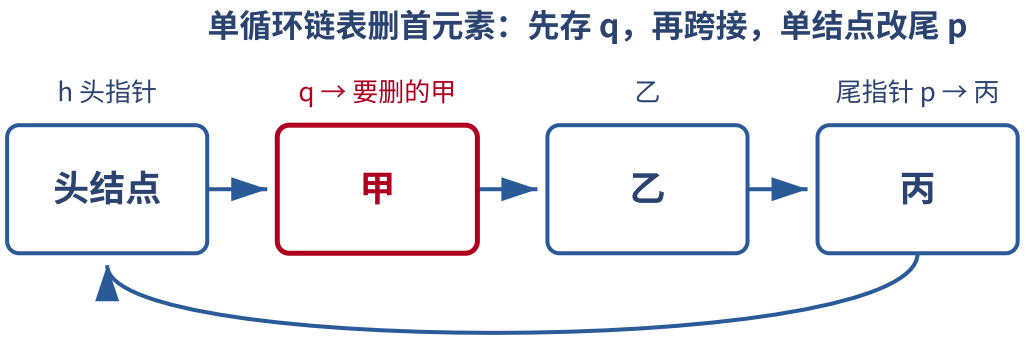
- C：判断条件写反了。举例：链表有甲、乙、丙三个数据结点，p 指向丙，q 指向甲，

p != q 成立，C 会把 p 改成 h，尾指针丢了真正的尾结点丙；

而只有一个结点甲时 p == q，C 反而不改 p，p 又悬空了，两边都错。

笔记本一句话： 第 1 题＋删首元素先存、再跨、单结点改尾、最后释放，条件写反或先改指针都会删错。

【图像】



单循环链表删除首元素四步：先存 q、再跨接、单结点改尾 p、最后释放

【定位】 数据结构 · 第 2 章线性表 · 单循环链表删除首元素与尾指针维护

【易混对比】

- "把链表接好"和"把结点删掉"是两回事：h->next = q->next 只是让链表绕过这个结点，结点占的空间还在，要 free(q) 才算真正释放。
- "先改指针再存地址"和"先存地址再改指针"也是两回事：前者存下来的是第二个结点的地址，释放就删错了；后者存的才是要删的第一个结点。
- p == q 和 p != q 是两种相反的情况：只有"要删的正好也是尾结点"，也就是只剩一个数据结点时，才改 p；条件写反了，多结点时丢尾、单结点时悬空。

释放的必须是第一步存下来的那个结点，不多不少。

【坑位】

删结点先把地址存进临时指针，改完指针旧地址就找不回来了。

最容易踩的坑，是先改 h->next 再让 q = h->next。改完指针之后，h->next 已经指向第二个结点，q 存下来的是第二个结点的地址，free(q) 就把第二个结点释放了；而第一个结点从链表里脱开却没人释放，成了找不到也收不回的垃圾结点。顺序只能是：先存、再改、最后释放。

【套路】

- 第一步，把要删结点的地址存进临时指针 q：q = h->next。
- 第二步，让链表绕过要删的结点：h->next = q->next。
- 第三步，只在"要删的恰好是尾结点"时改尾指针：if (p == q) p = h。
- 第四步，释放第一步存下来的结点：free(q)。

先存、再跨、单结点改尾、最后释放，顺序一乱就删错或悬空。

【类比】

像排队买票要撤掉排第一的人：先记住他的长相（存 q），再让第二个人直接顶到窗口（跨接）；如果队伍里只有他一个人，还得把"队尾"这块牌子摘下来（改 p）；最后才让他离开（free）。要是先让人顶上来再记长相，记住的就变成第二个人，赶错了人。

【口诀】 删首先存再跨接，单结改尾再释放。

【考频】 「尾指针」在 2009–2022 年解析中出现 4 次 · 中频

第 2 题 · 特殊队列不能得到的出队序列（正确答案：D）

题干与选项：

初始为空的队列 Q，一端仅能入队，另一端既能入队又能出队。

入队序列依次为 1、2、3、4、5，则不能得到的出队序列是（）。

- A. 5，4，3，1，2
- B. 5，3，1，2，4
- C. 4，2，1，3，5
- D. 4，1，3，2，5

入手点： 出队只能从一端出，入队却可以从两端进。把队列画成一条横线，

左端只进不出，右端又进又出；出队顺序就是从右端依次取走的顺序，

先想清楚每个元素进的是左端还是右端，再看右端露出的是谁。

题干专有名词从零解释：

- 队列 Q：本题是一种两端开口受限的队列，不是一端进一端出的普通队列。
- 仅能入队的一端：记作左端，元素可以从这里塞进队列，但不能从这里取出。
- 既能入队又能出队的一端：记作右端，元素可以从这里塞进，也可以从这里取出。
- 入队序列：元素到达并进入队列的先后次序，本题固定是 1、2、3、4、5。
- 出队序列：元素从右端被取出的先后次序，出队只能按右端露出的顺序取。
- 不能得到：按规则无论怎么安排入队端和出队时机，都走不出这个顺序。

考查知识点： 一端输入受限、另一端输入输出都允许的队列的出队序列合法性判断。

正确选项为什么对： 本题找的是不可能项。D 的前两个输出是 4、1，第三个是 3，这一步走不通。

先看第二个输出为什么能是 1：1 是最先到达的元素，要让它在 4 之后露出在右端，

2 和 3 必须都排在 1 的左端一侧，而且后到的 3 要比 2 更靠左，

因为从左端塞进来的元素越晚到，位置越靠左；所以 4 出队之后，队列从左到右只能是 3、2、1，

5 如果已经从左端进来，也只能排在更左的位置，不影响右端顺序。

接着取走 1，队列从左到右剩下 3、2，右端露出的是 2，不是 3；

如果这时把 5 从右端塞进来，右端露出的是 5，也不是 3；所以第三个输出不可能是 3，D 走不出来。

错误选项为什么错＋举例： 其余三项都能得到，配走法如下。

- A（5，4，3，1，2 能得到）：1 从右端进，队列是 1；2 从左端进，队列是 2、1；

3 从右端进，队列是 2、1、3；4 从右端进，队列是 2、1、3、4；

5 从右端进，队列是 2、1、3、4、5；再从右端依次取，得到 5、4、3、1、2。

- B（5，3，1，2，4 能得到）：1 从右端进，队列是 1；2 从左端进，队列是 2、1；

3 从右端进，队列是 2、1、3；4 从左端进，队列是 4、2、1、3；

5 从右端进，队列是 4、2、1、3、5；再从右端依次取，得到 5、3、1、2、4。

- C（4，2，1，3，5 能得到）：1 从右端进，队列是 1；2 从右端进，队列是 1、2；

3 从左端进，队列是 3、1、2；4 从右端进，队列是 3、1、2、4；

从右端依次取 4、2、1、3，队列空了，再把 5 从任一端放进并取出，得到 4、2、1、3、5。

笔记本一句话： 第 2 题＋右端出队看右端露出的是谁；要让最先到的 1 露出，后到的 2、3 只能按 3、2 排在它左边。

【图像】



D 的前两个输出 4、1 之后，队列从左到右只剩 3、2、1

右端露出的是 2 → 第三个取不出 3

警告：出队只能取右端露出的那一个，隔着 2 够不着 3

这时把 5 从右端塞进来，露出的还是 5，一样取不出 3

右端出队：队列只剩 3、2、1 时右端露出的是 2，第三个取不出 3

【定位】 数据结构 · 第 3 章栈、队列和数组 · 特殊队列的出队序列判断

【易混对比】

- 左端和右端是两回事：左端只进不出，右端又进又出；出队只能从右端取。
- "能选入队端"和"能改变出队顺序"是两回事：入队时可以选从哪一端进，但出队顺序永远是右端露出的那个先走。
- "最先到达"和"最先出队"是两回事：1 最先到达，但它要先被 2、3 压到左边，才能在 4 之后从右端出来。

出队只看右端露出的是谁，不看谁先到。

【坑位】

想让 1 早出队，就得把 2、3 藏到 1 的左边，而且 3 必须比 2 更靠左。

最容易踩的坑，是觉得"1 最先到，所以 1 出来以后 3 就能出来"。1 要从右端出来，前提是 2、3 都排在 1 左边；从左端塞进来的元素越晚到、位置越靠左，所以 3 在最左、2 在中间、1 在右端。取走 1 之后，右端露出的是 2——3 被 2 挡在里面，轮不到它。

【套路】

- 第一步，把队列画成横线，标出左端只进、右端又进又出。
- 第二步，按目标出队顺序倒推每个元素从哪一端进：要让谁先从右端出来，就把它往右端放；要让谁后出来，就把它藏到左边。
- 第三步，模拟右端取出的过程：每一步只能取右端露出的那个，取完看新露出来的是谁。
- 第四步，走到某一步右端露出的和目标对不上，这个序列就走不出来。

右端露出的和目标对不上一步，整个序列就判死。

【类比】

像一条死胡同只能从右边进出：想先拿走最先进来的东西，得先把后进来的都堆到左边去。东西只能从右口拿，右口露出来的是谁就先拿谁，隔着东西够不着。

【口诀】 右端出队看露头，倒推入端再模拟。

【考频】 「出队序列」在 2009–2022 年解析中出现 29 次 · 高频

第 3 题 · 二维数组行优先地址计算（正确答案：B）

题干与选项：

二维数组 A 按行优先存储，每个元素占 1 个存储单元。

已知 A[0][0] 的地址是 100，A[3][3] 的地址是 220，则 A[5][5] 的地址是（ ）。

- A. 295
- B. 300
- C. 301
- D. 306

入手点： 先用已知的一对地址反推每行有几列，再用同一个公式算目标地址。

行优先的地址公式里只有一个未知数，就是每行的列数。

题干专有名词从零解释：

- 二维数组 A：用两个下标定位元素的表，A[i][j] 表示第 i 行第 j 列的元素，下标从 0 开始数。
- 行优先存储：先把第 0 行全部存完，再存第 1 行，依此类推，同一行里按列号从小到大存。
- 存储单元：每个元素占用的存储空间大小，本题是 1 个单位，地址差就等于元素个数差。
- 地址：元素在存储空间里的位置编号，本题 A[0][0] 的地址 100 是起点。
- 每行列数：记作 C，一行里有 C 个元素；行号为 i、列号为 j 的元素前面共有 i × C + j 个元素。

考查知识点： 行优先存储的二维数组地址公式，以及用已知地址反推列数。

正确选项为什么对： 行优先地址公式是：地址 = 起始地址 + (行号 × 每行列数 + 列号) × 每个元素大小。

本题每个元素占 1 个单位，公式简化为：地址 = 100 + 行号 × C + 列号。

把 A[3][3] 的地址 220 代入：100 + 3 × C + 3 = 220，移项得 3 × C = 220 − 100 − 3 = 117，

两边除以 3，得 C = 39，即每行有 39 列。再算 A[5][5]：100 + 5 × 39 + 5 = 100 + 195 + 5 = 300。

验算一步：5 × 39 = 195，195 + 5 = 200，100 + 200 = 300，结果和 B 一致。

错误选项为什么错＋举例：

- A（295）：按 C = 39 代入应得 300，不是 295。举例：295 对应 100 + 5 × 38 + 5，

等于把每行列数错算成 38，而已知条件算出的是 39，少一列就少 5 个地址。

- C（301）：按 C = 39 代入应得 300，不是 301。举例：301 等于在 300 的基础上多加了 1，

像是把列号 5 多算了 1，或者把起始地址 100 多算了 1，都是计数时多走了一格。

- D（306）：按 C = 39 代入应得 300，不是 306。举例：306 比正确结果多 6，

地址差 206 拆不成 5 × 39 + 5 = 200，说明列数或列号至少有一处代错了。

笔记本一句话： 第 3 题＋行优先先拿已知地址反推列数 C，再用 100 + 行 × C + 列算目标地址。

【图像】

行优先：同一行先存满，再存下一行

	0	1	2	3	4
0	100	101	102	103	104
1	105	106	107	108	109
2	110	111	112	113	114
3	115	116	117	**118**	119

地址 = 起始地址 + (行号 × C + 列号) × 元素大小

100 + 3 × C + 3 = 220 → C = 39（每行 39 列）

A[5][5] = 100 + 5 × 39 + 5 = 300

行优先存储：同一行先存满再存下一行，地址连着往下排

【定位】 数据结构 · 第 3 章栈、队列和数组 · 二维数组行优先地址计算

【易混对比】

- 行优先和列优先是两回事：行优先是同一行先存满，地址公式是"起始 + 行×列数 + 列"；列优先是同一列先存满，公式里的行和列要对调。
- "每行有几列"和"一共有几行"是两回事：公式里乘在行号后面的是列数 C，不是行数；代错了就变成列优先的算法。
- "地址差"和"元素个数差"是两回事：本题每个元素占 1 个单元，两者才相等；如果每个元素占多个单元，地址差还要再乘元素大小。

行优先的公式里，乘在行号后面的是每行的列数。

【坑位】

先拿已知的一对地址反推出列数 C，再去算目标地址，别直接猜。

最容易踩的坑，是看到 A[3][3] 的地址是 220，就想当然地定下列数，或者把行数当成列数代入。本题的公式里只有一个未知数 C：100 + 3×C + 3 = 220，解出来 C = 39，这一步是整题的钥匙。C 代错一位数，后面全错。

【套路】

- 第一步，写出行优先的地址公式：地址 = 起始地址 + (行号 × C + 列号) × 元素大小。
- 第二步，把已知的一对地址代入公式，解出列数 C。
- 第三步，把 C 和目标下标代回公式，算出目标地址。
- 第四步，乘法先算再加，验算一遍有没有多走或少走一格。

先反推列数，再算目标地址，顺序不能反。

【类比】

像电影院按排卖票：先把第 0 排的座位从左到右卖完，再卖第 1 排。要找第 5 排第 5 个座位是第几张票，得先知道每排有几个座位（C），票号 = 100 + 5×C + 5。不知道每排几个座位，就数不出票号。

【口诀】 行优先来行乘列，先反推列数再定位。

【考频】 「行优先」在 2009–2022 年解析中出现 4 次 · 中频

第 4 题 · 森林棵数由先序中序还原（正确答案：C）

题干与选项：

森林 F 对应的二叉树是 T，T 的先序遍历序列是 a、b、d、c、e、g、f，

中序遍历序列是 b、d、a、e、g、c、f，则森林 F 中树的棵数是（）。

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 先用先序加中序把二叉树 T 还原出来，再按“左孩子是第一个孩子、

右孩子是下一个兄弟”的转换规则读森林；森林的棵数就是 T 的根沿右指针走到底的结点个数。

题干专有名词从零解释：

- 森林：由若干棵互不相交的树组成的集合，每棵树都有自己的根。
- 森林对应的二叉树：把森林转成一棵二叉树的方法是：每个结点的左指针指向它在森林里的第一个孩子，

右指针指向它在森林里的下一个兄弟；森林里第一棵树的根就是这棵二叉树的根。

- 先序遍历：访问顺序是根、左子树、右子树，所以先序序列的第一个元素一定是整棵树的根。
- 中序遍历：访问顺序是左子树、根、右子树，所以在中序序列里，根把左子树结点和右子树结点分在两边。
- 兄弟：在森林里同一个双亲的孩子之间互为兄弟，排在后面的兄弟是前面兄弟的“下一个兄弟”。
- 棵数：森林里树的个数，本题要数的是第一棵树之外还有几棵独立的树。

考查知识点： 由先序和中序序列还原二叉树，以及森林与二叉树的转换规则。

正确选项为什么对： 先还原 T。先序第一个是 a，所以 a 是 T 的根；

在中序里 a 左边是 b、d，右边是 e、g、c、f，所以 a 的左子树结点是 b、d，右子树结点是 e、g、c、f。

左子树先序是 b、d，中序是 b、d：根是 b，b 在中序里排在 d 前面且左子树为空，所以 d 是 b 的右孩子。

右子树先序是 c、e、g、f，中序是 e、g、c、f：根是 c，c 左边是 e、g，右边是 f；

左子树先序是 e、g，中序是 e、g：根是 e，g 是 e 的右孩子；右子树只有一个 f，是 c 的右孩子。

于是 T 的结构是：a 的左孩子是 b，b 的右孩子是 d；a 的右孩子是 c，c 的左孩子是 e，

e 的右孩子是 g，c 的右孩子是 f。按转换规则，a 的右指针指向下一个兄弟，

也就是森林里第二棵树的根 c；c 的右指针指向下一个兄弟，也就是第三棵树的根 f；

f 没有右孩子，右链到此为止。所以森林的根依次是 a、c、f，共 3 棵树。

错误选项为什么错＋举例：

- A（1 棵）：只有 a 一棵树时，a 不应有右孩子。举例：还原出的 T 里 a 的右孩子是 c，

按转换规则 c 是另一棵树的根，1 棵的说法直接把 c、f 这条右链忽略了。

- B（2 棵）：只数到 a、c 两棵树，漏了 f。举例：c 的右孩子是 f，按转换规则 f 是 c 的下一个兄弟，

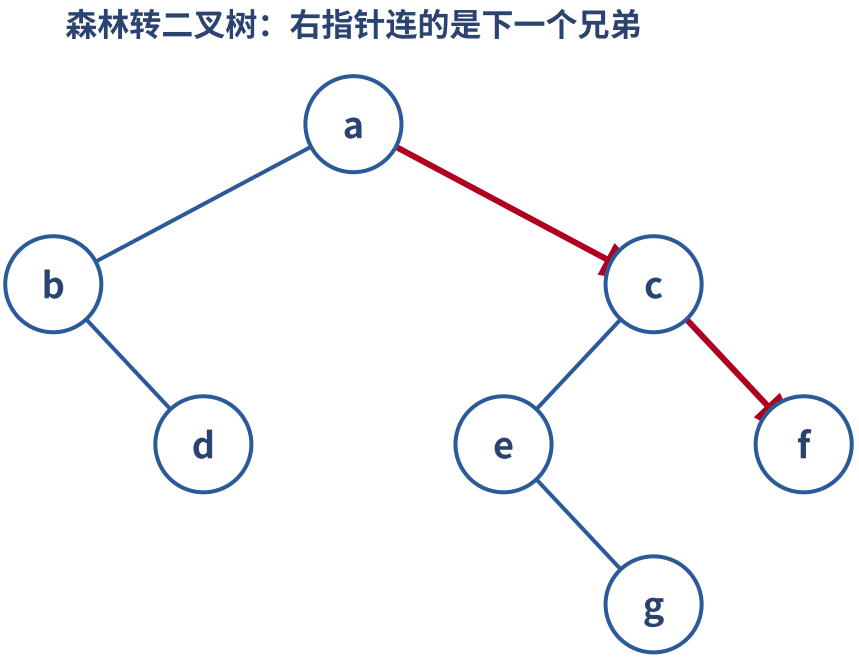
也就是森林里第三棵树的根；数到 c 就停，等于把 f 当成了 c 的孩子。

- D（4 棵）：需要第四个沿右链的根，但右链已经走完。举例：从 a 出发沿右指针走，

依次是 a、c、f，f 的右孩子为空，没有第四个结点能当根；多数的那一棵在序列里根本不存在。

笔记本一句话： 第 4 题＋先序定根、中序分左右还原 T，再沿根的右指针数结点，数到几个就是几棵树。

【图像】



森林转二叉树：沿根的右指针走 a、c、f，3 个结点就是 3 棵树

【定位】 数据结构 · 第 5 章树与二叉树 · 先序中序还原与森林棵数

【易混对比】

- 先序和中序的分工是两回事：先序的第一个元素定根，中序里根的位置把左右子树分开；只给一个序列还原不出唯一的树。
- 左指针和右指针的含义是两回事：左指针连的是"第一个孩子"，右指针连的是"下一个兄弟"——二叉树里的右孩子，在森林里是兄弟不是孩子。
- "还原出二叉树 T"和"数出森林棵数"是两回事：T 只是中间站，棵数要沿 T 的根的右指针数，不是数 T 一共有几个结点。

右指针连的是兄弟，沿右指针数到几个根就是几棵树。

【坑位】

还原出 T 之后别停，棵数要沿根的右指针数，别数 T 的总结点数。

最容易踩的坑，是把还原二叉树当成终点：T 还原出来有 7 个结点，就顺手往结点数上靠。题目问的是森林的棵数，森林的根就是 T 的根沿右指针一路走下来的结点：a、c、f，f 的右孩子是空，链到此为止，一共 3 棵。

【套路】

- 第一步，拿先序第一个元素定根，在中序里找到它，左右两边就是左右子树的结点集合。
- 第二步，对左右子树重复第一步，直到所有结点的位置都定下来，还原出 T。
- 第三步，从 T 的根出发沿右指针一直走，数经过几个结点。

- 第四步，走到右孩子为空就停，数到的个数就是森林的棵数。

先序定根、中序分边，还原完再沿右指针数根。

【类比】

像家族里排兄弟：每个人的左手牵的是自己的第一个孩子，右手牵的是自己的亲弟弟。要数这个家族有几户人家，就从第一户的户主开始，顺着"右手牵弟弟"这条线数：户主、他弟弟、他弟弟的弟弟，数到几个人就是几户。

【口诀】 先序定根中序分，右链数根得棵数。

【考频】 「先序」在 2009–2022 年解析中出现 23 次 · 高频

第 5 题 · 哈夫曼最小带权路径长度（正确答案：B）

题干与选项：

一棵二叉树有 5 个叶结点，权值依次为 10、12、16、21、30，求它的最小带权路径长度。

这里的“最小”是指按哈夫曼方法建树得到的最小值。选项是（）。

- A. 89
- B. 200
- C. 208
- D. 289

入手点： 最小带权路径长度不是把权值加起来，而是按哈夫曼建树：

每次取两个最小的权合并成一个新结点，重复到只剩一个根；

带权路径长度等于每个叶结点的权乘上它到根的边数，再全部相加。

题干专有名词从零解释：

- 叶结点：没有孩子的结点，本题 5 个叶结点就是 5 个要编码的字符对应的结点。
- 权值：叶结点上标的数字，可以理解为这个字符出现的频次，权越大出现越多。
- 路径长度：叶结点到根要经过的边数，也等于叶结点所在的层数减 1，根在第 1 层。
- 带权路径长度 WPL：每个叶结点的权值乘上它的路径长度，全部加起来。
- 哈夫曼建树：每次从还没合并的结点里取权最小的两个，合并成一个新结点，

新结点的权是这两个权之和，再把新结点放回去继续合并，直到只剩一个根。

- 最小带权路径长度：按哈夫曼建树得到的 WPL，任何其他建法得到的 WPL 都不会比它小。

考查知识点： 哈夫曼树的构造过程，以及 WPL 的两种算法：叶结点加权求和与内部结点权求和。

正确选项为什么对： 按哈夫曼方法合并。第一步取最小的 10 和 12，合并成权为 22 的新结点；

剩下 16、21、22、30。第二步取最小的 16 和 21，合并成权为 37 的新结点；剩下 22、30、37。

第三步取最小的 22 和 30，合并成权为 52 的新结点；剩下 37、52。

第四步合并 37 和 52，得到根，根的权是 89。由此得到各叶结点的深度：

16 和 21 在根的 37 那一支，深度是 2；30 在根的 52 那一支，深度是 2；

10 和 12 在 52 的 22 那一支，深度是 3。WPL = 16 × 2 + 21 × 2 + 30 × 2 + 10 × 3 + 12 × 3

= 32 + 42 + 60 + 30 + 36 = 200。验算：内部结点权之和是 22 + 37 + 52 + 89 = 200，结果一致。

错误选项为什么错＋举例：

- A（89）：89 只是 5 个权之和，也是根的权，不是 WPL。举例：10 + 12 + 16 + 21 + 30 = 89，

这个数没有乘深度；按定义每个叶结点都要乘上自己的深度，不乘深度算出来的只是总权。

- C（208）：不是哈夫曼建树算出的 WPL。举例：按上面哈夫曼合并得到的深度逐项相乘，

结果是 32 + 42 + 60 + 30 + 36 = 200；208 比 200 多 8，任何一项深度或权代错都会偏离 200，

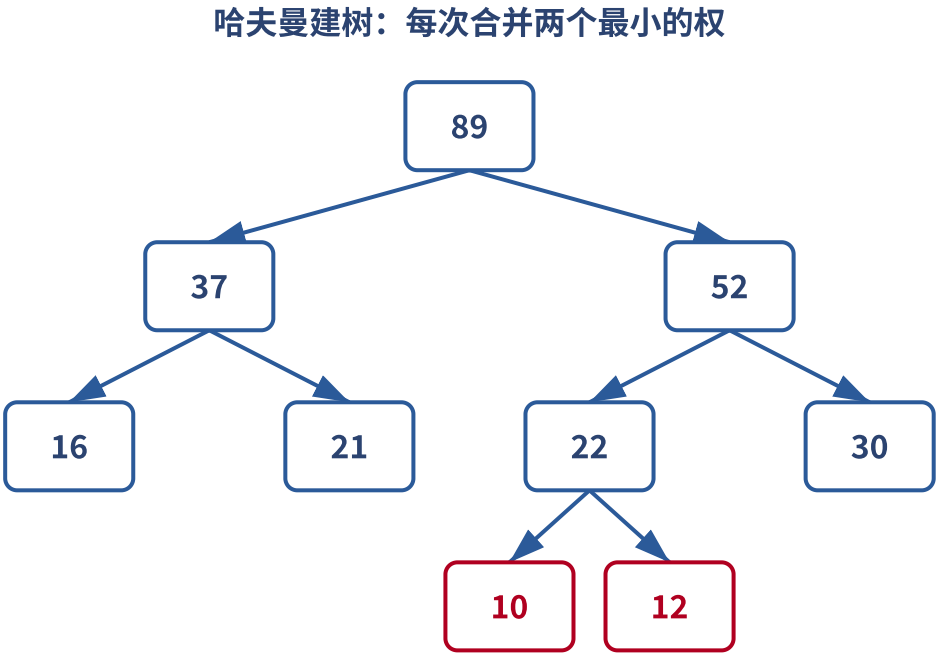
但哈夫曼的合并顺序和深度是唯一确定的，208 对不上这张深度表。

- D（289）：289 等于 200 + 89，是把 WPL 又加了一次根的权。举例：200 是正确的 WPL，

89 是根的权，两者相加等于重复计算了一次总权；WPL 的内部结点求和里根的权只算一次，不能再加一次。

笔记本一句话： 第 5 题＋哈夫曼每次合并两个最小权，WPL 等于叶结点权乘深度求和，也等于内部结点权求和。

【图像】



哈夫曼建树：每次合并两个最小权，WPL = 200

【定位】 数据结构 · 第 5 章树与二叉树 · 哈夫曼树构造与最小带权路径长度

【易混对比】

- 带权路径长度和权值之和是两回事：权值之和 89 只是根的权，WPL 是每个权乘上自己的深度再相加；不乘深度算的是总权，不是 WPL。
- 内部结点求和与叶结点加权求和是两种算法、同一个答案：前者是把每次合并产生的新结点权加起来（22 + 37 + 52 + 89），后者是权×深度逐项加；算出来都是 200，可以互相验算。
- 哈夫曼树的"最小"和"随便建一棵树"是两回事：题目说"最小"就是指定按哈夫曼方法建，别的建法 WPL 只会更大，不能随手排一棵树就算。

WPL 必须每个权都乘上自己的深度，不乘深度算的是总权不是 WPL。

【坑位】

合并完别把根的权 89 当答案，WPL 要乘深度再相加。

最容易踩的坑，是合并到最后看到根的权是 89，就顺手选 89。89 是 5 个权加起来的总权，只是建树过程的副产品。WPL 的定义是每个叶结点的权乘上它到根的边数再全部相加：16×2 + 21×2 + 30×2 + 10×3 + 12×3 = 200。另一个常见坑是 289：那是把 200 又加了一次根的权，等于重复计算了一次总权。

【套路】

- 第一步，每次从还没合并的结点里取两个最小的权合并，新结点权是两者之和，放回去继续。

- 第二步，重复到只剩一个根，标出每个叶结点的深度（到根的边数）。
- 第三步，权乘深度逐项相加，得到 WPL。
- 第四步，用内部结点权求和验算一遍：两次结果一致才算对。

每次合并两个最小的，WPL = 权×深度之和 = 内部结点权之和。

【类比】

像给货物贴运费标签：重的货物（权大）要放在离门口近的货架（深度小），轻的货物可以放远一点，这样总运费（WPL）最小。哈夫曼建树就是每次把最轻的两件货绑在一起往上放；最后算总运费时，每件货的运费 = 重量 × 搬运距离。

【口诀】 两小合并往上堆，权乘深度再相加。

【考频】 「哈夫曼」在 2009–2022 年解析中出现 8 次 · 中频

第 6 题 · 平衡二叉树插入 23 后根关键字（正确答案：D）

题干与选项：

给定平衡二叉树如下图所示：根 20，左孩子 16，右孩子 30；30 的左孩子 25、右孩子 40。

插入关键字 23 后，根中的关键字是（ ）。

- A. 16
- B. 20
- C. 23
- D. 25

入手点： 平衡二叉树的插入分三步：先按二叉排序树的规则找到 23 的插入位置并挂上新叶结点，再从新结点向上逐个检查平衡因子，最后在最低的不平衡结点处做一次旋转；旋转后整棵树的根关键字就是答案。

题干专有名词从零解释：

- 平衡二叉树：又叫 AVL 树，是一棵二叉排序树，并且每个结点的平衡因子只能是 −1、0、1。
- 二叉排序树：左子树所有关键字都小于根关键字，右子树所有关键字都大于根关键字，

插入时拿新关键字和结点关键字逐个比较，小了往左走，大了往右走，走到空位置就挂上去。

- 平衡因子：左子树的高度减去右子树的高度，记作左高减右高；绝对值超过 1 就失衡。
- 高度：从该结点到它最远的叶结点要经过的边数，叶结点的高度是 0，空树的高度记作 −1。
- 旋转：通过局部调整父子关系恢复平衡的操作，LL、RR、LR、RL 是四种旋转类型，

分别对应左左、右右、左右、右左四种失衡形态，旋转后子树高度降回原高度。

- 根中的关键字：整棵树最顶上那个结点里存的关键字，旋转可能把它换掉。

考查知识点： AVL 树的插入过程：按排序树规则插入、向上更新平衡因子、最低不平衡结点处旋转。

正确选项为什么对： 按图代入的步骤如下。

第一步，插入位置：拿 23 和图上结点关键字从根开始逐个比较：23 大于 20 往右走，23 小于 30 往左走，23 小于 25 往左走，走到 25 左边的空位置挂上新结点，搜索路径上的结点关键字为 20、30、25。

第二步，向上检查：从新结点向根逐个重算平衡因子（左高减右高）：23 的平衡因子是 0；25 的左子树高 0、右子树空（记 −1），平衡因子是 1；30 的左子树（25 一支）高 1、右子树（40）高 0，平衡因子是 1；20 的左子树（16）高 0、右子树（30 一支）高 2，平衡因子是 −2。找到最低的绝对值超过 1 的结点是 20，它的失衡形态是右重、且右孩子 30 左重，属于 RL 型。第三步，做一次对应的旋转：

旋转类型为 RL 旋转：先对 30 做一次右旋（25 上提、30 下沉为 25 的右孩子），再对 20 做一次左旋（25 上提为根、20 下沉为 25 的左孩子），旋转只调整失衡结点附近的父子关系，

不改变中序遍历顺序，也就是不改变关键字从小到大的排列。旋转完成后整棵树恢复平衡，

根中的关键字是 25。结论句：本题的正确选项是 D。

错误选项为什么错＋举例：

- A（16）：16 不是旋转后根中的关键字。举例：旋转类型示意（不是本题的图）：

一棵 LL 失衡的树旋转后，中间结点上提当根，原来的根下沉当孩子；

按答案表，本题旋转后上提当根的是 25，不是选项里的 16。

- B（20）：20 也不是旋转后根中的关键字。举例：还是用旋转示意来理解：

旋转后当根的是失衡结点附近按中序排在中间的那个结点，按答案表是 25；

把 20 当成根，等于在没有代入图数据的情况下猜了一个中间结点。

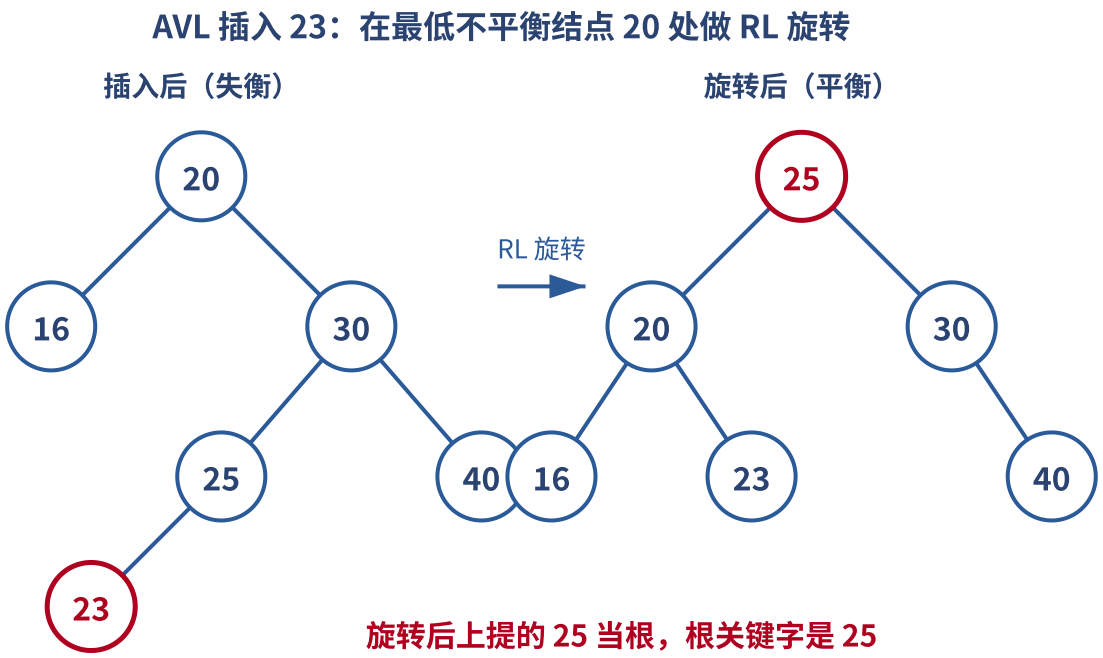
- C（23）：23 是新插入的关键字，插入后它先是叶结点。举例：旋转示意里，

新插的叶结点只有在它恰好是旋转后上提的中间结点时才可能当根；

按答案表，本题旋转后当根的是 25，23 仍在它应该在的子树位置上，不是根。

笔记本一句话： 第 6 题＋AVL 插入先挂叶结点、再向上找最低不平衡结点、做一次旋转，根关键字看旋转后上提的结点。

【图像】



AVL 插入 23 后在最低不平衡结点 20 处做 RL 旋转，根变成 25

【定位】 数据结构 · 第 7 章查找 · 平衡二叉树插入与旋转

【易混对比】

- 二叉排序树的插入和平衡调整是两回事：插入只管按大小找空位挂上去，挂完再从下往上检查平衡因子，失衡才旋转。
- "最低的不平衡结点"和"根"是两回事：旋转在最低的不平衡结点处做，本题是 20 不是 30；在 30 处转解决不了 20 的失衡。
- RL 和 RR 是两回事：RL 是"右孩子的左子树长高了"，要先右旋再左旋；只看"右边高了"就做一次左旋，转完还是歪的。

旋转只在最低的不平衡结点做一次，做完高度就降回来了。

【坑位】

先按排序树规则把 23 挂到 25 左边，再从 23 往上一个一个算平衡因子，别跳过中间的结点。

最容易踩的坑，是凭感觉猜旋转类型：看到右边高了就直接左旋，或者把失衡结点定在 30。从 23 往上算：23 的平衡因子是 0，25 是 1，30 是 1，都没超标；20 的左子树高 0、右子树高 2，平衡因子是 −2，它才是最低的不平衡结点。20 右重、右孩子 30 左重，这是 RL 型：先对 30 右旋，再对 20 左旋，25 上提当根。

【套路】

- 第一步，按二叉排序树规则找到空位挂上新结点，记下从根到新结点的路径。
- 第二步，从新结点往上逐个重算平衡因子（左高减右高），找到绝对值超过 1 的最低的那个结点。
- 第三步，看失衡结点和它的孩子哪边重，定旋转类型：LL、RR、LR、RL 四选一。
- 第四步，做一次对应的旋转，旋转后上提的中间结点就是新根，整棵树恢复平衡。

挂叶、往上找最低失衡点、转一次，根看上提的那个结点。

【类比】

像搭积木塔：新积木按"左边小右边大"的规矩塞进底层空位，塞完从下往上看哪一层歪了。只在最下面的那层歪的地方扶一把（旋转），扶的时候把中间那块积木顶到最上面当新的塔尖。扶完塔就正了，不用从头拆。

【口诀】 先挂叶来再上查，最低失衡转一次。

【考频】 「平衡因子」在 2009–2022 年解析中出现 10 次 · 高频

第 7 题 · 有向图拓扑有序序列个数（正确答案：A）

题干与选项：

给定如下有向图，该图的拓扑有序序列个数是（ ）。

（有向图的 9 条边：A→B；A→F；B→C；B→D；B→F；C→D；D→E；D→F；E→F。）

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 拓扑排序计数不用把所有序列都列出来，只数每一步有几个可选顶点：每一步只能选入度为零的顶点，选一个就删掉它和它出发的边；把每一步的可选个数相乘，就是拓扑有序序列的总数。

题干专有名词从零解释：

- 有向图：边带箭头的图，边从起点指向终点，只能沿箭头方向走，不能逆向走。
- 拓扑有序序列：把所有顶点排成一列，使得每条边的起点都排在终点前面，

也就是所有箭头都从左指向右，这样的排列才叫拓扑有序序列。

- 入度：指向某个顶点的边的条数，入度为零表示没有任何顶点必须排在它前面。
- 出边：从某个顶点出发指向其他顶点的边；删掉一个顶点时，它的所有出边也一起删掉，

因为这些边的起点已经排好，终点的前置约束就解除了一条。

- 分支：某一步有不止一个入度为零的顶点时，选谁先排就有几种走法，分支数就是可选顶点个数。
- 计数相乘：第一步有 a 种选法，第二步有 b 种选法，总走法就是 a × b，依此类推乘到底。

考查知识点： 有向图拓扑排序的计数方法：反复取入度为零的顶点，有分支时分支数相乘。

正确选项为什么对： 按图代入的步骤如下。先算每个顶点的入度：A 的入度是 0（没有边指向它），B 的入度是 1（来自 A），C 的入度是 1（来自 B），D 的入度是 2（来自 B、C），E 的入度是 1（来自 D），F 的入度是 4（来自 A、B、D、E）。

第一步，入度为零的顶点集合是 {A}，只有一个可选顶点，选 A；删掉 A 和它的出边 A→B、A→F 后，B 的入度降为 0，F 的入度降为 3。

第二步，入度为零的顶点集合是 {B}，只有一个可选顶点，选 B；删掉 B 和它的出边 B→C、B→D、B→F 后，C 的入度降为 0，D 的入度降为 1，F 的入度降为 2。

第三步，入度为零的顶点集合是 {C}，只有一个可选顶点，选 C；删掉 C 和它的出边 C→D 后，D 的入度降为 0。

第四步，入度为零的顶点集合是 {D}，只有一个可选顶点，选 D；删掉 D 和它的出边 D→E、D→F 后，E 的入度降为 0，F 的入度降为 1。

第五步，入度为零的顶点集合是 {E}，只有一个可选顶点，选 E；删掉 E 和它的出边 E→F 后，F 的入度降为 0。

第六步，入度为零的顶点集合是 {F}，选 F。六步的可选个数都是 1，把每一步的可选个数相乘：1 × 1 × 1 × 1 × 1 × 1 = 1，

所以拓扑有序序列只有 1 个，答案是 A。

错误选项为什么错＋举例：

- B（2 个）：2 个序列需要恰好出现一次 2 分支，其余步骤都不分支。举例：某一步同时有 2 个入度为零的顶点，

先选其中一个和先选另一个会走出 2 条不同的序列；按上面逐步验算，本图的每一步都只有一个可选顶点，

所以 2 分支从未出现，2 个序列的说法多算了一次分支。

- C（3 个）：3 个序列需要分支数乘积等于 3，例如某一步有 3 个可选顶点。举例：一步有 3 个入度为零的顶点，

先选谁就有 3 种开头；本图每一步都只有一个可选顶点，乘积只能是 1，不可能是 3。

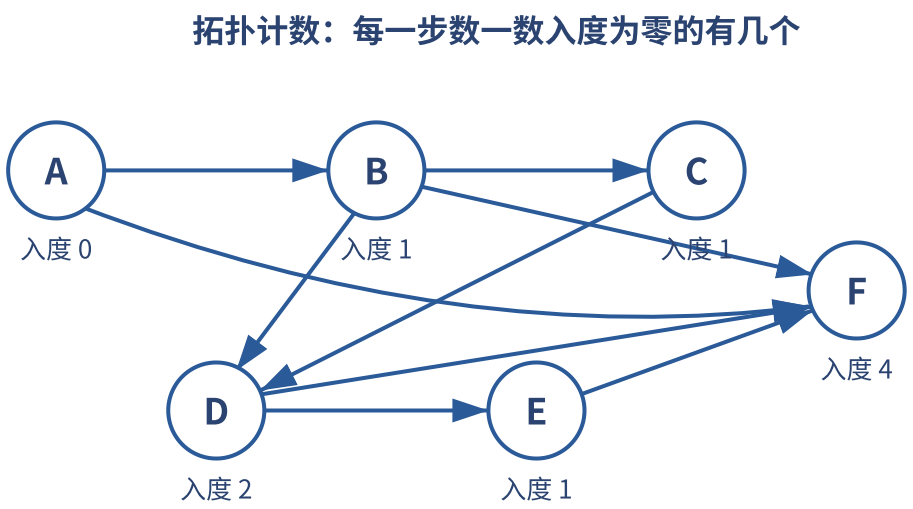
- D（4 个）：4 个序列需要分支数乘积等于 4，例如两步各 2 分支，或者一步 4 分支。举例：

第一步 2 个可选顶点，第二步又出现 2 个可选顶点，2 × 2 = 4；本图没有出现这样的分支组合，

所以 4 个序列的说法是把不存在的分支算了进来。

笔记本一句话： 第 7 题＋拓扑计数每步只看入度为零的顶点有几个，步步都是 1 个就只有 1 个序列，有分支才相乘。

【图像】



有向图拓扑计数：每一步入度为零的都只有 1 个，总数是 1

【定位】 数据结构 · 第 6 章图 · 拓扑排序与序列计数

【易混对比】

- 拓扑有序序列和"随便一种排列"是两回事：只有每条边的起点都排在终点前面才算数，箭头不能从右指回左。
- "某一步有几个可选"和"一共有几个序列"是两回事：前者是单步的分支数，后者是把每一步的分支数乘起来；步步都是 1，乘出来还是 1。
- 入度和出度是两回事：拓扑排序每一步看的是入度为零（没人必须排它前面），不是出度为零。

每一步只能选入度为零的顶点，有几个可选就乘几。

【坑位】

数分支别靠感觉，每一步都把入度为零的顶点集合列出来，选完删边再重数。

最容易踩的坑，是看到图里有 6 个顶点、9 条边，就觉得"分支肯定不少"，随手选 2、3、4。按部就班走：第一步入度为零的只有 A，删 A 及其出边后入度为零的只有 B，再删 B 后只有 C……六步的可选集合都是单元素，没有出现过一次分支，乘积只能是 1。分支是算出来的，不是猜出来的。

【套路】

- 第一步，算出每个顶点的入度。
- 第二步，挑出入度为零的顶点，数有几个：1 个就记 1，有多个就记分支数。
- 第三步，删掉选中的顶点和它出发的所有边，被指向的顶点入度减 1。
- 第四步，重复第二、三步到所有顶点排完，把每一步的个数相乘。

步步列入度为零的集合，删边重数，分支数相乘到底。

【类比】

像排队领号：每轮只有"前面没人挡着"的人能先走。第一轮只有 A 前面没人，A 走后 B 前面没人了，B 走……每一轮都只有一个人能走，整条队伍的顺序就只有一种走法。要是某一轮有两个人前面都没人，那才有两种走法。

【口诀】 入度为零才能排，步步分支乘起来。

【考频】 「拓扑排序」在 2009–2022 年解析中出现 20 次 · 高频

第 8 题 · Dijkstra 求出第二条最短路径后 dist（正确答案：C）

题干与选项：

用 Dijkstra 算法求下图中顶点 1 到其余顶点的最短路径（顶点 1 到 2、3、4、5 的直接边权分别是 26、3、无直接边、6），

当前顶点 1 到顶点 2、3、4、5 的最短路径长度存在 dist 数组里，

求出第二条最短路径后，dist 更新为（ ）。

- A. 26，3，14，6
- B. 25，3，14，6
- C. 21，3，14，6
- D. 15，3，14，6

入手点： Dijkstra 算法是一步定一个顶点：每次从还没确定的顶点里挑 dist 最小的一个确定下来，再用这个刚确定的顶点去松弛它的邻居；“第二条最短路径”就是第二次挑出来的那个顶点对应的路径，更新只看这一次松弛能不能把某个 dist 改小。

题干专有名词从零解释：

- Dijkstra 算法：求一个源点到其他所有顶点最短路径的算法，每次确定一个 dist 最小的未确定顶点。
- 源点：本题是顶点 1，所有最短路径都从顶点 1 出发。
- dist 数组：dist[v] 表示当前已知的从源点到顶点 v 的最短路径长度的候选值，算法结束时它就是真正的最短路径长度；本题 dist 按顶点 2、3、4、5 的顺序排列。
- 已确定集合：已经挑出来、dist 不会再变小的顶点集合；第一条最短路径对应源点自身，距离是 0。
- 第二条最短路径：第二次从未确定顶点里挑出的 dist 最小的顶点对应的路径，

它的 dist 在挑出那一刻就被确定下来，之后不再改变。

- 松弛：对刚确定的顶点 u 的每个邻居 v，比较旧的 dist[v] 和 dist[u] 加上边 u 到 v 的权，

取更小的一个作为新的 dist[v]；如果新值更小就更新，否则保持旧值。

- 边权：一条边上的数字，表示走这条边要付出的代价，本题边权都在原卷图里。

考查知识点： Dijkstra 算法的逐步确定过程，以及用刚确定的顶点松弛邻居的更新规则。

正确选项为什么对： 按图代入的步骤如下。

第一步，初始 dist：顶点 1 到 2、3、4、5 的直接边权分别是 26、3、无直接边、6，

没有直接边的顶点初始 dist 记作无穷大，初始 dist 按顶点 2、3、4、5 的顺序是 26，3，无穷大，6；第一条最短路径是源点 1 自身，距离 0。

第二步，挑第一个确定的顶点：在还没确定的 2、3、4、5 里挑 dist 最小的一个，

是顶点 3（dist 3），挑出后不再改变；用顶点 3 松弛邻居：3 到 2 的边权是 22，3＋22＝25，小于旧的 26，于是顶点 2 的 dist 改成 25；顶点 4 仍无更短路径，保持无穷大；此时 dist 是 25，3，无穷大，6。

第三步，挑第二个确定的顶点：在还没确定的 2、4、5 里挑 dist 最小的一个，

是顶点 5（dist 6），它的最短路径就是第二条最短路径，挑出后不再改变。

第四步，用顶点 5 松弛邻居：5 到 2 的边权是 15，6＋15＝21，小于旧的 25，于是顶点 2 的 dist 改成 21；5 到 4 的边权是 8，6＋8＝14，小于无穷大，于是顶点 4 的 dist 改成 14。

松弛后顶点 2 由 25 改小为 21，顶点 4 由无穷大改小为 14；顶点 3 已在上一步确定、顶点 5 是刚确定的，保持 3、6 不变。所以更新后的 dist 按顶点 2、3、4、5 的顺序是 21，3，14，6，答案是 C。

结论句：本题的正确选项是 C。

错误选项为什么错＋举例：

- A（26，3，14，6）：只有顶点 2 的 dist 和 C 不同。举例：26 等于没有把松弛后的更小值 21 写进去，

像是松弛那一步漏做了，或者把旧的 dist 直接抄了下来；按答案表，松弛后顶点 2 的 dist 应是 21，不是 26。

- B（25，3，14，6）：同样只有顶点 2 的 dist 不同。举例：25 比正确值 21 多 4，

像是松弛时边权多加了 4，或者拿错了一条边的权；按答案表，代入正确的边权后顶点 2 的 dist 是 21。

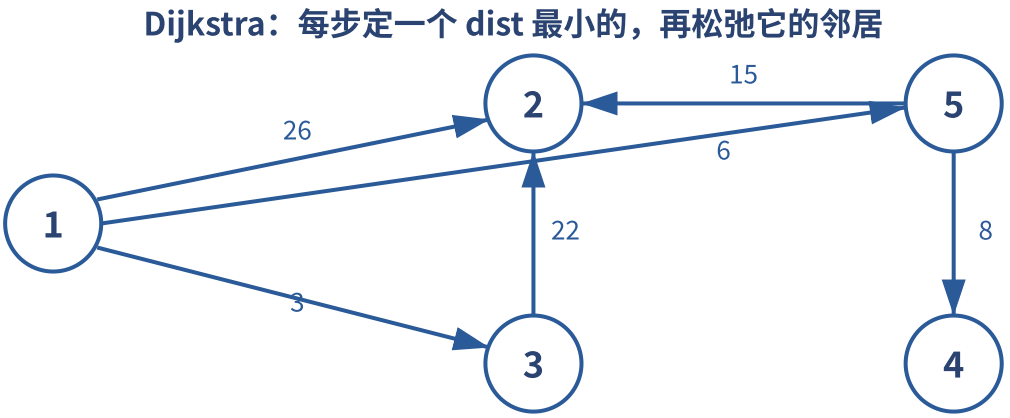
- D（15，3，14，6）：同样只有顶点 2 的 dist 不同。举例：15 比正确值 21 少 6，

像是松弛时边权少算了 6，或者把另一条更短但并不存在的路径算了进来；

按答案表，正确的松弛结果是 21，不是 15。

笔记本一句话： 第 8 题＋Dijkstra 每步定一个 dist 最小的顶点，再用它松弛邻居：新值等于旧 dist 和“刚定顶点 dist 加边权”里更小的那一个。

【图像】



初始 dist（按顶点 2、3、4、5）：26，3，∞，6

定下 3（dist 最小）后松弛：25，3，∞，6

定下 5（第二条最短路径）后松弛：21，3，14，6

Dijkstra 求出第二条最短路径后：定下顶点 5 再松弛，dist 变成 21、3、14、6

【定位】 数据结构 · 第 6 章图 · Dijkstra 算法的逐步确定与松弛

【易混对比】

- "已确定"和"还在候选"是两回事：挑出来的顶点 dist 定死不再改，没挑出来的 dist 还只是候选值，遇到更小的就改。

- "松弛"和"直接抄旧值"是两回事：松弛是拿"刚定顶点的 dist 加边权"和旧 dist 比大小、取小的；直接抄旧值等于这一步白做了。
- "被松弛一次"和"被松弛多次"是两回事：顶点 2 先被 3 松弛到 25，又被 5 松弛到 21；只记得第一次就停在 25。

没挑出来的顶点 dist 还是候选值，每次松弛都要重新比大小。

【坑位】

顶点 2 会被松弛两次：先被 3 改成 25，再被 5 改成 21，少做一次就错。

最容易踩的坑，是做完"定下 3、松弛邻居"这一步，看到顶点 2 的 dist 从 26 变成 25，就以为它定下来了。顶点 2 还没被挑出来，它的 25 只是候选值。接着挑出顶点 5（dist 6），用 5 松弛：6 + 15 = 21，比 25 小，顶点 2 的 dist 还要再改小一次。漏掉这第二次松弛，就会停在 25 或者干脆抄最初的 26。

【套路】

- 第一步，写出初始 dist：有直接边就写边权，没有就写无穷大；源点自身距离 0。
- 第二步，从没确定的顶点里挑 dist 最小的一个定下来。
- 第三步，用刚定的顶点松弛它的每个邻居：新值 = 刚定顶点的 dist + 边权，比旧值小就更新。
- 第四步，重复第二、三步；每次更新只看这一次松弛，别提前把没挑中的顶点当定死的。

每步定一个最小的，再用它松弛邻居：新值和旧值比，小的留下。

【类比】

像从公司出发去几个客户家拜访，已知直达的打车费（初始 dist）。每确定一家"目前已知最便宜能到的"（挑最小），就问：从这家再打车去别家，会不会比之前记的更便宜（松弛）？便宜就改记。去老王家（顶点 2）本来记 26，经老张家（顶点 3）中转变成 25，经老李家（顶点 5）中转变成 21——账本没定稿前，每次有更便宜的路线都要改。

【口诀】 步步定个最小值，松弛邻居比大小。

【考频】 「松弛」在 2009–2022 年解析中出现 7 次 · 中频

第 9 题 · 高度为 3 的 3 阶 B 树最多结点数（正确答案：A）

题干与选项：

一棵高度为 3 的 3 阶 B 树，根为第 1 层，若第 2 层共有 4 个关键字，

则这棵树的结点个数最多是（）。

- A. 11
- B. 10
- C. 9
- D. 8

入手点： 结点个数最多，就要让第 2 层结点个数尽可能多，同时让每个第 2 层结点带出的第 3 层孩子也尽可能多；

孩子个数不是固定 3 个，而是“关键字个数加 1”，所以第 2 层的关键字怎么分，直接决定第 3 层能有多少结点。

题干专有名词从零解释：

- B 树：一种多路平衡查找树，每个结点里可以存多个关键字，孩子个数比关键字个数多 1。
- 3 阶 B 树：每个结点最多有 3 个孩子，最多有 2 个关键字；除根以外的每个结点至少有 2 个孩子，

至少有 1 个关键字；根至少有 1 个关键字，根如果不是叶结点，至少有 2 个孩子。

- 高度为 3：根为第 1 层，树一共有 3 层，第 3 层是叶结点所在的层。
- 关键字：结点里存的、用于查找和排序的数值；第 2 层共有 4 个关键字，是指第 2 层所有结点里的关键字加起来是 4 个。
- 孩子个数：一个结点有 k 个关键字，就有 k + 1 个孩子，因为 k 个关键字把数值范围切成 k + 1 段，

每一段对应一个孩子子树。

- 结点个数：整棵树里结点的总数，本题是第 1 层、第 2 层、第 3 层的结点数相加。

考查知识点： 3 阶 B 树的结点关键字上下限，以及“孩子数等于关键字数加 1”对结点总数的约束。

正确选项为什么对： 第 1 层只有根，结点数是 1。第 2 层：根最多有 3 个孩子，

所以第 2 层最多有 3 个结点；第 2 层共有 4 个关键字，要分给最多 3 个结点，

每个结点至少 1 个关键字、最多 2 个关键字，分法只能是 2、1、1，

即一个结点有 2 个关键字，另外两个结点各有 1 个关键字。第 3 层结点数等于第 2 层各结点孩子数之和：

关键字为 2 的结点有 2 + 1 = 3 个孩子，关键字为 1 的结点各有 1 + 1 = 2 个孩子，

第 3 层最多有 3 + 2 + 2 = 7 个结点。三层相加：1 + 3 + 7 = 11，所以最多是 11 个结点。

对照一下另一种分法：第 2 层只有 2 个结点时关键字分法是 2、2，

第 3 层只有 3 + 3 = 6 个结点，总数是 1 + 2 + 6 = 9，比 11 少，说明 2、1、1 的分法确实更优。

错误选项为什么错＋举例：

- B（10）：总数 10 对应第 3 层只有 6 个结点的一种组合。举例：第 2 层 3 个结点、

第 3 层 6 个结点的组合是 1 + 3 + 6 = 10，但第 2 层关键字 2、1、1 对应的孩子数是 3、2、2，

加起来是 7 不是 6，10 是把其中一个结点的孩子数少算了 1。

- C（9）：总数 9 是第 2 层只有 2 个结点时的结果，不是最大值。举例：第 2 层 2 个结点、

关键字分法 2、2，第 3 层 3 + 3 = 6，总数 1 + 2 + 6 = 9；题目问的是“最多”，

第 2 层能排 3 个结点时就不能只排 2 个，9 只是其中一种分法，不是最大值。

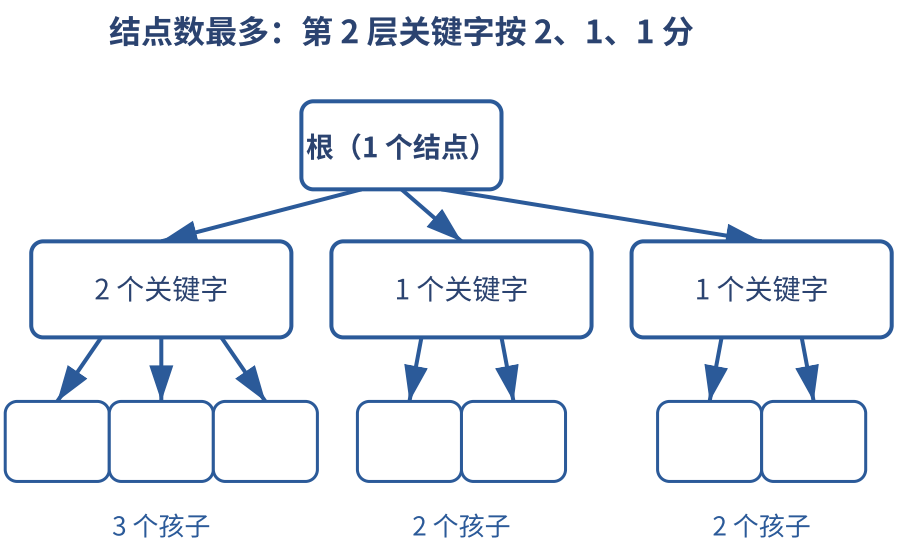
- D（8）：总数 8 比 9 还小，连第 2 层 2 个结点的情况都不到。举例：1 + 2 + 5 = 8，

第 3 层只有 5 个结点，但第 2 层关键字 2、2 对应的孩子数已经是 3 + 3 = 6，

8 是把第 3 层孩子数算少了，最大值不可能是它。

笔记本一句话： 第 9 题＋B 树孩子数等于关键字数加 1；第 2 层关键字 4 个分给 3 个结点按 2、1、1 分，第 3 层得 7 个，总数 11。

【图像】



3 阶 B 树高度 3：结点数最多时第 2 层按 2、1、1 分关键字，第 3 层 7 个结点，总数 11

【定位】 数据结构 · 第 7 章查找 · B 树的结点关键字上下限与结点数

【易混对比】

- "关键字个数"和"孩子个数"是两回事：有 k 个关键字就有 k + 1 个孩子，孩子永远比关键字多 1；数孩子时别直接数关键字。
- "第 2 层关键字总数"和"第 2 层结点数"是两回事：总数 4 个是题目给死的，结点数是要凑的；把 4 个关键字摊到 3 个结点上（2、1、1），孩子总数才最大。
- "结点数最多"和"关键字数最多"是两回事：关键字总数固定时，结点数最多要求每个结点关键字尽量少，但不能少于下限 1 个。

孩子数 = 关键字数 + 1；关键字摊得越散，孩子总数越多。

【坑位】

第 2 层能放 3 个结点就别只放 2 个，问的是"最多"不是"某一种"。

最容易踩的坑，是第 2 层只排 2 个结点、关键字按 2、2 分，算出总数 1 + 2 + 6 = 9 就停了。根最多有 3 个孩子，第 2 层最多能排 3 个结点；4 个关键字分给 3 个结点只能是 2、1、1，对应孩子 3、2、2，第 3 层是 7 个，总数 1

+ 3 + 7 = 11。9 只是其中一种分法，不是最大值。

【套路】

- 第一步，先定每一层最多能有几个结点：根 1 个，第 2 层不超过根的孩子数上限。
- 第二步，把该层的关键字总数摊到尽量多的结点上，每个结点不低于下限（除根外至少 1 个关键字）。
- 第三步，每个结点的孩子数 = 关键字数 + 1，加起来就是下一层的结点数。
- 第四步，各层相加；换一种分法对照一下，确认手里的是最大值。

关键字摊得越散孩子越多，层层按上限排就是最大值。

【类比】

像分蛋糕：4 块蛋糕（关键字）分给小朋友（结点），每个小朋友至少拿 1 块、最多拿 2 块，每个小朋友还要带走"块数 + 1"个盘子（孩子）。想让盘子总数最多，就让拿蛋糕的小朋友尽量多：3 个小朋友按 2、1、1 分，盘子是 3 + 2 + 2 = 7 个；只叫 2 个小朋友来，只能拿走 6 个盘子。

【口诀】 关键字加一是孩子，摊散分法孩子多。

【考频】 「B树」在 2009–2022 年解析中出现 27 次 · 高频

第 10 题 · LSD 基数排序第一趟后 372 的邻居（正确答案：C）

题干与选项：

数组 S[] = {93，946，372，9，146，151，301，485，236，327，43，892}，

用最低位优先（LSD）基数排序按升序排序。第 1 趟分配、收集后，

元素 372 之前、之后紧邻的元素分别是（）。

- A. 43，892
- B. 236，301
- C. 301，892
- D. 485，301

入手点： LSD 第一趟只看个位数：把所有数按个位数字分进 0 到 9 号桶，同一个桶里保持原来的先后次序，再按桶号从小到大依次倒出来；372 的邻居就看它在个位是 2 的桶里排第几个，以及前后两个桶的边界元素是谁。

题干专有名词从零解释：

- 基数排序：按数字的每一位分桶、收集，重复多趟后排好序的排序方法，关键字是数字本身。
- 最低位优先 LSD：先排最低位，也就是先排个位，再排十位、百位，依此类推；

本题只问第 1 趟，所以只排个位。

- 分配：按个位数字把每个元素放进对应编号的桶，个位是几就进几号桶。
- 收集：按桶号从小到大，把各桶里的元素依次倒出来排成新序列，桶号小的在前。
- 稳定：同一个桶里的元素保持进入桶之前的先后次序，不打乱；基数排序的每一趟都要求稳定，

否则低位排好的次序会被高位的一趟破坏。

- 紧邻：收集后的新序列里，某个元素前面一个元素和后面一个元素。

考查知识点： LSD 基数排序第一趟的分配与收集过程，以及稳定性对同桶元素次序的影响。

正确选项为什么对： 先算每个数的个位：93 是 3，946 是 6，372 是 2，9 是 9，

146 是 6，151 是 1，301 是 1，485 是 5，236 是 6，327 是 7，43 是 3，892 是 2。

按个位分桶，保持原顺序：0 号桶空；1 号桶是 151、301；2 号桶是 372、892；

3 号桶是 93、43；4 号桶空；5 号桶是 485；6 号桶是 946、146、236；

7 号桶是 327；8 号桶空；9 号桶是 9。按桶号从小到大收集，得到新序列：

151、301、372、892、93、43、485、946、146、236、327、9。

在这个序列里，372 前面一个是 301，后面一个是 892，所以答案是 C。注意 372 前面是 1 号桶的最后一个元素 301，不是 2 号桶内部的其他元素，因为 2 号桶里 372 排在第一个；372 后面是同一个桶的下一个元素 892。

错误选项为什么错＋举例：

- A（43，892）：后面一个 892 对了，前面一个 43 错了。举例：43 在 3 号桶，

3 号桶在 2 号桶之后收集，不可能排在 372 前面；372 前面是 1 号桶的尾巴 301，

43 要到 372、892 之后才出现。

- B（236，301）：前后都错了。举例：236 在 6 号桶，301 在 1 号桶；

按桶号收集，301 在 372 前面，236 在 372 后面很远的位置，两者都不紧挨着 372，

把桶号顺序颠倒了才会得到这个答案。

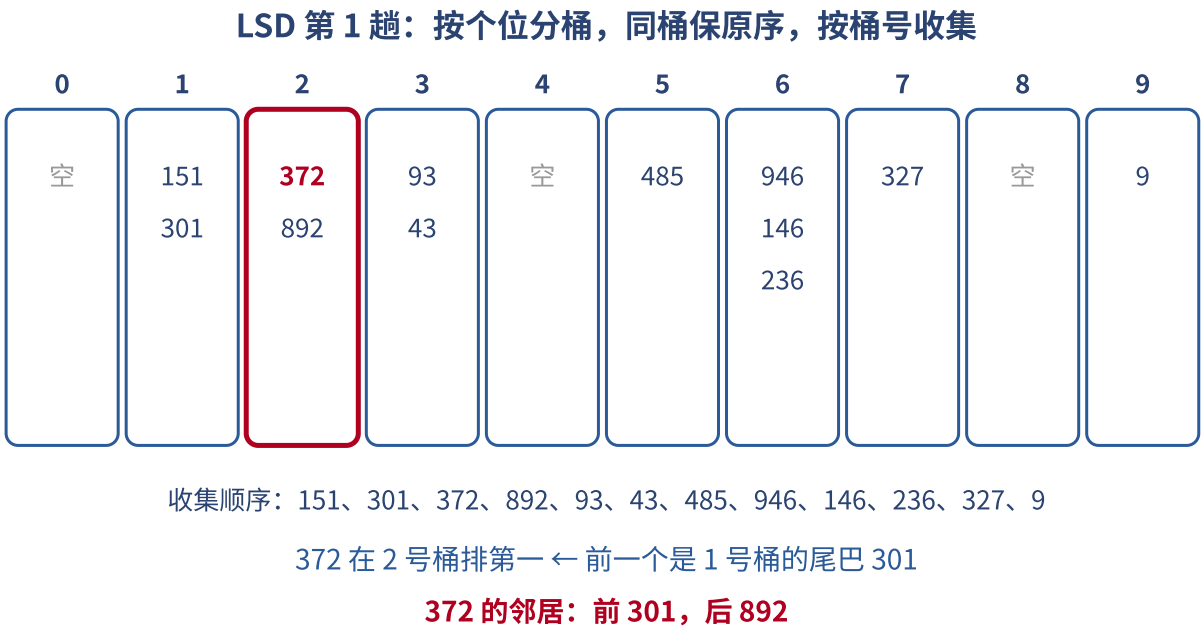
- D（485，301）：前后顺序颠倒了。举例：485 在 5 号桶，301 在 1 号桶；

收集时 1 号桶先倒出，5 号桶后倒出，所以 301 在 372 前面、485 在 372 后面隔着好几个元素，

D 把 485 放到前面、301 放到后面，正好说反了。

笔记本一句话： 第 10 题＋LSD 第一趟只看个位分桶，同桶保原序、按桶号收集；372 在 2 号桶排第一，前面是 1 号桶尾 301，后面是同桶 892。

【图像】



LSD 第 1 趟按个位分桶：372 在 2 号桶排第一，邻居是 301 和 892

【定位】 数据结构 · 第 8 章排序 · 基数排序第一趟分配与收集

【易混对比】

- LSD 和 MSD 是两回事：LSD 从个位开始排，MSD 从最高位开始排；本题只问第 1 趟，LSD 的第 1 趟只看个位。
- "分配"和"收集"是两回事：分配是按个位把数分进 0–9 号桶，收集是按桶号从小到大倒出来；邻居关系看的是收集后的序列。
- "同桶内的顺序"和"桶之间的顺序"是两回事：同桶内保持原来的先后次序（稳定），桶之间按桶号排；372 的前一个是上一个桶的尾巴 301，不是同桶的 892。

同桶保原序，桶间按桶号；邻居要看收集后的序列。

【坑位】

372 在 2 号桶里排第一个，它的前一个是 1 号桶的尾巴 301，别在 2 号桶里找。

最容易踩的坑有两个：一是把 43 当成 372 的前一个——43 在 3 号桶，3 号桶在 2 号桶之后收集，不可能排在 372 前面；二是把 485 放前面——那是把桶号顺序弄反了。2 号桶里只有 372、892（保持原序），372 排第一，所以前面是 1 号桶最后一个 301，后面是同桶的 892。

【套路】

- 第一步，算出每个数的个位数字。
- 第二步，按个位分进 0–9 号桶，同桶保持原顺序。
- 第三步，按桶号从小到大依次倒出，写出收集后的序列。
- 第四步，在收集后的序列里找目标元素的前一个和后一个；跨桶的邻居看相邻桶的头和尾。

分桶看个位，同桶保原序，按桶号收集后数邻居。

【类比】

像按尾号分快递柜：只看手机号最后一位，尾号几就进几号柜，同一个柜子里先到的放前面。取件时按柜号从小到大开柜。372 的尾号是 2，进了 2 号柜且是第一个，所以它前面是 1 号柜最后一个包裹 301，后面是 2 号柜第二个包裹 892。

【口诀】 个位分桶保原序，按号收集数邻居。

【考频】 「基数排序」在 2009–2022 年解析中出现 9 次 · 中频

第 11 题 · 大根堆依次插入的结果（正确答案：B）

题干与选项：

关键字 6、9、1、5、8、4、7 依次插入初始为空的大根堆 H，得到的 H 是（）。

这里的 H 按堆的数组顺序（也就是从上到下、从左到右的层序）写出。

- A. 9，8，7，6，5，4，1
- B. 9，8，7，5，6，1，4
- C. 9，8，7，5，6，4，1
- D. 9，6，7，5，8，4，1

入手点： 大根堆插入只做一件事：新元素先挂在完全二叉树的下一个空位置，再和它的双亲比较，大了就上浮交换，小了就停下；每插一个都这样走一遍，最后的数组顺序就是答案，不要凭感觉把所有数从大到小排一遍。

题干专有名词从零解释：

- 大根堆：一种完全二叉树，堆顶是最大值，每个双亲结点的值都不小于它的孩子结点的值。
- 完全二叉树：除最后一层外，每一层都排满，最后一层的结点从左到右依次排，没有空缺。
- 数组顺序：把完全二叉树按层序写进数组，从第 1 个位置开始数，第 i 个位置的双亲在第 i ÷ 2 取整的位置，

第 i 个位置的孩子在第 2 × i 和第 2 × i + 1 的位置；本题按这个顺序比较选项。

- 插入：新元素先放在数组末尾，也就是完全二叉树下一个空位置上。
- 上浮：新元素和它的双亲比较，如果新元素更大，就和双亲交换位置，再继续和新的双亲比较，

直到双亲更大或者到达堆顶为止。

- 初始为空：堆一开始没有任何元素，第一个插入的元素直接成为堆顶。

考查知识点： 大根堆的插入算法：末尾插入后向上调整，以及堆的数组表示。

正确选项为什么对： 按顺序逐步插入，数组位置从 1 开始数。

插入 6：堆是 6。插入 9：先挂在位置 2 得到 6、9，双亲位置 1 是 6，9 更大，交换后是 9、6。

插入 1：挂在位置 3 得到 9、6、1，双亲位置 1 是 9，1 更小，停下。

插入 5：挂在位置 4 得到 9、6、1、5，双亲位置 2 是 6，5 更小，停下。

插入 8：挂在位置 5 得到 9、6、1、5、8，双亲位置 2 是 6，8 更大，交换后是 9、8、1、5、6；

新的双亲位置 1 是 9，8 更小，停下。插入 4：挂在位置 6 得到 9、8、1、5、6、4，

双亲位置 3 是 1，4 更大，交换后是 9、8、4、5、6、1；新的双亲位置 1 是 9，4 更小，停下。

插入 7：挂在位置 7 得到 9、8、4、5、6、1、7，双亲位置 3 是 4，7 更大，交换后是 9、8、7、5、6、1、4；

新的双亲位置 1 是 9，7 更小，停下。最终数组顺序是 9，8，7，5，6，1，4，答案是 B。

验算大根堆性质：9 的孩子是 8、7，8 的孩子是 5、6，7 的孩子是 1、4，双亲都不小于孩子。

错误选项为什么错＋举例：

- A（9，8，7，6，5，4，1）：第 4 个位置写成了 6，第 5 个位置写成了 5。举例：插入 8 那一步，

8 挂在位置 5 后和位置 2 的 6 交换，位置 5 变成 6，位置 4 仍是 5；A 把 5、6 的位置互换了，

等于把插入 8 后的交换记错了方向。

- C（9，8，7，5，6，4，1）：最后两个位置写成了 4、1。举例：插入 7 那一步，

7 挂在位置 7 后和位置 3 的 4 交换，位置 3 变成 7，位置 7 变成 4，位置 6 仍是 1；

最终顺序是位置 6 为 1、位置 7 为 4，C 把这两个位置写反了。

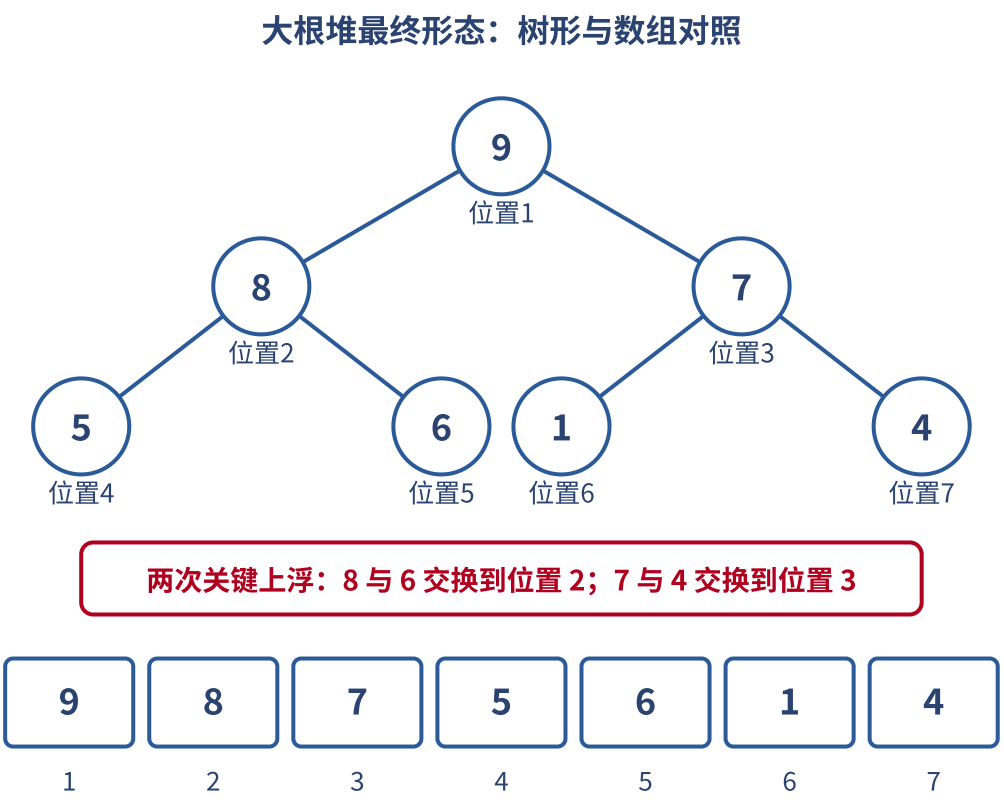
- D（9，6，7，5，8，4，1）：第 2 个位置是 6、第 5 个位置是 8，违反大根堆性质。举例：

位置 2 的双亲是位置 1 的 9 没问题，但位置 2 是 6、位置 5 是 8，孩子 8 大于双亲 6，

说明插入 8 后没有做上浮交换；按正确步骤，8 必须和 6 交换，位置 2 应是 8。

笔记本一句话： 第 11 题＋大根堆插入先挂末尾再上浮：和双亲比，大了交换、到了堆顶或双亲更大才停。

【图像】



大根堆插入完成后的树形与数组对照

【定位】 数据结构 · 第 5 章树与二叉树 · 大根堆的插入与数组表示

【易混对比】

- 插入新元素"先挂末尾再上浮"，和"把所有数从大到小排一遍"是两回事：前者每次只动新元素这一条上浮路，后者会把整棵树重排，中间过程完全不同。
- "双亲不小于孩子"（堆性质）和"层序从左到右递减"是两回事：堆只要求每个双亲不小于它的孩子，不要求同一层从左到右递减；全递减的写法只是巧合，不是插入过程的必然结果。
- "上浮停下"和"上浮到堆顶"是两回事：新元素只要遇到比它更大的双亲就停下，只有比一路上的双亲都大才会一路换到堆顶。

插入只走新元素这一条上浮路，别把整棵树重排一遍。

【坑位】

插入时别凭感觉从大到小排一遍，一定要按"挂末尾、比双亲、大了换"的步骤一步一步走。

最容易踩的坑，是看到"大根堆"三个字就想"堆顶最大、往下越来越小"，于是把 7 个数从大到小排成 9，8，7，6，5，4，1 直接选。但堆插入是增量过程：每个数来的时候只和自己的双亲比、只走自己的上浮路。插入 8 那一步，8 只和位置 2 的 6 交换，不会把位置 4 的 5 也挪动；插入 7 那一步，7 只和位置 3 的 4 交换。凭感觉重排会把 5、6 的位置写反、把 1、4 的位置写反，正好掉进错误选项的陷阱里。

【套路】

- 第一步，把新元素挂到数组末尾，也就是完全二叉树下一个空位置。
- 第二步，拿它和双亲（位置 i÷2 取整）比较：大了就交换，小了就停下。
- 第三步，交换后继续和新的双亲比较，直到双亲更大或者到达堆顶。
- 第四步，每插一个数都走完这三步，最后的数组顺序直接对照选项。

大了就换、换完继续比，停下的条件是双亲更大或到堆顶。

【类比】

像公司空降一个新人，先让他坐在最后排的空位上，再和他的直属上司比业绩：比上司强就和上司换座位，再和新的上司比；不如上司就坐下不动。每次只换一条线，不会把全公司座位重新排一遍。

【口诀】 插入先挂末尾处，比爹大了就上浮。

【考频】 「大根堆」在 2009–2022 年解析中出现 32 次 · 高频

第 12 题 · PFLOPS 换算成每秒浮点操作次数（正确答案：D）

题干与选项：

2017 年超级计算机 TOP500 排名中，我国“神威·太湖之光”蝉联第一，其浮点运算速度为 93.0146 PFLOPS。该计算机每秒钟执行的浮点操作次数是（ ）。

- A. 9.3×10^13 次
- B. 9.3×10^15 次
- C. 9.3 千万亿次
- D. 9.3 亿亿次

入手点： 先把 P 这个数量级单位换成 10 的乘方，再把 93.0146 的小数点挪到 9.3 的位置，最后把中文大数单位也换成 10 的乘方，两边对上号再选。

题干专有名词从零解释：

- 浮点操作：对带小数点的数做一次加、减、乘、除这类运算，算一次就是一个浮点操作。
- FLOPS：每秒浮点操作次数，衡量一台计算机算浮点数有多快，数字越大算得越快。
- P（Peta）：数量级前缀，1 P = 10^15，也就是 1 后面跟 15 个 0。
- 93.0146 PFLOPS：每秒 93.0146×10^15 次浮点操作。
- 亿：10^8；亿亿：10^8×10^8 = 10^16；千万亿：10^7×10^8 = 10^15。

考查知识点： 数量级单位换算、科学记数法、中文大数单位换算。

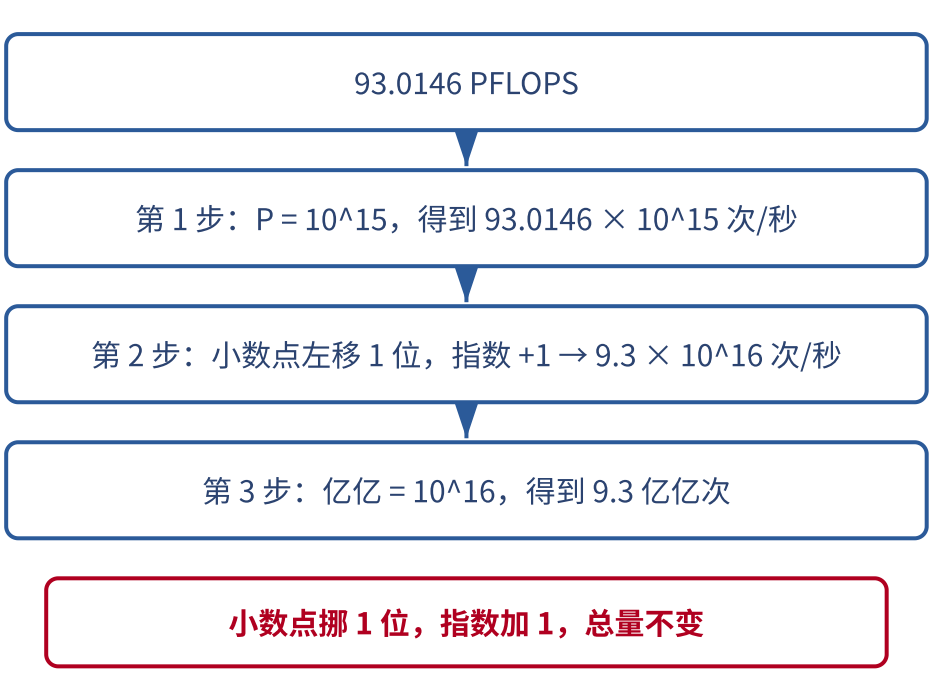
正确选项为什么对： 从 PFLOPS 的定义出发：93.0146 PFLOPS = 93.0146×10^15 次/秒。把 93.0146 写成科学记数法：93.0146 = 9.30146×10，把 10 的这一次方和 10^15 合并，指数相加：1 + 15 = 16，得到 9.30146×10^16 次/秒，约等于 9.3×10^16 次/秒。再看中文单位：亿亿 = 10^16，所以 9.3 亿亿次 = 9.3×10^16 次，和上面算出的结果对上了，D 正确。

错误选项为什么错＋举例：

- A（9.3×10^13 次）：比正确结果小了 1000 倍。举例：把 1 P 误当成 10^12，就会算出 93.0146×10^12 = 9.3×10^13，数量级前缀记错一位，答案就差 1000 倍。
- B（9.3×10^15 次）：比正确结果小了 10 倍。举例：93.0146×10^15 只挪了小数点却忘了补上挪出来的那一个 10，相当于把 93.0146 直接约成 9.3 而没有乘 10，少算了 10 倍。
- C（9.3 千万亿次）：千万亿 = 10^15，所以 9.3 千万亿次 = 9.3×10^15 次，和 B 是同一个数，只是换了中文写法，同样少算了 10 倍。举例：93 个“千万亿”才是 93×10^15，约成 9.3 之后单位要升一级变成“亿亿”，不能还停在“千万亿”。

笔记本一句话： 第 12 题＋P = 10^15，93.0146 PFLOPS = 9.3×10^16 次/秒 = 9.3 亿亿次，小数点挪一位就要给指数加 1。

【图像】



PFLOPS 到每秒浮点操作次数的换算流程

【定位】 计算机组成原理 · 第 1 章计算机系统概述 · 计算机性能指标与数量级单位换算

【易混对比】

- P（Peta）是 10^{15}，和 T（Tera）的 10^{12}、G（Giga）的 10^{9} 是三档不同的数量级：前缀差一档，结果就差 1000 倍。
- "93.0146 约成 9.3"和"93.0146×10^15 化成 9.3×10^16"是两回事：前者是直接丢掉小数凑整，后者是把小数点左移一位、同时给指数加 1，总量不变。
- "千万亿"（10^{15}）和"亿亿"（10^{16}）是两回事：中文大数单位差一级就是 10 倍，9.3 后面跟哪个单位，答案就差 10 倍。

小数点挪一位，指数就要加 1，总量不能变。

【坑位】

把 93.0146 直接约成 9.3 时，别忘了补上挪小数点腾出来的那个 10。

最容易踩的坑是两步只做一步：把 P 换成 10^{15} 对了，93.0146 约成 9.3 也对了，但约掉的那个 10 没有加到指数上，于是写出 9.3×10^{15}。科学记数法要求小数点前只留一位，挪出去的每一位都要变成指数上的 +1：93.0146 变成 9.30146 是小数点左移一位，指数就从 15 变成 16。同样，中文单位也要跟着升一级：9.3 后面跟的是"亿亿"不是"千万亿"，否则又会掉进另一个错误选项的陷阱。

【套路】

- 第一步，把数量级前缀换成 10 的乘方：P 对应 10^{15}。
- 第二步，把系数化成科学记数法：小数点移到第一位非零数字后面，挪几位指数加几。
- 第三步，把中文大数单位也换成 10 的乘方：亿是 10^{8}，亿亿是 10^{16}，千万亿是 10^{15}。

- 第四步，两边都化成"系数×10 的乘方"再对号选。

前缀、系数、中文单位三路都要换成 10 的乘方再比较。

【类比】

像把"93 米"说成"9.3 丈"：数字变小了，单位就要变大，总长度不能变。小数点左移一位，指数（单位）就要升一级，这是同一杆秤的两头。

【口诀】 P 是十的十五方，点左移位指数涨。

【考频】 「PFLOPS」在 2009–2022 年解析中出现 7 次 · 中频

第 13 题 · 补码机器数的带符号与无符号比较（正确答案：D）

题干与选项：

带符号整数用补码表示。x、y、z 的机器数分别为 FFFDH、FFDFH、7FFCH。下列叙述中正确的是（）。

- A. 若 x、y、z 为无符号数，则 z < x < y
- B. 若 x、y、z 为无符号数，则 x < y < z
- C. 若 x、y、z 为带符号数，则 x < y < z
- D. 若 x、y、z 为带符号数，则 y < x < z

入手点： 同一串二进制，戴不戴符号帽子意思完全不同：先分两种帽子各算一遍真值，再排大小，不要拿无符号的大小去套带符号的结论。

题干专有名词从零解释：

- 机器数：存在计算机里的二进制编码，本题是 16 位，每位是一个十六进制数字对应 4 位二进制。
- 补码：带符号整数的一种编码，正数的补码就是它本身，负数的补码是“按位取反再加 1”，最高位是符号位，0 表示正数，1 表示负数。
- 无符号数：所有位都表示数值大小，没有符号位，数值就是二进制本身的大小。
- 十六进制数：逢 16 进 1 的记数法，一位十六进制数字对应 4 位二进制，F = 15，D = 13，C = 12。
- 求负数真值的小办法：负数的补码取“补数”，即用 10000H 减去该补码，差就是负数绝对值的大小。

考查知识点： 补码的符号位判断、负数补码求真值、无符号数与带符号数的大小比较。

正确选项为什么对： 按带符号数算。z = 7FFCH，最高位是 7，也就是二进制 0111，最高位是 0，所以 z 是正数，真值按位展开：7×4096 + 15×256 + 15×16 + 12 = 28672 + 3840 + 240 + 12 = 32764。x = FFFDH，最高位是 1，是负数，用 10000H − FFFDH = 0003H = 3，所以 x = −3。y = FFDFH，最高位是 1，是负数，用 10000H − FFDFH = 0021H，0021H = 2×16 + 1 = 33，所以 y = −33。三个真值排队：−33 < −3 < 32764，也就是 y < x < z，D 正确。

错误选项为什么错＋举例：

- A（无符号时 z < x < y）：无符号时直接比十六进制大小：7FFCH 以 7 开头，FFDFH 和 FFFDH 以 F 开头，所以 z 最小，这半句是对的；但 FFDFH 和 FFFDH 比到第三位：D = 13，F = 15，13 小于 15，所以 FFDFH 小于 FFFDH，也就是 y 小于 x，A 写成 x 小于 y，把后两位排反了。
- B（无符号时 x < y < z）：按无符号数算，z = 7FFCH = 32764 是最小的，根本不可能排在最大；x 和 y 都是以 F 开头的五万多的数，B 把最小的 z 放到了最大，把最大的两个放到了前面，顺序全反了。
- C（带符号时 x < y < z）：带符号时 x = −3，y = −33，−33 比 −3 更小，所以应该是 y 在前、x 在后；C 把 x 和 y 的位置写反了，错在负数越“绝对值大”反而越小这一点上。举例：−33 元欠债比 −3 元欠债更少，−33 排在 −3 前面。

笔记本一句话： 第 13 题＋同一机器数两种读法：无符号直接比大小，带符号先看最高位，负数用 10000H 减补码求绝对值，负得越多反而越小。

【图像】

同一机器数，两种读法

机器数	无符号读法	带符号读法
FFFDH	无符号：65533	带符号：−3
FFDFH	无符号：65501	带符号：−33
7FFCH	无符号：32764	带符号：+32764

无符号排序：z(32764) ＜ y(65501) ＜ x(65533)

带符号排序：y(−33) ＜ x(−3) ＜ z(32764)

同一机器数带符号与无符号两种读法的对照

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 补码的带符号数与无符号数比较

【易混对比】

- 无符号读法和带符号读法是两回事：无符号时所有位都算数值、直接比十六进制大小；带符号时最高位是符号位，先定正负再算真值。
- 负数"绝对值大"和"数值大"是反的：补码里 −33 比 −3 小，绝对值越大、数值反而越小，和无符号时"数越大值越大"正好相反。
- "用 10000H 减补码"是求负数绝对值的小办法，和"按位取反加 1 求真值"算的是同一个东西，只是前者直接给绝对值、更适合比大小。

同一串机器数，戴不戴符号帽子，大小关系可能完全反过来。

【坑位】

带符号比较时，先看最高位定正负，千万别拿无符号的大小顺序直接套。

最容易踩的坑是两种读法混用：FFDFH 和 FFFDH 无符号时确实是 FFDFH 更小，但带符号时两者都是负数，FFDFH 的绝对值是 33、FFFDH 的绝对值是 3，绝对值大的反而更小，顺序就反过来了。错误选项里就有一个把无符号的顺序直接搬到带符号里，错在负数"欠得越多反而越少"这一点上。记住：无符号直接比十六进制，带符号先定正负、负数用 10000H 减补码求绝对值再比。

【套路】

- 第一步，看清题目戴的是哪顶帽子：无符号还是带符号。
- 第二步，无符号就直接比十六进制大小，从高位往低位逐位比。
- 第三步，带符号就先看最高位定正负：0 是正数直接算值，1 是负数用 10000H 减补码求绝对值。
- 第四步，按真值排大小：正数一定大于负数，负数之间绝对值大的反而小。

正数永远大于负数，负数比的是"谁欠得少谁大"。

【类比】

像看银行账单：无符号读法是只看"交易金额"那一栏，数字大就是大；带符号读法还要看"借/贷"方向，借方（欠钱）的数字越大反而越穷。同一张账单，戴不戴"方向"这顶帽子，结论完全不同。

【口诀】 无符直接比大小，带符先看最高位，负数补数求真值，欠多反而更小。

【考频】 「补码」在 2009–2022 年解析中出现 19 次 · 高频

第 14 题 · 不能用 IEEE 754 精确表示的数（正确答案：A）

题干与选项：

下列数值中，不能用 IEEE 754 浮点格式精确表示的是（）。

- A. 1.2
- B. 1.25
- C. 2.0
- D. 2.5

入手点： 本题问的是“不能”，要找那个小数部分化成二进制会没完没了的数：小数部分反复乘 2 取整，能在有限步内归零的就能精确表示，绕回老路循环的就不能。

题干专有名词从零解释：

- IEEE 754 浮点格式：计算机表示带小数点的数的标准格式，用有限位二进制存一个数，位数是有限的。
- 精确表示：用有限位二进制把一个数一分不差地写出来，不需要四舍五入。
- 二进制小数：小数点后第 1 位是 1/2，第 2 位是 1/4，第 3 位是 1/8，每往后一位，分母就多乘一个 2。
- 十进制小数转二进制：小数部分反复乘 2，取整数部分作二进制位，剩下的小数部分继续乘 2，直到小数部分变成 0 为止。
- 有限二进制小数：小数部分乘 2 若干步后能归零的数；无限循环二进制小数：乘 2 的过程绕回以前出现过的小数、永远归不了零的数。

考查知识点： 十进制小数转二进制、有限二进制小数与无限循环二进制小数的区分、IEEE 754 精确表示的条件。

正确选项为什么对： A 的 1.2，整数部分 1 就是二进制 1，小数部分 0.2 转二进制：0.2×2 = 0.4，取 0，剩 0.4；0.4×2 = 0.8，取 0，剩 0.8；0.8×2 = 1.6，取 1，剩 0.6；0.6×2 = 1.2，取 1，剩 0.2。剩下的 0.2 正是出发时的小数，从这里起，后面的步骤会把 0.4、0.8、0.6、0.2 再走一遍，无限循环下去，永远归不了零。所以 1.2 的二进制小数部分是无限循环的 0.00110011……，IEEE 754 只有有限位，存不下无限循环的尾巴，只能截断或舍入，1.2 不能被精确表示。

错误选项为什么错＋举例：

其余三项都能精确表示，下面逐项给出二进制写法。

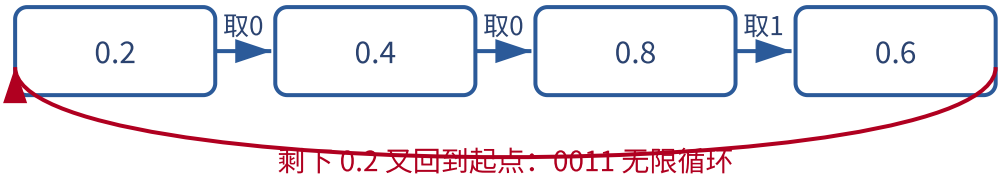
- B（1.25）：小数部分 0.25 转二进制：0.25×2 = 0.5，取 0，剩 0.5；0.5×2 = 1.0，取 1，剩 0.0，两步就归零了，所以 $1.25 = 1.01_2$，1 是整数部分，01 是小数部分 1/4 + 0/8，有限位就能写完，可以精确表示。
- C（2.0）：整数 $2 = 10_2$，小数部分是 0，不需要任何小数位，$2.0 = 10.0_2$，有限位写完，可以精确表示。
- D（2.5）：小数部分 0.5 转二进制：0.5×2 = 1.0，取 1，剩 0.0，一步归零，所以 $2.5 = 10.1_2$，整数 10，小数 1 表示 1/2，有限位写完，可以精确表示。

笔记本一句话： 第 14 题＋小数乘 2 取整能归零就能精确表示，绕回老路循环的就不能，1.2 的 0.2 部分会无限循环 0011。

【图像】

小数部分乘 2：能归零 vs 绕回老路

0.2：绕回老路（不能精确表示）



0.25：两步归零（能精确表示）



0.25 = 0.01（2 进制），两步归零 → 1.25 能精确表示

1.2 的小数部分 0.2 无限循环 → 不能精确表示

0.2 乘2无限循环与 0.25 乘2归零的对照

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · IEEE 754 浮点数的精确表示条件

【易混对比】

- "十进制小数位数有限"和"二进制小数位数有限"是两回事：0.2 十进制只有一位小数，但转成二进制是无限循环的 0.00110011……，十进制看着整齐不代表二进制也整齐。
- "能归零"和"绕回老路"是两种结局：小数部分反复乘 2，能在有限步内变成 0 的就能精确表示；中途绕回以前出现过的小数、永远到不了 0 的就不能。
- "不能精确表示"和"不能表示"是两回事：1.2 在 IEEE 754 里照样能存，只是存的是截断或舍入后的近似值，不是一分不差的精确值。

十进制看着整齐的数，转成二进制可能没完没了。

【坑位】

判断能不能精确表示，只看小数部分乘 2 取整能不能归零，别被"十进制只有一位小数"骗了。

最容易踩的坑是觉得 1.2、2.5 都是"一位小数"，要么都能精确表示、要么都不能，于是乱选。其实小数部分 0.5 乘 2 一步就归零（2.5 可以），0.25 乘 2 两步归零（1.25 可以），只有 0.2 会走出 0.2→0.4→0.8→0.6→0.2 的循环、永远归不了零。规律是：分数约分后分母如果只有因子 2，二进制小数就有限；0.2 = 1/5，分母里有个 5，注定无限循环。做题时老老实实乘 2 走几步，看到绕回老路就判"不能"。

【套路】

- 第一步，把整数部分和小数部分拆开，整数部分转二进制总没问题。
- 第二步，小数部分反复乘 2 取整，取出的整数位就是二进制小数位。
- 第三步，盯着剩下的小数部分：变成 0 就判"能精确表示"；绕回以前出现过的值就判"不能"。
- 第四步，问"不能"就选那个绕回老路的。

乘 2 能归零就能精确表示，绕回老路就不能。

【类比】

像把 1 米平均分给 3 个人：十进制写 0.333……没完没了，不是因为米这个单位不好，是因为 3 这个分母在十进制里除不尽。0.2 在二进制里也一样：分母 5 在二进制里除不尽，所以 0.00110011……没完没了，有限位的 IEEE 754 格子装不下。

【口诀】 小数乘二看归零，绕回老路表不清。

【考频】 「IEEE 754」在 2009–2022 年解析中出现 7 次 · 中频

第 15 题 · 按字编址的存储器所需 RAM 芯片数（正确答案：C）

题干与选项：

某存储器有 24 位地址线、32 位数据线，按字编址，字长为 32 位。其中 000000H～3FFFFFH 的地址范围为 RAM 区。组成该 RAM 区需要 512K×8 位的 RAM 芯片的数量是（ ）。

- A. 8 片
- B. 16 片
- C. 32 片
- D. 64 片

入手点： 芯片数分两路算再相乘：一路是深度够不够，一路是宽度够不够，深度差几倍就并几排，宽度差几倍就并几列，两路相乘才是总数。

题干专有名词从零解释：

- 地址线：传送地址信号的线路，24 位地址线表示地址有 24 位二进制位。
- 数据线：传送数据信号的线路，32 位数据线表示一次能传 32 位数据。
- 按字编址：一个地址对应一个字，而不是一个字节；本题字长 32 位，所以一个地址对应 32 位数据。
- RAM 区地址范围 000000H～3FFFFFH：从 0 号地址到 3FFFFFH 号地址，首尾两个地址都要算在内。
- 芯片规格 512K×8 位：每片芯片有 512K 个地址单元，每个单元 8 位；K = 1024，所以 512K = 512×1024 = 524288 个单元。
- 深度扩展：多片芯片排成多排，地址数不够时用排数补；位扩展：多片芯片并排拼宽度，位数不够时用并排数补。

考查知识点： 存储器容量计算、地址范围求单元数、存储器扩展（深度扩展与位扩展）。

正确选项为什么对： 先算 RAM 区有多少个字：末地址 3FFFFFH 减首地址 000000H，再加 1（首尾都算），得到 400000H。400000H 换成十进制：4×16^5 = 4×1048576 = 4194304，也就是 4M 个字，每个字 32 位。再算深度一路：每片芯片有 512K 个单元，RAM 区需要 4M 个地址，4M ÷ 512K = 4194304 ÷ 524288 = 8，需要 8 排才够深度。再算宽度一路：一个字是 32 位，每片芯片一个单元只出 8 位，32 ÷ 8 = 4，需要 4 片并排才够宽度。两路相乘：8×4 = 32 片，C 正确。验算总位数：32 片×512K×8 位 = 32×4194304 位 = 134217728 位，除以 32 位每字，得到 4194304 个字，正好是 4M 个字，对上了。

错误选项为什么错＋举例：

- A（8 片）：只算了深度一路，忘了宽度一路。举例：8 片排成 8 排，深度够了，但每排只有 8 位宽，一个 32 位的字还差 24 位，读一次只能读出字的四分之一。
- B（16 片）：深度和宽度各算了一半的典型错法。举例：8 排×2 列 = 16 片，宽度只拼到 16 位，离 32 位还差一半；或者 4 排×4 列，深度只有 2M 个字，离 4M 个字还差一半。
- D（64 片）：多乘了一倍。举例：把 24 位地址线误当成 RAM 区地址位数，用 24 位去算地址数，或者把深度一路算成 16，都会多出一倍；RAM 区实际只有 400000H 个地址，不是整个 24 位地址空间。

笔记本一句话： 第 15 题＋芯片数 =（需要地址数 ÷ 每片单元数）×（字长 ÷ 每片位数），深度一路和宽度一路都要乘，地址范围首尾都算要加 1。

【图像】



深度 8 排乘宽度 4 列的芯片阵列

【定位】 计算机组成原理 · 第 3 章存储器层次结构 · 存储器扩展的深度扩展与位扩展

【易混对比】

- 深度扩展和位扩展补的是两样东西：深度扩展补的是"地址够不够"（排数），位扩展补的是"一次读出的数据够不够宽"（列数），两路都要乘，缺一路总数就错。
- "24 位地址线"和"RAM 区地址范围"是两回事：24 位是整台机器的地址线，RAM 区只占其中 000000H～3FFFFFH 这一段，算芯片数只能用 RAM 区的实际地址数，不能用 24 位算。
- "按字编址"和"按字节编址"算出的单元数不同：按字编址一个地址对应一个 32 位的字，地址数就是字数；换成按字节编址，同样地址范围的单元数会完全不同。

芯片数是深度路和宽度路相乘，不是只算一路。

【坑位】

算完深度别急着选，宽度那一路也要乘，地址范围首尾都算还要加 1。

最容易踩的坑有两个：一是只算深度 4M÷512K=8 就停手，忘了每片一个单元只出 8 位、32 位的字还要 4 片并排拼宽度；二是地址数算成 3FFFFFH−0=3FFFFFH，忘了首尾两个地址都要算、必须加 1 才是 400000H。记住公式：芯片数 =（需要地址数 ÷ 每片单元数）×（字长 ÷ 每片位数），两路相乘才是总数。

【套路】

- 第一步，算 RAM 区有多少个地址：末地址 − 首地址 + 1，注意按字编址时地址数就是字数。
- 第二步，算深度路：需要地址数 ÷ 每片单元数，得到排数。
- 第三步，算宽度路：字长 ÷ 每片位数，得到列数。
- 第四步，排数 × 列数，再用总位数验算一遍。

深度补地址数，宽度补数据位，两路相乘再验算。

【类比】

像用瓷砖铺一个大厅：深度路是"大厅有多长、需要铺几排"，宽度路是"大厅有多宽、一排要并几块砖"。只算排数不算列数，铺出来的地面要么长度够了宽度露缝，要么反过来。

【口诀】 深度排数宽度列，两路相乘芯片齐。

【考频】 「位扩展」在 2009–2022 年解析中出现 5 次 · 中频

第 16 题 · 直接映射回写 Cache 的行位数（正确答案：A）

题干与选项：

某计算机主存地址为 32 位，按字节编址。Cache 数据区容量为 32 KB，主存块大小为 32 B，采用直接映射方式，回写策略。则 Cache 一行所占的位数至少是（）。

- A. 275 位
- B. 274 位
- C. 258 位
- D. 257 位

入手点： 一行的位数不是只有数据：先把主存地址切成标记、索引、块内偏移三段，再把数据位、标记位、有效位、脏位四笔账加起来，缺了哪一笔都会少算。

题干专有名词从零解释：

- 按字节编址：一个地址对应一个字节，主存地址的每一位变化都对应字节位置的变化。
- Cache 数据区：Cache 里真正存放主存块数据副本的部分，本题是 32 KB。
- 主存块：主存按固定大小分块传送的单位，本题一块是 32 B，Cache 一行装一块。
- 直接映射：每个主存块只能进 Cache 里固定的一行，行号由地址里的索引段决定。
- 块内偏移：地址里指出块内第几个字节的字段，块有 32 个字节，需要区分 32 种位置。
- 索引：地址里指出 Cache 哪一行的字段，Cache 有多少行就需要多少位来区分。
- 标记：地址里剩下的高位，存进 Cache 行里，用来比对这一行装的到底是哪个主存块。
- 有效位：1 位，标记这一行里的数据是不是有效的，1 表示有效，0 表示空行。
- 脏位：回写策略下才有的 1 位，标记这一行的数据有没有被 CPU 改过，改过为 1，换出时要写回主存，没改过为 0，换出时不用写回。
- 回写策略：CPU 只改 Cache，改过的数据不立刻写主存，等到该行被换出时才一次性写回。

考查知识点： 直接映射 Cache 的地址划分、行结构（数据位＋标记位＋有效位＋脏位）、回写策略对脏位的要求。

正确选项为什么对： 一步一步算。Cache 行数 = 数据区容量 ÷ 块大小 = 32 KB ÷ 32 B = 32768 ÷ 32 = 1024 行。块内偏移位数：块内有 32 个字节，2 的 5 次方等于 32，所以需要 5 位。索引位数：Cache 有 1024 行，2 的 10 次方等于 1024，所以需要 10 位。标记位数：主存地址一共 32 位，减去索引 10 位，再减去块内偏移 5 位，得到 32 − 10 − 5 = 17 位。一行四笔账：数据位 = 32 B×8 位每字节 = 256 位；标记位 17 位；有效位 1 位；脏位 1 位，因为本题是回写策略，改过的数据要靠脏位记住，换出时才知道要不要写回。四笔相加：256 + 17 + 1 + 1 = 275 位，A 正确。

错误选项为什么错＋举例：

- B（274 位）：漏了脏位。举例：256 + 17 + 1 = 274，只算了数据、标记、有效三笔；回写策略下没有脏位，换出行时就分不清这一行改没改过，改过的数据可能丢掉不写回，等于白改。
- C（258 位）：漏了标记位。举例：256 + 1 + 1 = 258，数据、有效、脏三笔都在，唯独没有标记；没有标记，CPU 就没法比对这一行装的是哪个主存块，直接映射的比对就无从谈起。
- D（257 位）：只算了数据位和有效位。举例：256 + 1 = 257，既没有标记位来比对主存块，也没有脏位来记录修改，回写策略和直接映射都运转不起来。

笔记本一句话： 第 16 题＋Cache 行位数 = 数据位 + 标记位 + 有效位 + 脏位（回写才有脏位），标记 = 地址总位数 − 索引位数 − 块内偏移位数。

【图像】

32 位地址切三段，一行记四笔账



标记 = 32 − 10 − 5 = 17 位；行数 = 32KB ÷ 32B = 1024 → 索引 10 位；块 32B → 偏移 5 位



256 + 17 + 1 + 1　　**= 275 位**

回写策略：脏位不能少，换出时靠它判断要不要写回主存

直写策略才不需要脏位；有效位任何策略都要

32 位地址三段划分与 Cache 行四笔账

【定位】 计算机组成原理 · 第 3 章存储器层次结构 · 直接映射 Cache 的行结构

【易混对比】

- 标记位和索引位分工不同：索引位决定"这个主存块只能进 Cache 的哪一行"，标记位存进 Cache 行里用来比对"这一行装的到底是哪个主存块"，一个管定位、一个管比对。
- 有效位和脏位各记一件事：有效位记"这一行有没有装有效数据"，脏位记"这一行的数据被 CPU 改没改过"；回写策略下脏位不能少，直写策略下才不需要脏位。
- "Cache 数据区 32 KB"和"Cache 一行 275 位"是两回事：32 KB 只是数据部分，一行的总位数还要加上标记、有效、脏这些"账本位"。

一行的位数是四笔账相加，数据位只是其中最大的一笔。

【坑位】

回写策略一定要加上脏位那 1 位，漏了它就分不清换出时要不要写回。

最容易踩的坑是两种：一是看到"回写"没反应过来，漏掉脏位算出 274；二是只记得"数据+标记"，把有效位和脏位两个 1 位小账全漏掉。回写策略下 CPU 只改 Cache 不立刻写主存，换出行时全靠脏位判断"改过才写回、没改过直接丢"；没有脏位，改过的数据可能被悄悄丢掉，等于白改。记住四笔账：数据位 256 + 标记位 17 + 有效位 1 + 脏位 1 = 275。

【套路】

- 第一步，把主存地址切成三段：块内偏移 = 块大小取 log2，索引 = Cache 行数取 log2，标记 = 总位数减去前两段。
- 第二步，算 Cache 行数：数据区容量 ÷ 块大小。

- 第三步，把一行的四笔账加起来：数据位 + 标记位 + 有效位 1 位 + 脏位（回写才有）1 位。
- 第四步，对照选项，重点检查脏位有没有漏。

先切地址三段，再加四笔账，回写必有脏位。

【类比】

像图书馆的书架格子：每格能放 32 本书（数据位），格子外贴着标签写"这格放的是哪套书"（标记位），标签旁有个小牌"已占用/空"（有效位），回写策略下再加个小牌"书被涂改过、还回去时要登记"（脏位）。数一格占多少地方，四样都要算。

【口诀】 数据标记加两旗，回写脏位不能丢。

【考频】 「直接映射」在 2009–2022 年解析中出现 7 次 · 中频

第 17 题 · 汇编语言程序员可见的存储器（正确答案：D）

题干与选项：

下列存储器中，汇编语言程序员可见的是（）。

Ⅰ. 指令寄存器

Ⅱ. 微指令寄存器

Ⅲ. 基址寄存器

Ⅳ. 标志状态寄存器

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅰ、Ⅳ
- C. 仅Ⅱ、Ⅳ
- D. 仅Ⅲ、Ⅳ

入手点： 判断“可见”只看一条线：汇编程序能不能直接点名用它、或者直接读到它留下的结果；CPU 内部自己周转用的寄存器，程序点不了名，就是不可见。

题干专有名词从零解释：

- 汇编语言程序员：用汇编指令写程序的人，程序里能点名用的寄存器、能靠条件跳转读到的标志，才算对他可见。
- 指令寄存器：CPU 取出当前正在执行的那条指令后，暂存这条指令的内部寄存器，指令译码就靠它。
- 微指令寄存器：微程序控制的 CPU 内部寄存器，暂存当前正在执行的那条微指令，控制信号就从它发出。
- 基址寄存器：程序员能直接用的寄存器，存一段地址的起始地址，寻址时和偏移量相加得到真正要访问的地址。
- 标志状态寄存器：存运算结果状态的寄存器，里面有进位标志、零标志、溢出标志、符号标志等位，条件跳转指令就靠读这些标志位来决定跳不跳。
- 可见与不可见：可见不是“存在于 CPU 里”，而是“程序员在汇编程序里能直接操作或直接感知”；内部周转用的寄存器即使真实存在，程序员也点不了名，就是不可见。

考查知识点： 程序员可见寄存器与 CPU 内部寄存器的区分、基址寄存器与标志寄存器的用途。

正确选项为什么对： Ⅲ 基址寄存器可见：汇编程序里可以直接写基址寄存器的名字，把它当作寻址的基地址用。举例：把数组首地址装进基址寄存器，再加偏移量就能按顺序访问数组里的各个元素，这个用法是汇编程序员天天用的寻址手段。Ⅳ 标志状态寄存器可见：汇编程序虽然不直接给标志寄存器里的每一位起名字读写，但条件跳转指令直接读它的结果。举例：两数相减后结果为零，零标志位变成 1，接下来一条“结果为零则跳转”指令就会跳转，程序员正是靠标志位来控制程序走向的。Ⅲ 和 Ⅳ 都符合“程序员能直接用、能直接感知”的标准，所以是仅Ⅲ、Ⅳ，D 正确。

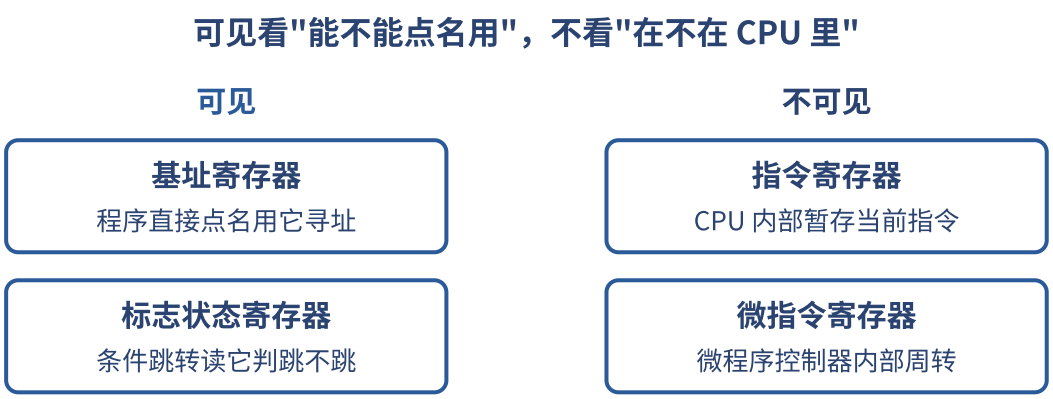
错误选项为什么错＋举例：

- A（仅Ⅰ、Ⅱ）：Ⅰ 和 Ⅱ 恰好都是内部周转寄存器。举例：指令寄存器里装的是 CPU 正在译码的那条指令，程序员写不出一条指令去读它、改它；微指令寄存器里装的是微指令，微指令是 CPU 内部的控制语言，汇编程序员连它的名字都点不了。
- B（仅Ⅰ、Ⅳ）：Ⅳ 是对的，但 Ⅰ 是错的。举例：程序员能感知标志位、能用条件跳转，Ⅳ 没错；但指令寄存器是取指后内部暂存的地方，程序员没有任何汇编指令能访问它，把它算进来就错了。

- C（仅 II、IV）：IV 是对的，但 II 是错的。举例：微指令寄存器属于微程序控制器内部，控制信号从它发出没错，但它只对 CPU 内部可见；汇编程序员写程序时根本接触不到微指令这一层，II 不能算可见。

笔记本一句话： 第 17 题＋汇编可见看能不能点名用、能不能靠它控制程序：基址寄存器能寻址、标志寄存器能判跳转，指令寄存器和微指令寄存器只是 CPU 内部周转。

【图像】



程序员可见与不可见寄存器的对照

【定位】 计算机组成原理 · 第 5 章中央处理器 · 程序员可见寄存器与内部寄存器

【易混对比】

- "存在于 CPU 里"和"对程序员可见"是两回事：指令寄存器、微指令寄存器真实存在，但程序员在汇编里点不了它们的名字，就是不可见。
- "直接点名用"和"间接感知到"是两种可见：基址寄存器是前者（程序里直接写它的名字），标志状态寄存器是后者（程序靠条件跳转读它的结果），两种都算可见。
- "微指令寄存器"和"指令寄存器"虽然名字像，但一个是微程序控制器的内部周转、一个是取指译码的内部暂存，两个都不可见，别因为名字熟就判可见。

可见看的是程序能不能点名用、能不能靠它控制程序，不是它在不在 CPU 里。

【坑位】

别被"寄存器"三个字骗了，先问"汇编程序里能不能点它的名"。

最容易踩的坑是看到四个都是寄存器就凭印象选：指令寄存器天天听就觉得可见，基址寄存器不熟就觉得不可见，正好反了。判断只看一条线：汇编程序能不能直接写它的名字（基址寄存器能）、能不能靠读它的结果控制程序走向（标志寄存器靠条件跳转能）。指令寄存器和微指令寄存器是 CPU 内部周转用的，程序没有任何指令能访问它们，再熟也不可见。

【套路】

- 第一步，问"程序里能不能直接点名用它"：能点名就是可见（如基址寄存器）。
- 第二步，问"程序能不能靠它控制走向"：条件跳转读标志位的结果，这也算可见（如标志状态寄存器）。
- 第三步，两条都不沾的，判为 CPU 内部周转寄存器，不可见。

点名能用、靠它能判跳转，就是可见；只在 CPU 内部周转，就是不可见。

【类比】

像饭店的厨房：服务员（汇编程序员）能点名要"大盘鸡"（基址寄存器），也能看"叫号屏"知道轮到谁（标志寄存器）；但配菜间里的备菜盆（指令寄存器）和厨师长的内部对讲机（微指令寄存器），服务员既点不了名也听不到，就是不可见。

【口诀】 点名能用才可见，内部周转都靠边。

【考频】 「程序员可见」在 2009–2022 年解析中出现 7 次 · 中频

第 18 题 · 关于数据通路的错误叙述（正确答案：C）

题干与选项：

关于数据通路的叙述中，错误的是（）。

- A. 数据通路包含 ALU 等组合逻辑元件
- B. 数据通路包含寄存器等时序逻辑元件
- C. 数据通路不包含用于异常事件检测及响应的电路
- D. 数据通路中数据的流动路径由控制信号控制

入手点： 本题问的是“错误的是”，先把数据通路的家底盘一遍：组合元件、时序元件、控制信号、异常检测，四个里面哪个说法和家底对不上，哪个就是答案。

题干专有名词从零解释：

- 数据通路：CPU 里数据实际流过的路径和部件的集合，运算、传送、暂存都在这条路上完成。
- ALU：算术逻辑单元，做加、减、与、或、移位等运算的组合逻辑部件。
- 组合逻辑元件：输出只由当前输入决定、不靠时钟、不记忆历史的部件，ALU、多路选择器、加法器都属于这一类。
- 时序逻辑元件：输出和时钟、历史状态有关的部件，寄存器、程序计数器都属于这一类，时钟一到才更新内容。
- 控制信号：控制部件发出的信号，决定多路选择器选哪一路、ALU 做哪种运算、寄存器写不写。
- 异常事件：执行过程中出现的不正常情况，例如算术溢出、除数为零、地址越界。
- 异常事件检测及响应的电路：发现异常并发出信号、配合控制部件转去异常处理程序的电路。

考查知识点： 数据通路的组成（组合逻辑元件、时序逻辑元件、控制信号的作用、异常检测电路）。

正确选项为什么对： C 说数据通路不包含异常检测及响应的电路，这个说法是错的。异常不是凭空被控制部件猜出来的，必须由数据通路里的电路先检测出来，再把信号送给控制部件，控制部件才能转去异常处理。举例：ALU 做加法时，内部的溢出检测电路会根据符号位和进位情况算出有没有溢出，溢出标志一出来，控制部件才知道这次运算出了异常；再举一例：访存地址越界时，地址比较电路在数据通路里先发现地址超出了界限，异常信号就是这样产生的。如果数据通路里根本没有检测电路，控制部件就收不到任何异常信号，异常处理也就无从启动，所以“不包含”这个说法站不住，C 就是本题要找的错误叙述。

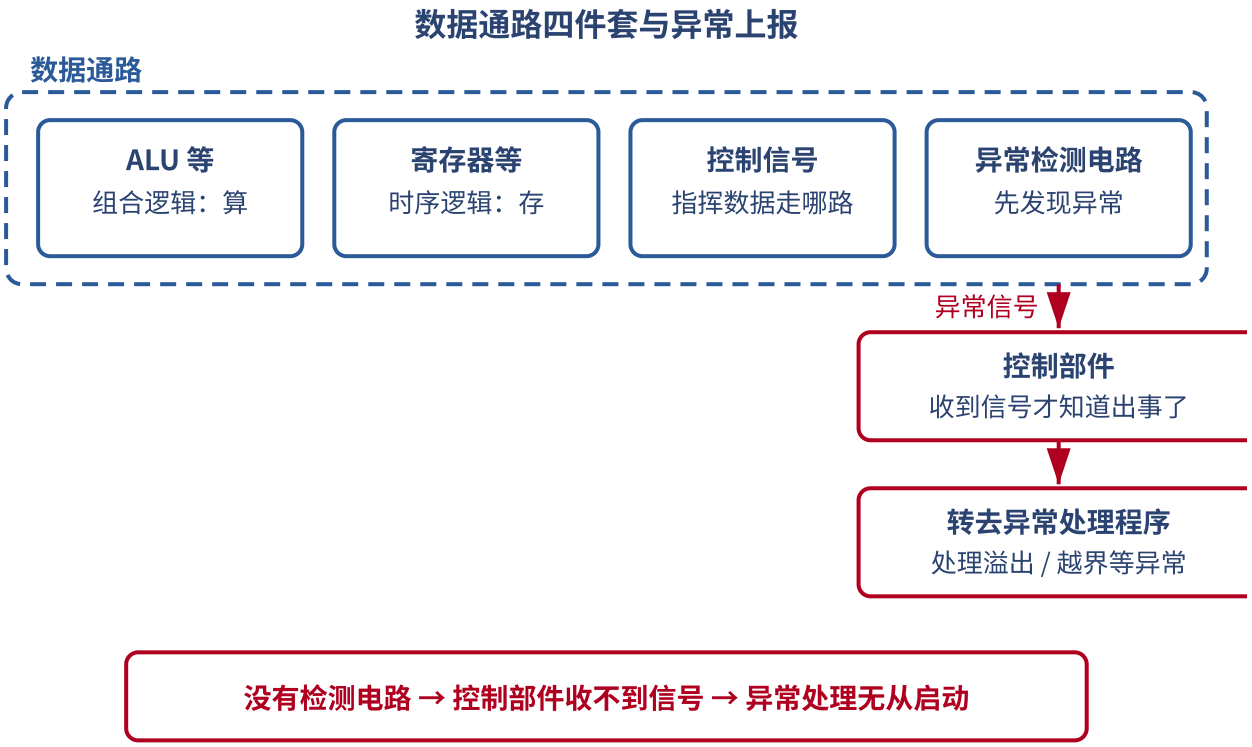
错误选项为什么错＋举例：

其余三项都是正确的叙述，下面逐项说明。

- A（包含 ALU 等组合逻辑元件）：正确。举例：加法、逻辑运算都在 ALU 里完成，ALU 的输出只由当前输入决定，是典型组合逻辑元件，数据通路离开它就没法运算。
- B（包含寄存器等时序逻辑元件）：正确。举例：通用寄存器存操作数和中间结果，程序计数器存下一条指令地址，这些寄存器都要靠时钟节拍更新，是典型时序逻辑元件，数据通路离开它们就没地方暂存数据。
- D（数据流动路径由控制信号控制）：正确。举例：多路选择器面前摆着两路数据，控制信号说选左路，数据就走左路；ALU 面前摆着加法和减法两种运算，控制信号说做加法，ALU 就做加法；寄存器写使能信号说写，数据才写进寄存器，整条路的走向全由控制信号指挥。

笔记本一句话： 第 18 题＋数据通路家底四件套：组合元件算、时序元件存、控制信号指挥路、异常检测电路先发现异常再报给控制部件。

【图像】



数据通路四件套与异常上报流程

【定位】 计算机组成原理 · 第 5 章中央处理器 · 数据通路的组成

【易混对比】

- 组合逻辑元件和时序逻辑元件是两回事：前者输出只看当前输入、不记历史（ALU、多路选择器），后者靠时钟更新、能记住状态（寄存器、程序计数器），数据通路两样都要。
- "控制信号指挥数据通路"和"数据通路向控制部件报告"是两个方向：控制信号决定多路选择器选哪路、ALU 做什么运算；异常信号反过来从数据通路流向控制部件，告诉它出事了。
- "异常处理程序"和"异常检测电路"分工不同：检测电路在数据通路里负责发现异常并发出信号，控制部件收到信号后才转去执行异常处理程序；没有检测，处理就无从启动。

异常是数据通路先发现、再报给控制部件的，不是控制部件凭空猜出来的。

【坑位】

看到"数据通路不包含异常检测电路"先别信，异常信号必须有人先发现才能报上去。

最容易踩的坑是把数据通路想成"只管算数传数据的傻管道"，觉得异常检测是控制部件的事。实际上控制部件只管"收到异常信号后转去处理"，而"发现异常"这件事必须由数据通路里的电路完成：ALU 里的溢出检测电路根据符号位和进位算出溢出，地址比较电路发现访存越界，信号都是这样先产生、再送给控制部件的。如果数据通路里没有检测电路，控制部件就收不到任何异常信号，整套异常机制根本转不起来。

数据通路的家底是四件套：组合元件算、时序元件存、控制信号指挥路、异常检测先报信。

【类比】

像工厂的生产线：机器负责加工（组合元件），仓库负责暂存半成品（时序元件），调度员的指令牌决定每批货走哪条传送带（控制信号），生产线上的质检探头发现次品就拉响警报（异常检测电路）。没有质检探头，调度员再

厉害也不知道出了次品。

【口诀】 组合算来时序存，控制指挥异常先报。

【考频】 「数据通路」在 2009–2022 年解析中出现 7 次 · 中频

第 19 题 · 关于总线的错误叙述（正确答案：C）

题干与选项：

关于总线的叙述中，错误的是（）。

- A. 总线是两个或多个部件间数据交换的传输介质
- B. 同步总线由时钟信号定时，时钟频率不一定等于总线工作频率
- C. 异步总线由握手信号定时，一次握手过程完成一位数据交换
- D. 突发传送总线事务可连续传送多个数据

入手点： 本题问的是“错误的是”，握手那句最值得盯住：握手一次换的是多少数据，是“一根线上的 1 位”还是“一组线上的一个数据”，想清总线是多根线并行传数据，这句的毛病就露出来了。

题干专有名词从零解释：

- 总线：一组公共传输线，CPU、主存、I/O 设备这些部件都挂在上面，轮流用它交换数据。
- 传输介质：数据实际走过的通道，总线就是部件之间交换数据的公共通道。
- 同步总线：用统一的时钟信号给传输定节拍，发送方和接收方都按时钟边沿行动。
- 时钟频率：时钟信号每秒跳变多少次；总线工作频率：总线每秒完成多少次数据传送，两者不一定相等，因为一次传送可能占多个时钟周期。
- 异步总线：不用统一时钟，靠握手信号定节拍，发送方和接收方互相确认后再行动。
- 握手信号：发送方说“我把数据放好了”，接收方说“我已经收到了”，一问一答完成一次确认。
- 一次握手过程：从发送方发出请求、接收方确认收到，到双方完成这一次数据交接的完整过程。
- 突发传送：一次总线事务里只给一个首地址，后面多个数据按地址连续自动传送，不用每个数据都重新给地址、重新仲裁。

考查知识点： 总线的定义、同步总线与异步总线的定时方式、握手过程的数据量、突发传送的特点。

正确选项为什么对： C 说一次握手完成一位数据交换，这个说法是错的。总线是多根数据线并行工作的，一次握手交接的是一个完整的数据单位，例如一个字，而不是一根线上的 1 位。举例：32 位数据总线有 32 根数据线，一次握手里 32 根线同时各传 1 位，合起来就是一个 32 位的字一次交接完毕；如果一次握手只换 1 位，传一个 32 位的字就要握手 32 次，并行总线就白并行了。握手定的是“这一次交接什么时候算完成”，不是“这一次交接只传 1 位”，C 把定时单位和数据单位混在了一起，所以是错误叙述。

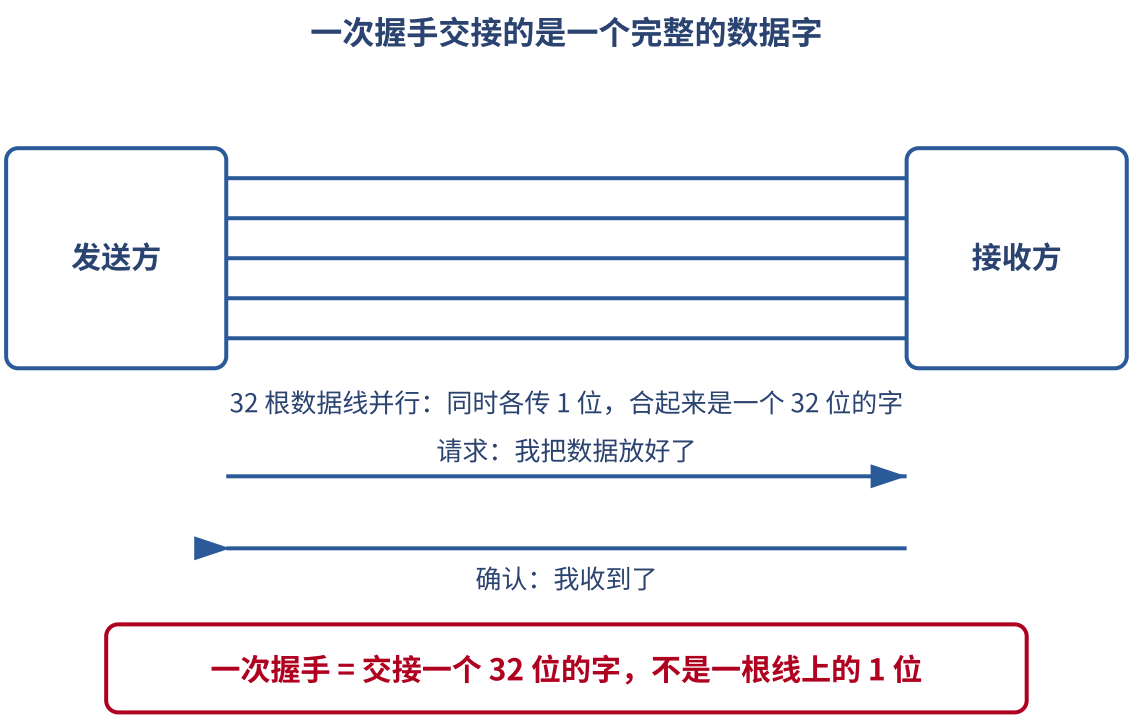
错误选项为什么错＋举例：

其余三项都是正确的叙述，下面逐项说明。

- A（总线是两个或多个部件间数据交换的传输介质）：正确。举例：CPU 要读主存里的数据，数据就是走总线从主存送到 CPU 的；打印机要收 CPU 发来的数据，数据也是走总线过去的，总线正是部件之间共用的数据通道。
- B（同步总线由时钟定时，时钟频率不一定等于工作频率）：正确。举例：时钟每秒跳 100 次，一次传送占 2 个时钟周期，那么每秒只能完成 50 次传送，工作频率 50 就不等于时钟频率 100；同步只说明“按时钟节拍行动”，没说明“一个周期完成一次传送”。
- D（突发传送总线事务可连续传送多个数据）：正确。举例：CPU 要连续读主存里地址相邻的 4 个字，突发传送只发一次首地址和一次传送命令，后面 4 个字一个接一个送出来，不用每个字都重新发地址、重新争用总线。

笔记本一句话： 第 19 题＋异步总线一次握手交接一个完整数据单位（例如一个字），不是 1 位；同步看时钟节拍不等于一个周期传一次，突发是一次地址连传多个数据。

【图像】



一次握手交接一个完整数据字的示意图

【定位】 计算机组成原理 · 第 6 章总线 · 同步总线、异步总线与突发传送

【易混对比】

- "定时"和"数据量"是两回事：握手信号定的是"这一次交接什么时候算完成"，不是"这一次交接传多少位"；一次交接的数据量由数据线的根数决定。
- 同步总线和异步总线差在"谁定节拍"：同步靠统一时钟、双方按时钟边沿行动；异步靠双方一问一答的握手信号，不用统一时钟。
- "时钟频率"和"总线工作频率"是两回事：同步总线一次传送可能占多个时钟周期，工作频率可以低于时钟频率，"按时钟节拍"不等于"一个周期传一次"。

握手定的是"什么时候算完"，不是"一次传几位"。

【坑位】

看到"一次握手"配"一位"先打问号：总线是多根线并行，一次交接的是一个完整的数据单位。

最容易踩的坑是把"一次"和"一位"绑在一起：握手一次 → 只传 1 位。但 32 位数据总线有 32 根数据线，一次握手里 32 根线同时各传 1 位，合起来就是一个 32 位的字一次交接完毕。如果一次握手只传 1 位，传一个字要握手 32 次，并行总线就白并行了。错误选项错就错在把定时单位（一握手）和数据单位（一位）混在了一起。

【套路】

- 第一步，把"定时"和"数据量"拆开看：定时回答"什么时候算完成"，数据量回答"一次交接多少"。
- 第二步，看到"一位""一根线"先打问号：总线是多根数据线并行，一次交接的是一个完整的数据单位（如一个字）。

- 第三步，同步看时钟节拍、异步看握手问答、突发看一次地址连传多个，逐项核对。

定时管"何时完"，线数管"传多少"，突发管"连传几个"。

【类比】

像两个人交接一箱苹果：握手是"数清楚、签收"这个确认动作，一次交接的是一整箱，不是一只苹果。说"一次握手只交接一只苹果"，就和错误选项的毛病一样：把"确认动作"和"交接数量"混了。

【口诀】 握手一次一整字，同步异步别混搭。

【考频】 「同步总线」在 2009–2022 年解析中出现 15 次 · 高频

第 20 题 · 不属于 I/O 接口的部件（正确答案：A）

题干与选项：

不属于 I/O 接口的是（）。

- A. 磁盘驱动器
- B. 打印机适配器
- C. 网络控制器
- D. 可编程中断控制器

入手点： 本题问的是“不属于”，要先分清两类东西：一类是外设本身，一类是夹在主机和外设之间的连接逻辑；接口属于后一类，外设本身不属于接口。

题干专有名词从零解释：

- I/O 接口：夹在主机和外设之间的连接逻辑，负责信号转换、数据缓冲、设备选择、向 CPU 报告状态这些事，让主机能和速度、信号格式都不同的外设打交道。
- 外设：主机之外的设备本身，例如磁盘、打印机、键盘，完成存储、打印、输入这些实际功能。
- 磁盘驱动器：磁盘设备本身，包括盘片、磁头、读写机械和驱动电路，负责把数据写进盘片、从盘片读出来。
- 打印机适配器：夹在主机和打印机之间的接口卡或接口逻辑，把主机的数据和控制信号转换成打印机能接受的格式。
- 网络控制器：夹在主机和网络之间的接口逻辑，负责按网络协议收发数据帧，让主机能接入网络。
- 可编程中断控制器：夹在外设和 CPU 之间的中断管理逻辑，收集多个外设的中断请求，按优先级排队，再向 CPU 发出中断信号。

考查知识点： I/O 接口的定义、接口与外设本身的区分、常见接口部件的识别。

正确选项为什么对： A 的磁盘驱动器是外设本身，不是接口。接口的活是“翻译和转接”，驱动器的活是“存数据”：盘片转起来、磁头定位、磁信号和电信号转换，这些都是磁盘设备自己的存储功能。举例：主机要访问磁盘时，数据要先经过磁盘接口电路（转接逻辑）才能到达磁盘驱动器，驱动器再把数据写进盘片；驱动器是被接口连接的对象，不是接口本身。就像打印机是被连接的对象、打印机适配器才是接口一样，磁盘驱动器和磁盘接口是两个东西，A 就是那个不属于接口的项。

错误选项为什么错＋举例：

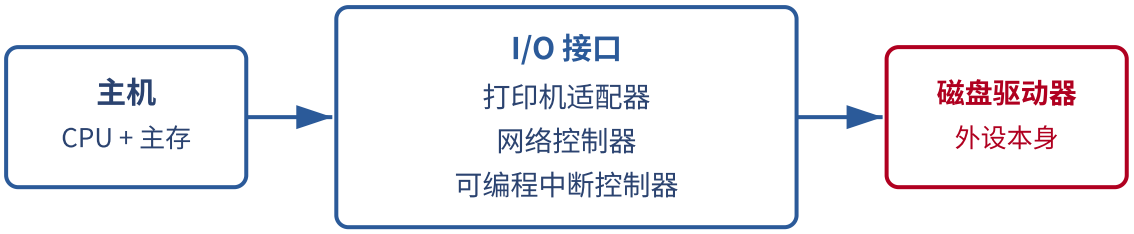
其余三项都属于 I/O 接口，下面逐项说明。

- B（打印机适配器）：属于接口。举例：主机发来的数据是并行格式、速度很快，打印机接受的是另一种信号格式、速度很慢，打印机适配器夹在中间做格式转换和速度缓冲，这正是接口干的活。
- C（网络控制器）：属于接口。举例：主机要上网发数据，网络控制器把主机的数据按网络协议打包成帧、通过网线发出去，收到的数据帧再解包交给主机，主机和网络之间全靠它转接。
- D（可编程中断控制器）：属于接口。举例：键盘、打印机、磁盘好几个设备同时想打断 CPU，中断控制器把这些请求收拢、按优先级排队、一次只把最高优先级的请求发给 CPU，CPU 和外设之间的中断往来全靠它管理。

笔记本一句话： 第 20 题＋接口是夹在中间的转接逻辑，驱动器、打印机这些外设本身不是接口；适配器、控制器才是接口。

【图像】

接口是转接逻辑，外设是干活的设备本身



磁盘驱动器是外设本身，不是接口

分界线：数据先过接口、再到驱动器；被接口连着的是外设

I/O 接口与外设本身的对照

【定位】 计算机组成原理 · 第 7 章输入输出系统 · I/O 接口的定义与组成

【易混对比】

- 接口和外设本身是两回事：接口是夹在中间的转接逻辑（翻译、缓冲、选设备），外设是干活的设备本身（存数据、打印、上网），磁盘驱动器属于后者。
- "适配器/控制器"和"驱动器"是两类名字：名字里带适配器、控制器的，通常是接口；名字里带驱动器、设备的，通常是外设本身。
- 可编程中断控制器虽然名字里有"控制器"、管的是中断不是数据，但它干的还是"收拢转接"的活，所以它是接口，不是外设。

接口干的是"翻译转接"，外设干的是"存、打、算"本身。

【坑位】

看到"磁盘"两个字别急着判接口，先问"它是干活的设备本身，还是夹在中间的转接逻辑"。

最容易踩的坑是把"和磁盘有关的东西"都当成接口：磁盘驱动器里确实有驱动电路，但它的活是"把数据写进盘片、从盘片读出来"——这是存储功能，是外设本身的活。真正的磁盘接口是夹在主机和驱动器之间的那块转接逻辑。记住分界线：主机要访问磁盘，数据先过接口、再到驱动器；被接口连接的那个对象，本身不是接口。

【套路】

- 第一步，问"它干的是转接翻译，还是存、打、算本身"：转接翻译就是接口。
- 第二步，看名字：适配器、控制器多半是接口；驱动器、设备多半是外设本身。
- 第三步，画一条线：主机—接口—外设，被接口连着的那个是外设。

夹在中间做转接的才是接口，被连接的设备本身不是。

【类比】

像出国旅游的翻译：翻译（接口）夹在你和外国人（外设）中间做沟通，你要买的东西（数据）先过翻译再到店主手里。店主本人不是翻译，磁盘驱动器就是那个店主，打印机适配器才是翻译。

【口诀】 接口转接外设干，驱动器是干活的汉。

【考频】 「I/O 接口」在 2009–2022 年解析中出现 8 次 · 中频

第 21 题 · 处理后必须回到当前指令重新执行的事件（正确答案：B）

题干与选项：

异常事件在当前指令执行过程中检测，中断请求在当前指令执行后检测。下列事件中，处理程序执行后必须回到当前指令重新执行的是（）。

- A. 系统调用
- B. 页缺失
- C. DMA 传送结束
- D. 打印机缺纸

入手点： 回不回当前指令，只看一句话：当前指令有没有干完。干了一半被打断的要重干，干完了才被打断的接着干下一条，主动“叫服务”的那条本身已经完成了使命。

题干专有名词从零解释：

- 异常：执行指令过程中由 CPU 内部或指令本身引起的特殊事件，检测时机在当前指令执行过程中。
- 中断：由外部设备引起、请求 CPU 服务的事件，检测时机在当前指令执行完之后。
- 系统调用：程序主动请求操作系统服务的指令，例如申请读写文件，执行这条指令本身就是“叫服务”，服务做完就接着执行下一条指令。
- 页缺失：访问的页面不在主存里，地址转换时发现页表项的存在位是 0，CPU 暂停当前指令、转去把页面从外存调入主存的异常。
- DMA 传送结束：直接存储器访问方式下，外设和主存之间成批传送数据的工作做完了，外设向 CPU 发出的中断请求，告诉 CPU 这一批数据已经传完。
- 打印机缺纸：打印机纸用完了，外设向 CPU 发出的中断请求，告诉 CPU 打印没法继续。
- 重新执行：处理程序做完后，程序计数器指回当前指令的地址，让这条指令从头再执行一次。

考查知识点： 异常与中断的检测时机、故障类异常必须重执行、中断和自陷返回下一条指令。

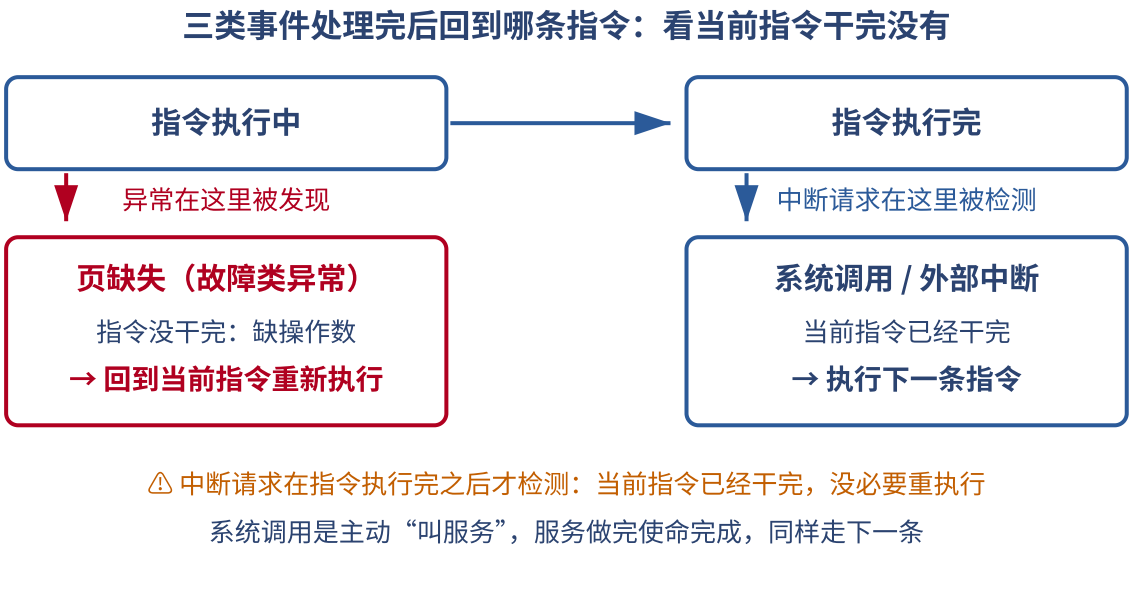
正确选项为什么对： B 的页缺失是故障类异常，在当前指令执行过程中、地址转换那一步就被发现了：指令要访问的那个操作数所在的页面不在主存里，指令拿不到操作数，根本没法干完。处理程序把页面从外存调入主存、填好页表项之后，指令缺的那个操作数已经有了，但指令本身还没执行完，所以必须回到当前指令的地址重新执行一次，这一次地址转换能找到页面，指令才能真正干完。举例：指令要把主存某单元的数加 1，地址转换发现那个页面不在主存，指令卡在取数这一步；处理程序把页面调进来之后，指令重执行一遍，取数、加 1、写回三步才能走完。

错误选项为什么错＋举例：

- A（系统调用）：系统调用指令本身就是“叫服务”，服务做完使命就完成了，返回时接着执行下一条指令。举例：程序执行“打开文件”这条系统调用，操作系统把文件打开、把结果交回来，程序接着执行打开文件之后的下一条指令，没必要把“打开文件”再叫一遍。
- C（DMA 传送结束）：这是外部中断，在当前指令执行完之后才被检测，当前指令已经干完了。举例：CPU 正在执行一条加法指令，加法做完了，DMA 控制器才报告“这一批数据传完了”，CPU 处理完报告，接着执行加法之后的下一条指令，加法没必要重做。
- D（打印机缺纸）：这也是外部中断，同样在当前指令执行完之后才被检测，当前指令已经干完了。举例：CPU 正在执行一条传送指令，传送做完了，打印机才报告“纸没了”，CPU 处理完缺纸报告，接着执行传送之后的下一条指令，传送没必要重做；缺纸影响的是后续打印任务，不是当前这条已经干完的指令。

笔记本一句话： 第 21 题＋故障类异常卡在指令中间要重执行（页缺失），系统调用和外部中断都是当前指令干完后才处理，返回执行下一条。

【图像】



三类事件处理完后回到哪条指令：页缺失回当前指令重执行，系统调用与外部中断走下一条

【定位】 计算机组成原理 · 第 7 章 · 输入输出系统 · 中断与异常 · 故障类异常重执行

【易混对比】

- 异常和中断：异常在指令执行过程中被发现，指令没干完；中断请求在指令执行完之后才被检测，指令已经干完。回不回当前指令，看的就是这条分界线。
- 故障类异常和系统调用：故障是"想干没干成"（页缺失），处理完要重干；系统调用是"主动叫服务"，服务做完使命完成，走下一条。
- 页缺失和打印机缺纸：页缺失卡在指令中间（异常），打印机缺纸是外设在指令干完后报告（中断），一个要重执行，一个走下一条。

回不回当前指令，只看一句话：当前指令有没有干完。

【坑位】

中断请求都是在指令执行完之后才检测的，别把任何中断当成异常来处理。

最容易踩的坑，是看到 DMA 传送结束、打印机缺纸这类带"中断"字眼的事件，就以为它们和页缺失一样要重执行。题干已经把检测时机写明白了：中断请求在当前指令执行后检测。既然是执行完之后才被发现，当前指令必然已经干完，处理完直接走下一条就行。反过来，页缺失是在地址转换那一步、指令执行过程中被发现的，指令缺操作数没干完，处理完必须回到当前指令重新执行。

【套路】

- 第一步，看检测时机：指令执行过程中发现的是异常，指令执行完之后才检测的是中断。
- 第二步，问当前指令干完没有：没干完的是故障类异常（页缺失），干完了的是中断和系统调用。
- 第三步，没干完 → 回到当前指令重新执行；干完了 → 执行下一条指令。

没干完重执行，干完了走下一条。

【类比】

像在厨房做菜：做到一半发现盐没了（页缺失），你只能停下，出门买盐回来，这道菜从头重做一遍——因为上一步根本没做完。做完一道菜、端上桌之后，手机响了（外部中断），你接完电话直接做下一道菜，不会把刚做完的菜重做一遍。系统调用像主动打电话叫外卖：电话打完了使命就完成了，接着干下一件事。

【口诀】 故障卡半道，必返重执行；中断指令后，下一条继续。

【考频】 「页缺失」在 2009–2022 年解析中出现 24 次 · 高频

第 22 题 · 关于多重中断系统中 CPU 响应中断的错误叙述（正确答案：A）

题干与选项：

关于多重中断系统中 CPU 响应中断的叙述，错误的是（）。

- A. 仅在用户态下 CPU 才能检测和响应中断
- B. CPU 只有在检测到中断请求信号后，才会进入中断响应周期
- C. 进入中断响应周期时，CPU 一定处于中断允许（开中断）状态
- D. 若 CPU 检测到中断请求信号，则一定存在未被屏蔽的中断源请求信号

入手点： 本题问的是“错误的是”，先把响应中断的门槛拆开：特权级是一道门，中断允许是一道门，请求信号是一道门，哪道门被说成了唯一的门，哪句就是错的。

题干专有名词从零解释：

- 多重中断系统：允许多个中断源存在、且高优先级中断可以打断低优先级中断处理过程的中断系统。
- 用户态：CPU 执行普通用户程序时的特权级，权限受限，不能执行特权指令。
- 内核态：CPU 执行操作系统内核代码时的特权级，权限最高，能执行全部指令。
- 中断请求信号：外设或中断控制器送给 CPU 的“请服务”信号，CPU 在指令执行的边界检查有没有这个信号。
- 中断响应周期：CPU 收到中断请求后，暂停当前程序、保存断点、转去中断服务程序的那段处理过程。
- 中断允许（开中断）：CPU 内部的中断允许标志处于允许状态，允许响应可屏蔽中断；关中断时 CPU 不响应可屏蔽中断。
- 屏蔽：中断控制器或 CPU 把某个中断源的请求挡住，不让它送到 CPU；未被屏蔽的中断源请求才能真正到达 CPU。
- 中断源：发出中断请求的设备或部件，例如键盘、磁盘、定时器。

考查知识点： 中断响应的条件（请求信号、开中断状态、屏蔽机制）、特权级与中断响应的关系。

正确选项为什么对： A 说仅在用户态下 CPU 才能检测和响应中断，这个说法是错的。特权级管的是“这条指令有没有权限执行”，中断允许标志管的是“要不要响应中断”，这是两道不同的门，中断响应看的是后一道门，不是前一道门。举例：CPU 正在内核态执行操作系统代码，定时器中断请求到了，只要中断允许标志是开着的，CPU 照样暂停内核代码、转去处理定时器中断，处理完再回来；再举一例：用户程序正在用户态运行，磁盘中断请求到了，CPU 同样可以响应。用户态和内核态都能响应中断，“仅在用户态”把特权级错当成了响应门槛，所以 A 是错误叙述。

错误选项为什么错＋举例：

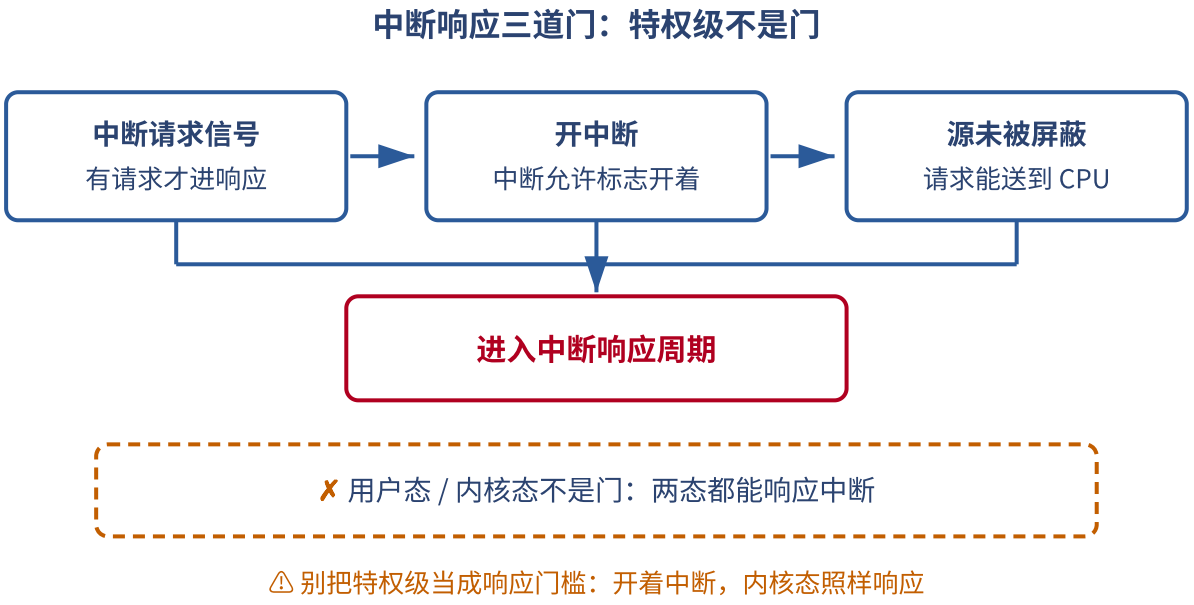
其余三项都是正确的叙述，下面逐项说明。

- B（只有检测到请求信号后才进入响应周期）：正确。举例：没有外设发请求，CPU 一条指令接一条指令正常执行，不会无缘无故停下来进响应周期；只有指令边界检查到了请求信号，CPU 才停下当前程序、保存断点、进响应周期。
- C（进入响应周期时一定处于开中断状态）：正确。举例：中断允许标志关着的时候，可屏蔽中断请求即使到了，CPU 也不会进响应周期，而是等到开中断之后再响应；能进响应周期，说明当时一定是开着中断的，关中断状态下进不了响应周期。

- D（检测到请求信号则一定存在未被屏蔽的中断源请求）：正确。举例：键盘中断源被屏蔽了，它的请求在中断控制器那一级就被挡住了，根本送不到 CPU；CPU 能检测到的请求信号，一定是某个没被屏蔽的中断源发出的、经过屏蔽检查和优先级排队后真正送达的信号。

笔记本一句话： 第 22 题＋中断响应看三道门：有请求信号、开着中断、源没被屏蔽，特权级（用户态还是内核态）不是响应门槛，两态都能响应。

【图像】



中断响应三道门：请求信号、开中断、源未被屏蔽，用户态与内核态不是门

【定位】 计算机组成原理 · 第 7 章 · 输入输出系统 · 中断与异常 · 中断响应条件

【易混对比】

- 用户态 / 内核态和开中断 / 关中断：前者管"这条指令有没有权限执行"，后者管"要不要响应中断"，是两道不同的门，响应中断看的是后一道。
- 屏蔽和关中断：屏蔽是把某个中断源的请求挡在送达 CPU 之前（请求到不了 CPU）；关中断是 CPU 层面的不响应（请求到了也不理）。一个管"送不送得到"，一个管"理不理"。
- 多重中断和单重中断：多重允许高优先级打断低优先级中断的处理过程，单重不允许；但两者响应中断的三道门槛是一样的。

中断响应看的是中断允许这道门，不是特权级这道门。

【坑位】

看到"仅在用户态"里的"仅"字先打个问号：凡是把某一条件说成唯一门槛的，先问还有没有别的门。

最容易踩的坑，是把"特权"和"中断响应"绑在一起：用户态权限低，直觉上会觉得中断这种"大事"只能在用户态响应、内核态反而不能。但特权级管的是指令执行权限，中断响应看的是有没有请求信号、开没开中断、源有没有被屏蔽。开着中断时，CPU 在内核态执行操作系统代码，定时器中断到了照样响应。A 项错就错在把特权级说成了唯一的门。

【套路】

- 第一步，先找绝对词：仅、一定、必须、只能，这类词最容易翻车。

- 第二步，把每句话翻译成"谁管谁"：A 说特权级管响应，B 说请求信号是进响应的前提，C 说开中断是进响应的前提，D 说请求信号意味着有未被屏蔽的源。
- 第三步，看哪句话把门安错了地方：响应中断的三道门是请求信号、开中断、源未被屏蔽，特权级不在其中，所以 A 是错误叙述。

绝对词先圈出来，再看它把门安在了谁身上。

【类比】

像公司前台接待访客：访客来了（请求信号）、前台今天开门营业（开中断）、访客不在黑名单上（未被屏蔽），三个条件齐了就接待。至于前台此刻是在接待普通客户还是在跟老板开会（用户态还是内核态），不影响"要不要接待这位访客"的决定——开着门，访客合规，就接待。

【口诀】 响应三道门：有请求、开中断、源未屏；特权级，不看门。

【考频】 「中断响应」在 2009–2022 年解析中出现 25 次 · 高频

第 23 题 · 仅内核态执行的指令（正确答案：B）

题干与选项：

下列指令中，只能在内核态执行的是（）。

- A. trap 指令
- B. I/O 指令
- C. 数据传送指令
- D. 设置断点指令

入手点： 先分清两类指令：碰了会危害全系统的归内核管，人畜无害的普通指令用户态也能跑；trap 指令恰恰是用户态向内核求助的入口，不能把它也划给内核。

题干专有名词从零解释：

- 内核态：操作系统内核运行的管理员模式，能执行全部指令，包括特权指令。
- 用户态：普通应用程序运行的访客模式，只能执行非特权指令，碰特权指令会被硬件拦下并触发异常。
- 特权指令：一旦用错会危害整个系统的指令，例如开关中断、修改页表基址、启动 I/O 操作。
- trap 指令：用户程序主动陷入内核的指令，例如系统调用指令；执行它会使 CPU 从用户态切换到内核态。
- I/O 指令：直接操作外设控制器寄存器、启动或控制输入输出操作的指令。
- 数据传送指令：把数据在寄存器与内存之间搬运的普通指令，例如 MOV 指令。
- 设置断点指令：调试用的指令，执行到它时会暂停程序并转入调试处理，例如 INT 3 指令。

考查知识点： 特权指令的概念，用户态与内核态的指令执行权限分界。

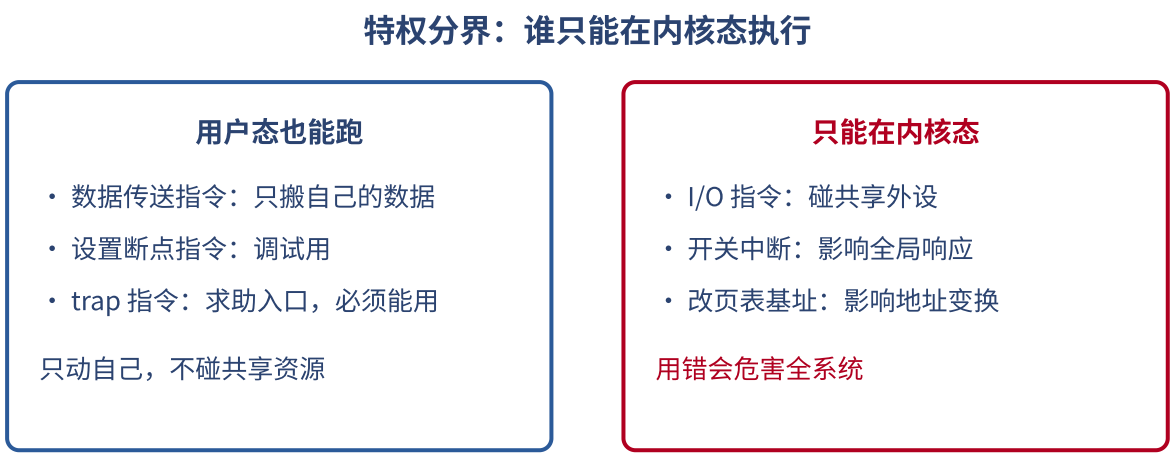
正确选项为什么对： I/O 指令是典型的特权指令。外设是全系统共享的资源，磁盘、网卡、打印机上的数据分属不同的用户和进程。如果允许任何用户程序直接发 I/O 指令，一个程序就可以绕过操作系统的检查，直接读别人的磁盘数据，或者向正在被别人使用的打印机乱发数据，全系统的安全和秩序就没了。所以硬件规定 I/O 指令只能在内核态执行，用户程序想用外设，必须通过系统调用请内核代劳，由内核检查权限后再发 I/O 指令。

错误选项为什么错＋举例：

- A（trap 指令）：trap 指令恰恰必须能在用户态执行。举例：一个普通程序想读文件，它自己没有读磁盘的权限，唯一的正规办法就是在用户态执行 trap 指令，陷入内核请内核帮忙。如果 trap 指令只能在内核态执行，用户程序就永远发不出求助信号，系统调用这条路就断了。
- C（数据传送指令）：数据传送是最普通的非特权指令。举例：程序算出一个数，把它从寄存器存回自己的内存变量，这就是一条数据传送指令；它只动自己的数据，不碰共享外设、不改系统配置，用户态天天都在执行它。
- D（设置断点指令）：设置断点是调试手段，可以在用户态由被调试的程序执行。举例：程序员给自己的程序下一个断点，程序跑到断点处停下来，转入调试处理；断点指令本身并不直接操纵外设或系统配置，所以不属于只能由内核执行的特权指令。

笔记本一句话： 第 23 题＋I/O 指令碰共享外设，只能内核执行；trap 指令是用户态向内核求助的入口，数据传送和断点指令都是用户态能跑的普通指令。

【图像】



特权分界对照：用户态能跑数据传送、断点、trap 指令，内核态专属 I/O 等特权指令

【定位】 操作系统 · 第 1 章 · 操作系统概述 · 特权指令与运行模式

【易混对比】

- trap 指令和 I/O 指令：trap 是用户程序向内核求助的入口，必须在用户态能用；I/O 指令直接碰共享外设，只能在内核态用。一个负责"敲门"，一个负责"动手"，方向正好相反。
- 特权指令和普通指令：特权指令用错会危害全系统（动外设、改系统配置），归内核专管；普通指令只动自己的数据，谁都能跑。
- 设置断点指令和 I/O 指令：断点指令只是让程序暂停转入调试，不直接操纵外设或系统配置，所以不算特权指令。

会不会危害全系统，是区分特权指令的唯一标准。

【坑位】

trap 指令最容易被误判：名字听起来像"陷入"很高级，但它恰恰是访客敲门用的。

最容易踩的坑，是把 trap 指令也划进"只能内核执行"。trap 指令的作用是让用户程序主动陷入内核请服务，如果它只能在内核态执行，用户程序就永远发不出求助信号，系统调用这条路就断了。记住它的方向：从用户态敲开内核的门，所以必须在用户态能用。

【套路】

- 第一步，问这条指令碰了会不会危害全系统或别人：动共享外设、改系统配置 → 特权指令。
- 第二步，特权指令 → 只能在内核态执行；只动自己数据的普通指令 → 用户态也能跑。
- 第三步，单独记忆 trap 指令：它是求助入口，方向是从用户态进内核，必须在用户态能用。

碰共享的归内核，敲门的必须能在门外用。

【类比】

像小区管理：访客（用户程序）按门铃（trap 指令）叫门完全没问题，门铃就是给访客按的；但配电箱、水泵房（I/O 指令、开关中断）只有物业（内核）能动，访客碰了会影响整栋楼。数据传送像在自己屋里搬家具，断点像给自己定的闹钟，都不影响别人。

【口诀】 碰共享归内核，trap 是敲门砖。

【考频】 「特权指令」在 2009–2022 年解析中出现 10 次 · 高频

第 24 题 · 创建进程必须完成的工作（正确答案：B，仅 I、II）

题干与选项：

操作系统创建新进程时，必须完成的工作是（ ）。

I. 申请空白 PCB

II. 初始化 PCB

III. 设置进程状态为执行态

- A. 仅 I
- B. 仅 I、II
- C. 仅 I、III
- D. 仅 II、III

入手点： 创建进程就是给一个新生命办户口：先领一张空白表格，再把信息填好；至于它什么时候上台跑，那是调度程序以后才决定的事，不能在出生那一刻就写死。

题干专有名词从零解释：

- 进程：正在运行的程序的一次执行过程，是操作系统分配资源的基本单位。
- PCB（进程控制块）：操作系统为每个进程建立的一张管理卡片，记录进程标识、运行状态、程序计数器、寄存器、内存和资源使用情况。
- 申请空白 PCB：从系统里找一张还没有被别的进程占用的空白管理卡片，分给新进程。
- 初始化 PCB：往这张卡片里填写新进程的初始信息，例如分配进程标识、设置初始状态、设置程序入口地址、清零或设置寄存器初值、登记它占用的资源。
- 执行态：进程正在 CPU 上运行的状态。
- 就绪态：进程只差 CPU 就能运行、正在排队等 CPU 的状态。
- 调度程序：操作系统里负责从就绪队列中挑选下一个上 CPU 的进程的部件。

考查知识点： 进程创建的基本步骤，新进程的初始状态。

正确选项为什么对： I 必须做：没有 PCB，操作系统就没有地方记录这个进程，调度程序、内存管理和资源管理都找不到它，进程等于不存在。II 必须做：一张空白卡片不填信息没法用，必须填上进程标识、初始状态、程序入口和资源登记，操作系统才能按这张卡片管理它。III 不必做：新进程出生时应当进入就绪态排队，等调度程序挑选它，它才能上 CPU。如果创建时就直接写成执行态，等于不经过调度就抢占了正在运行的进程，破坏了调度规则；更何况同一时刻能运行的进程数量受 CPU 个数限制，新进程不可能一出生就保证有空闲 CPU。所以必须完成的是 I 和 II，答案是 B。

错误选项为什么错＋举例：

- A（仅 I）：漏了 II。举例：只领了一张空白表格却一个字不填，调度程序拿到它也不知道这个进程是谁、从哪条指令开始跑、处在什么状态，这张卡片等于废纸，进程还是没法运行。
- C（仅 I、III）：漏了 II，又多带了不该做的 III。举例：表格没填就直接写成执行态，调度程序既不知道它是谁，也没法把它和其他进程放在一起排队比较，等于让一个身份不明的进程直接上台。
- D（仅 II、III）：漏了 I。举例：没有先申请到一张空白卡片，初始化就无处可填；如果直接往一张正被别的进程使用的卡片上写，就会把别人的信息覆盖掉，两个进程的管理信息就乱套了。

笔记本一句话： 第 24 题＋创建进程必做两件：领空白 PCB、填好 PCB；新进程先进就绪态排队，上不上 CPU 由调度程序说了算。

【图像】



创建进程流程：申请空白 PCB、初始化 PCB、进入就绪态排队，执行态是调度以后的安排

【定位】 操作系统 · 第 2 章 · 进程管理 · 进程的创建

【易混对比】

- 申请空白 PCB 和初始化 PCB：前者是领一张没人用的空卡片，后者是往卡片上填信息，两步都不能少，顺序也不能反。
- 创建时做的事和调度时做的事：创建只负责"办户口"（领卡、填卡、进就绪队），"上台"（进执行态）是调度程序以后的安排。
- 就绪态和执行态：新进程出生先进就绪态排队；执行态是被调度选中、上 CPU 之后的状态，创建那一刻给不了。

创建管出生登记，调度管上台，出生不等于上台。

【坑位】

别把"初始化 PCB"和"设置成执行态"当成一件事：填表是出生登记，上台是调度安排。

最容易踩的坑，是觉得"创建进程"听起来很厉害，新进程一出生就该是执行态。但同一时刻能上 CPU 的进程数受 CPU 个数限制，新进程不可能一出生就保证有空闲 CPU；更重要的是，不经调度直接写成执行态，等于抢占了正在运行的进程，调度规则就乱了。所以创建只做到进就绪态排队为止。

【套路】

- 第一步，默写创建必做的两件：申请空白 PCB（领卡）、初始化 PCB（填卡）。
- 第二步，新进程的去向写成就绪态：排队等调度，不直接给执行态。
- 第三步，看到选项里出现"设置成执行态""直接运行"这类说法，直接排除。

领卡、填卡、先进队，上台等调度。

【类比】

像新生入学报到：先领一张空白学籍卡（申请 PCB），再把姓名、班级填好（初始化），然后去候场区排队（就绪态）；校长点名（调度程序）叫到你，你才能上台领奖（执行态）。不能一入学就直接站上领奖台，也不能领了空白卡一个字不填。

【口诀】 领卡填卡先进队，上台还得调度批。

【考频】 「进程创建」在 2009–2022 年解析中出现 13 次 · 高频

第 25 题 · 时间片轮转需要使用的机制（正确答案：C，仅 I、II、III）

题干与选项：

分时系统实现时间片轮转调度，需要使用的是（）。

I. 进程控制块

II. 时钟中断处理程序

III. 进程就绪队列

IV. 进程阻塞队列

- A. 仅 II、III
- B. 仅 I、IV
- C. 仅 I、II、III
- D. 仅 I、II、IV

入手点： 把一次轮转拆成三个动作：时间到了要有人喊停，喊停后要有人保存现场并挑下一个，被挑的下一个要排在一个队里等着；哪个部件对不上这三个动作，哪个就是多余的。

题干专有名词从零解释：

- 分时系统：多个用户或进程轮流使用 CPU 的系统，每个进程每次只能用一小段时间。
- 时间片：分给一个进程的一段连续 CPU 使用时间，例如几十毫秒；时间片用完，进程就被换下来。
- 时间片轮转调度：进程按排队顺序轮流上 CPU，每人每次最多用一个时间片的调度方法。
- 进程控制块（PCB）：记录每个进程状态、寄存器和资源的管理卡片，换上换下时靠它保存和恢复现场。
- 时钟中断处理程序：时钟每隔一段时间发一次中断信号，操作系统响应这个信号、执行换进程操作的那段程序。
- 进程就绪队列：排着只差 CPU 就能运行的进程的队，调度程序从这个队里挑下一个上 CPU 的进程。
- 进程阻塞队列：排着正在等待某个事件（例如等待 I/O 完成）的进程的队，队里的进程即使有 CPU 也不能跑。

考查知识点： 时间片轮转调度的实现要素，时钟中断在抢占中的作用。

正确选项为什么对： I 需要：时间片用完要把正在运行的进程换下来，必须把它的程序计数器、寄存器等现场存进它的 PCB，下次轮到它时再从 PCB 里恢复，否则它跑到哪一步就丢了。II 需要：时间片的长短要有人计时，时钟中断就是这个闹钟；闹钟一响，时钟中断处理程序接管，执行保存现场和重新调度，否则进程会一直赖在 CPU 上不下来，轮转就转不起来。III 需要：被换下来的进程和还没轮到的进程都要有个排队的地方，调度程序每次从就绪队列队首挑一个上 CPU，轮转的顺序就是这个队的顺序。IV 不需要：阻塞队列里是等 I/O 或等事件的进程，它们不能因为时间片到了就上 CPU；时间片轮转管的是就绪进程之间的轮换，阻塞进程要先被事件唤醒、回到就绪队列，才能参与轮转。

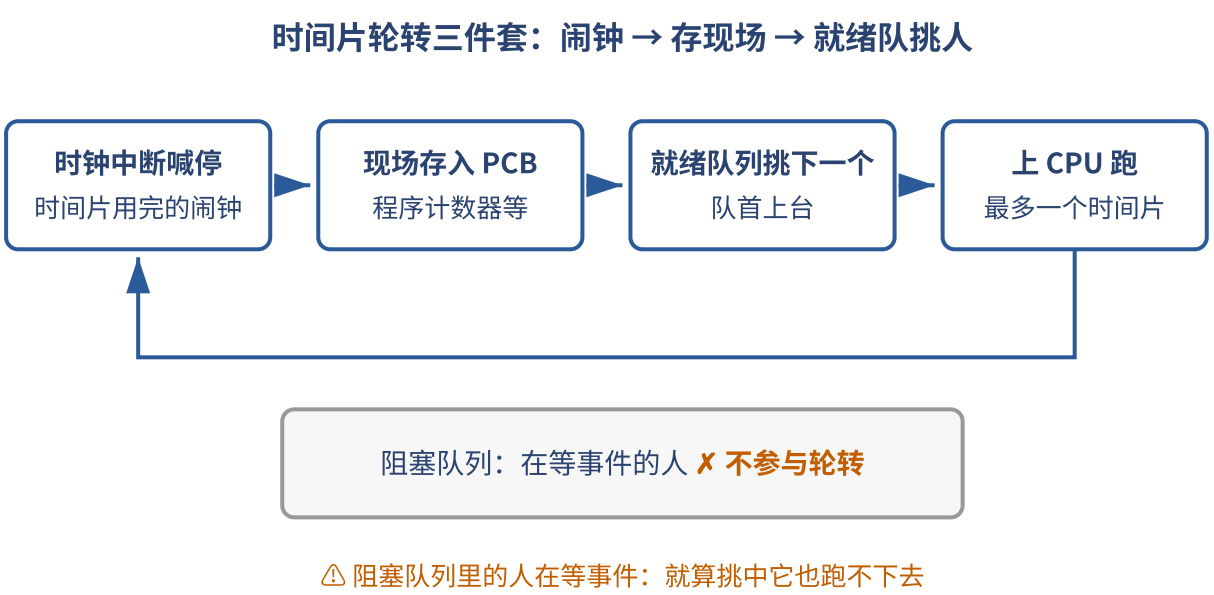
错误选项为什么错＋举例：

- A（仅 II、III）：漏了 I。举例：闹钟响了、队里也有人等着，但正在跑的进程现场没地方存，换下来就等于把它跑到哪一步丢了，下次再轮到它只能从头开始，这样的轮转没法用。
- B（仅 I、IV）：漏了 II 和 III，又错带了 IV。举例：没有时钟中断当闹钟，没人喊停，第一个进程可以一直跑下去；没有就绪队列，调度程序也不知道下一个该挑谁；反而把等 I/O 的阻塞队列搬进来，队里的进程即使被挑中也不能跑。

- D（仅 I、II、IV）：把 IV 当成了必需，把 III 漏了。举例：一个进程在等磁盘数据时进了阻塞队列，磁盘数据没回来之前，就算把它挑上 CPU，它也只能干等数据，跑不下去；真正能跑的候选人都在就绪队列里，不用就绪队列，调度程序就挑不到能跑的人。

笔记本一句话： 第 25 题＋时间片轮转三件套：PCB 存现场、时钟中断喊停、就绪队列排队挑人；阻塞队列里的人在等事件，不参与轮转。

【图像】



时间片轮转三件套循环：时钟中断喊停、现场存入 PCB、就绪队列挑人，阻塞队列不参与

【定位】 操作系统 · 第 2 章 · 进程管理 · 处理机调度 · 时间片轮转

【易混对比】

- 就绪队列和阻塞队列：就绪队列里是只差 CPU 的人，是轮转的候选人；阻塞队列里是在等事件的人，事件没到给 CPU 也跑不下去，不参与轮转。
- 时钟中断和其他中断：时钟中断是时间片轮转的"闹钟"，到点喊停；其他中断（如 I/O 完成）是报告事件，不负责计时。
- PCB 和就绪队列：PCB 存的是单个进程的现场（跑到哪一步了），就绪队列管的是候选人的排队顺序，一个管"状态"，一个管"顺序"。

轮转的三件套：闹钟喊停、现场有处存、候选人有队排。

【坑位】

阻塞队列最容易被误选：名字里有"队列"两个字，就觉得调度都要用它。

最容易踩的坑，是看到"进程阻塞队列"就把它当成轮转的必需品。但阻塞队列里的人在等 I/O 或等事件，事件没回来之前，就算把它挑上 CPU，它也只能干等数据，跑不下去。时间片轮转管的是就绪进程之间的轮换，阻塞进程要先被事件唤醒、回到就绪队列，才能参与轮转。

【套路】

- 第一步，把一次轮转拆成三个动作：时间到了谁喊停、喊停后现场存哪、下一个从哪挑。
- 第二步，对动作找部件：喊停靠时钟中断，存现场靠 PCB，挑人靠就绪队列。

- 第三步，看哪个部件对不上这三个动作：阻塞队列里的人不能跑，对不上"挑人上台"，排除。

三动作对三部件，对不上的就是多余项。

【类比】

像 KTV 包间轮唱：闹钟到点切歌（时钟中断喊停），把上一位唱到哪句记在小本上（现场存入 PCB），下一位从排队名单（就绪队列）里叫上来。在包间外面等朋友的人（阻塞队列）不在轮唱名单里——朋友没到，就算叫他也唱不了。

【口诀】 闹钟喊停存现场，就绪排队轮着上。

【考频】 「时间片轮转」在 2009–2022 年解析中出现 19 次 · 高频

第 26 题 · SSTF 磁头移动距离（正确答案：C，41）

题干与选项：

某磁盘的磁道数为 200（编号 0～199），磁头当前位于 184 号磁道。现有磁盘请求依次到达，请求的磁道号依次为 184、187、176、182、199。采用最短寻道时间优先（SSTF）算法完成这些访问后，磁头移动的总距离（磁道数）是（ ）。

- A. 37
- B. 38
- C. 41
- D. 42

入手点： SSTF 不看请求到达的先后顺序，每一步只看离磁头当前位置最近的那个还没访问的请求；一步一步算距离、一步一步划掉已访问的请求，最后把每步距离加起来。

题干专有名词从零解释：

- 磁道：磁盘盘面上一圈一圈的同心圆轨道，编号从 0 开始，本题编号 0～199。
- 磁头：读写磁盘数据的部件，在不同磁道之间移动；移动的距离按跨过的磁道数算。
- 磁盘请求：某个进程要求访问某个磁道上的数据，本题的请求就是那 5 个磁道号。
- SSTF（最短寻道时间优先）：每次从还没访问的请求里挑离磁头当前位置最近的那个先访问的调度算法。
- 移动距离：磁头从一个磁道走到下一个磁道跨过的磁道数，等于两个磁道号相减后的绝对值。

考查知识点： SSTF 磁盘调度算法的执行过程与总寻道距离计算。

正确选项为什么对： 从 184 号磁道出发，未访问的请求是 184、187、176、182、199。第一步：磁头已经在 184 号，请求 184 的距离是 184 − 184 = 0，先访问 184，累计移动 0。第二步：剩下 187、176、182、199，离当前位置 184 的距离分别是 3、8、2、15，最近的是 182，距离 2，访问 182，累计移动 0 + 2 = 2。第三步：剩下 187、176、199，离当前位置 182 的距离分别是 5、6、17，最近的是 187，距离 5，访问 187，累计移动 2 + 5 = 7。第四步：剩下 176、199，离当前位置 187 的距离分别是 11、12，最近的是 176，距离 11，访问 176，累计移动 7 + 11 = 18。第五步：只剩 199，离当前位置 176 的距离是 199 − 176 = 23，访问 199，累计移动 18 + 23 = 41。所以总移动距离是 41，答案是 C。

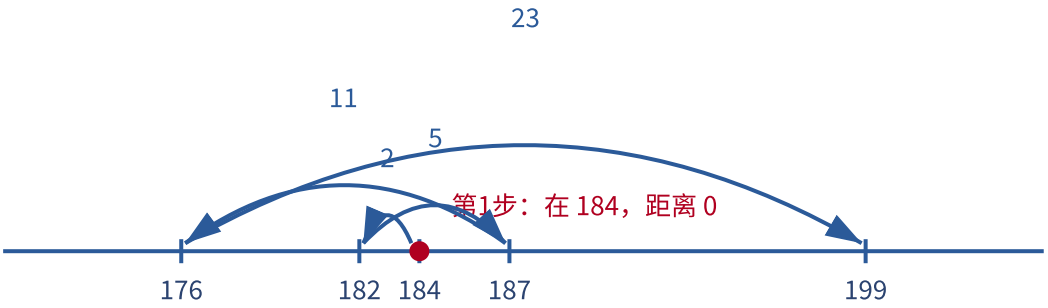
错误选项为什么错＋举例：

- A（37）：少算了 4，常见于某一步把距离算小了。举例：第四步从 187 走到 176，距离是 187 − 176 = 11，不是 7；如果把这一步误算成 7，总和就会从 41 掉到 37。
- B（38）：少算了 3，常见于把某一步的距离看错一位。举例：第二步从 184 走到 182，距离是 2；如果误以为磁头还在 184 号不动、直接按请求顺序先去 187，整条路线就变了，距离也会跟着错。
- D（42）：多算了 1，常见于把第一步的距离算成 1。举例：磁头本来就在 184 号，访问 184 号请求根本不用动，距离是 0；如果误以为“访问一次就要动 1”，总和就会从 41 涨到 42。

笔记本一句话： 第 26 题＋SSTF 每步只挑离当前位置最近的未访问请求，距离是磁道号之差的绝对值，一步一步加，别按到达顺序走。

【图像】

SSTF：从 184 出发，每步挑离当前位置最近的未访问请求



总距离 = 0 + 2 + 5 + 11 + 23 = 41

⚠ 别按请求到达顺序走：SSTF 每步只看离当前位置最近的未访问请求

SSTF 磁头从 184 出发的访问路径与每段距离：0、2、5、11、23，总距离 41

【定位】 操作系统 · 第 5 章 · 输入/输出管理 · 磁盘调度算法 · SSTF

【易混对比】

- SSTF 和 FCFS：SSTF 每步挑离当前位置最近的未访问请求，不管请求到达的先后；FCFS 严格按到达顺序走。本题如果按到达顺序走，路线和总距离全都会变。
- 距离和磁道号：距离是两个磁道号相减的绝对值（跨过的磁道数），不是磁道号本身；起点就在 184，访问 184 的距离是 0。
- "最近"和"最小磁道号"：最近是相对当前位置的距离最小，不是磁道号最小；从 187 出发时，176（距离 11）比 199（距离 12）近，所以先去 176。

SSTF 每步只认"离我最近"，不认"谁先到"。

【坑位】

第一步最容易多算 1：磁头本来就在 184 号，访问 184 根本不用动，距离是 0。

最容易踩的坑有两处：一是起点 184 和第一个请求 184 重合，访问它距离是 0，如果误以为"访问一次就要动 1"，总和就从 41 涨到 42；二是某一步的距离算小，比如从 187 到 176 是 11 不是 7，算小一步，总和就对不上。保险做法是每步都写出"从哪到哪、距离多少"，最后再加总。

【套路】

- 第一步，列出所有未访问请求，记下磁头当前位置。
- 第二步，每步算出各未访问请求到当前位置的距离（绝对值），挑最小的访问，划掉它并更新当前位置。
- 第三步，把每步的距离记下来，最后加总。
- 第四步，距离相等时任选其一（本题没出现，不用纠结）。

一步一算、一步一划、最后加总，别按到达顺序走。

【类比】

像外卖员送单：手里 5 个单子，不看哪个单先下单，每次都看离自己当前位置最近的那个没送的地址先去。送完一个，位置变了，重新看剩下单子里哪个最近。最后把每段路程加起来，就是总路程。

【口诀】 每步找最近，距离取绝对。

【考频】 「SSTF」在 2009–2022 年解析中出现 9 次 · 中频

第 27 题 · 可能引起进程调度的事件（正确答案：D，I、II、III、IV）

题干与选项：

下列事件中，可能引起进程调度程序执行的是（）。

I. 中断处理结束

II. 进程阻塞

III. 进程执行结束

IV. 时间片用完

- A. 仅 I、III
- B. 仅 II、IV
- C. 仅 III、IV
- D. I、II、III、IV

入手点： 只要出现“当前进程不能继续独占 CPU，或者出现了更值得上 CPU 的进程”这两种情况中的一种，调度程序就可能被请出来重新挑人；四个事件逐个往这两条上套。

题干专有名词从零解释：

- 进程调度程序：操作系统里负责挑选下一个上 CPU 运行的进程的部件。
- 中断处理结束：CPU 处理完一次中断（例如 I/O 完成中断、时钟中断），准备回到正常执行流的时刻。
- 进程阻塞：进程因为等待某个事件（例如等待 I/O 完成、等待资源）而主动让出 CPU、进入阻塞态。
- 进程执行结束：进程运行完毕、退出，释放它占用的 CPU 和资源。
- 时间片用完：分时系统里分给进程的一段 CPU 时间耗尽，进程被强制换下来。
- 就绪态：进程只差 CPU 就能运行的状态；阻塞态：进程在等事件、暂时不能运行的状态。

考查知识点： 引起进程调度的时机，进程状态转换与调度的关系。

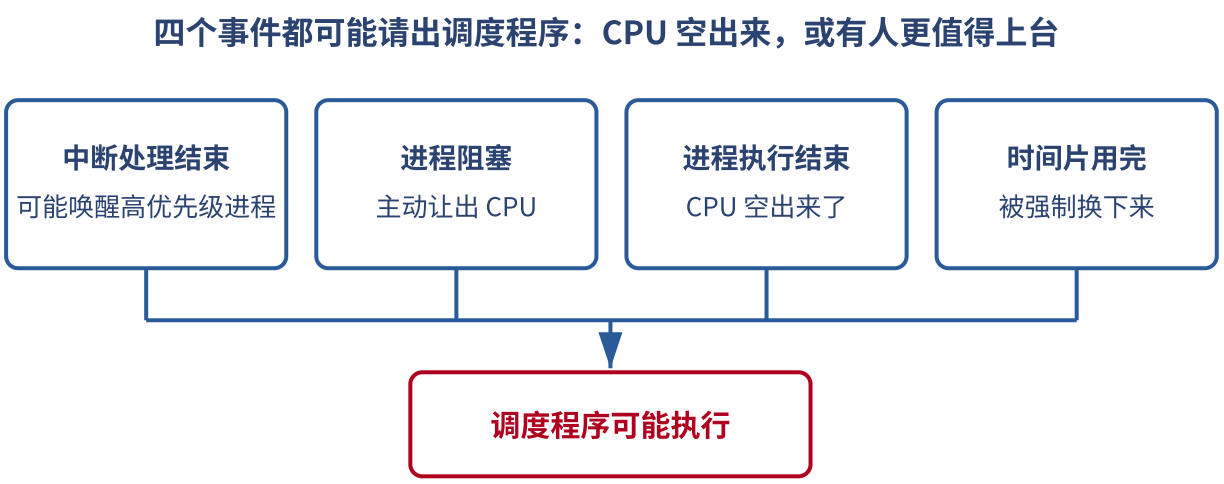
正确选项为什么对： I 可能引起：中断处理结束时，情况可能已经变了。举例：磁盘 I/O 完成的中断唤醒了一个优先级更高的进程，原来那个进程虽然没做错什么，但调度程序可能决定让刚被唤醒的高优先级进程先上。II 可能引起：进程一阻塞就主动让出了 CPU，CPU 空了出来，调度程序必须马上挑一个就绪进程补上，否则 CPU 就空转了。举例：进程发起读磁盘请求后进入阻塞态，调度程序立刻从就绪队列里挑下一个进程上 CPU。III 可能引起：进程结束就等于它占的 CPU 空了出来，调度程序必须挑下一个进程，否则 CPU 同样空转。IV 可能引起：时间片用完是抢占式调度的标准触发点，时钟中断处理程序会调用调度程序，把 CPU 换给下一个就绪进程。四个事件都可能让调度程序执行，答案是 D。

错误选项为什么错＋举例：

- A（仅 I、III）：漏了 II 和 IV。举例：进程等磁盘时阻塞了，CPU 空出来却不调度，新进程就上不去；时间片用完了也不调度，第一个进程就能一直赖在 CPU 上，分时轮转就转不起来了。
- B（仅 II、IV）：漏了 I 和 III。举例：进程正常运行结束退出后，CPU 空了，不调度就没有下一个进程接班；中断处理结束唤醒了高优先级进程，不调度就等于让低优先级进程继续占着 CPU，高优先级进程只能干等。
- C（仅 III、IV）：漏了 I 和 II。举例：中断唤醒高优先级进程是调度程序重新挑人的典型时机，进程阻塞让出 CPU 更是必须马上调度的时刻，这两个漏掉，调度时机就缺了一大半。

笔记本一句话： 第 27 题＋调度程序出场的时机就看两条：当前进程让出或用完 CPU，或者出现了更值得上台的就绪进程；中断结束、阻塞、结束、时间片用完四条全占。

【图像】



⚠ 问的是“可能”不是“一定”：中断结束不每次都换人，但给了重新挑人的机会

四个事件各自的理由都指向调度程序可能执行：中断结束、阻塞、结束、时间片用完

【定位】 操作系统 · 第 2 章 · 进程管理 · 进程调度的时机

【易混对比】

- 中断处理结束和其他三项：阻塞、结束、时间片用完都是"当前进程不能继续占 CPU"；中断处理结束是"可能出现了更值得上台的人"（比如唤醒了高优先级进程）。前者是"旧人下台"，后者是"新人可能上位"。
- "可能引起"和"一定引起"：中断处理结束不每次都换人，但给了调度程序重新挑人的机会；问的是"可能"，有机会就算。
- 进程阻塞和时间片用完：阻塞是进程自己让出 CPU（等事件），时间片用完是被时钟强制换下，一个主动一个被动，但都会请出调度程序。

调度出场的时机就看两条：CPU 空出来了，或有人更值得上台。

【坑位】

看到"可能"别较真成"一定"：中断结束不每次都调度，但只要给了机会就算"可能"。

最容易踩的坑，是跟"中断处理结束"较劲：觉得中断处理完一般都是回原进程，所以不算。但中断处理结束时情况可能已经变了——比如 I/O 完成中断唤醒了一个优先级更高的进程，调度程序就可能让新人先上。题目问的是"可能引起"，只要存在这种机会，这一项就成立。

【套路】

- 第一步，看当前进程还能不能独占 CPU：阻塞、结束、时间片用完 → 不能，调度程序必须出来补人。
- 第二步，看有没有更值得上台的就绪进程出现：中断处理结束可能唤醒高优先级进程 → 调度程序可能重新挑人。
- 第三步，两条占一条就算"可能引起"，四项逐项套，全中的选全选。

旧人下台要补人，新人上位要重挑，两条占一条就出场。

【类比】

像晚会主持人换人上台：台上的嘉宾自己下台了（阻塞/结束）、表演时间到了（时间片用完），主持人必须叫下一个；台下刚到了一位大咖（中断唤醒高优先级进程），主持人也要重新考虑派谁上。四种情况，主持人都要出场重新安排。

【口诀】 人空出来要补人，有人更配要重挑。

【考频】 「进程调度」在 2009–2022 年解析中出现 41 次 · 高频

第 28 题 · 请求分页地址变换（正确答案：C）

题干与选项：

某请求分页系统中，页大小为 4 KB，按字节编址。进程 P 分配了 2 个固定页框，采用改进型 Clock 置换算法。进程 P 访问虚拟地址 02A01H，经地址变换后得到的物理地址是（ ）。

- A. 00A01H
- B. 20A01H
- C. 60A01H
- D. 80A01H

入手点： 分页地址变换只有三步：先按页大小把虚拟地址切成页号和页内偏移，再用页号查页表得到页框号，最后把页框号和页内偏移拼成物理地址；本题的关键一步是查表：页 2 不在内存，查表后还要走缺页＋置换才定下页框号。

题干专有名词从零解释：

- 请求分页系统：进程运行时只把需要的页装入内存，访问到不在内存的页时再按需调入的虚拟存储管理方式。
- 页大小：虚拟地址空间和主存空间划分成页和页框的单位，本题是 4 KB。
- 按字节编址：内存地址每增加 1 就对应 1 个字节，地址里的偏移量直接按字节数算。
- 虚拟地址：程序看到的地址，由页号和页内偏移两部分组成。
- 页号：虚拟地址里指出“这是第几页”的高位部分。
- 页内偏移：虚拟地址里指出“在这一页内的第几个字节”的低位部分，页内偏移在物理地址里原样保留。
- 页表：记录每一页的页框号和存在位等信息的对照表，页号是查这张表的索引。
- 页框号：主存里放一页的格子的编号，物理地址的高位部分就是页框号。
- 存在位：页表里标记该页是否已在内存的位，存在位是 1 表示在内存，存在位是 0 表示不在内存、访问它会触发缺页异常。
- 固定页框：分配给进程的页框个数固定不变，本题是 2 个；页框满了要调新页进来时，必须按置换算法淘汰一页。
- 改进型 Clock 置换算法：Clock 算法的一种改进形式，同时看引用位和修改位，优先淘汰既没被引用过又没被修改过的页；本题的地址变换步骤本身不依赖它，只有当查表发现页不在内存时，置换算法才起作用。

考查知识点： 请求分页地址变换（切地址、查页表、拼地址），页大小与页内偏移位数的关系。

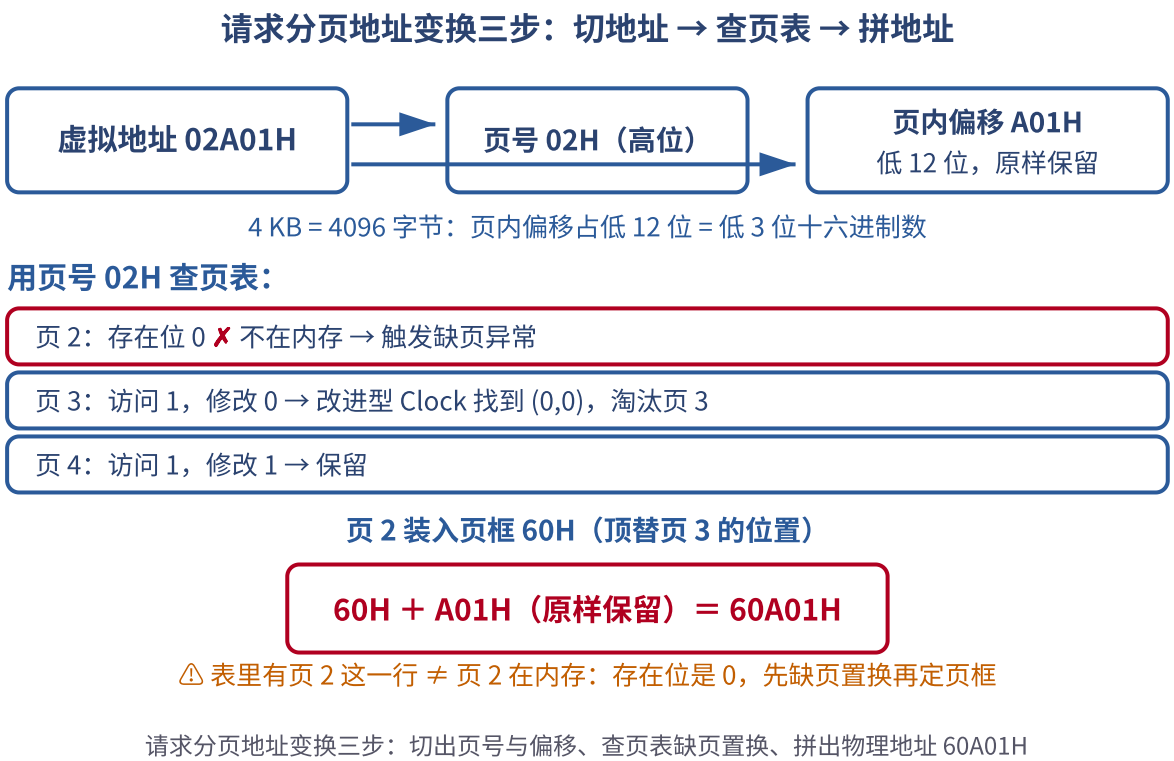
正确选项为什么对： 第一步切地址：页大小 4 KB = 4096 字节 = 2 的 12 次方，所以页内偏移占虚拟地址的低 12 位。虚拟地址 02A01H 写成十六进制是 5 位，低 3 位十六进制数（每位十六进制数对应 4 位二进制，3 位就是 12 位）是 A01H，这就是页内偏移；剩下的高位 02H 就是虚页号。第二步查页表：用虚页号 02H 查进程 P 的页表。原卷页表给出：页号 2 对应页框 20H、存在位 0、访问位 0、修改位 0；页号 3 对应页框 60H、存在位 1、访问位 1、修改位 0；页号 4 对应页框 80H、存在位 1、访问位 1、修改位 1。虚页号 2 的存在位是 0，说明页 2 不在内存，触发缺页：进程只有 2 个固定页框（已装页 3、页 4），必须按改进型 Clock 淘汰一页。改进型 Clock 先找访问位 0、修改位 0 的页：页 3 是（1，0）、页 4 是（1，1），没有（0，0）；再找访问位 0、修改位 1 的页，同时把扫过的访问位清零：页 3 的访问位清为 0、页 4 的访问位清为 0；第二轮再找（0，0），页 3 已是（0，0），淘汰页 3，页 2 装入页框 60H。第三步拼地址：把页框号 60H 放在高位，把页内偏移 A01H 原样放在低 12 位，拼成的物理地址就是 60A01H。结论句：本题的正确选项是 C。

错误选项为什么错＋举例：

- A（00A01H）：这个选项的页内偏移同样是 A01H，差别只在高位的页框号部分。举例：页框号查错一行，或者把页号取错，拼出来的高位就会变成另一个数；只有查对了虚页号对应的那一行，拼出来的高位才和正确选项一致。
- B（20A01H）：同样是页框号部分不同。举例：查表时看串了相邻的一行，或者把页号的十六进制值换算错了，都会让高位落到别的页框号上；地址变换里高位错一位，整个物理地址就指到另一页去了。
- D（80A01H）：同样是页框号部分不同。举例：如果没有按“先切地址、再用页号查表、最后拼偏移”的顺序走，而是直接拿虚拟地址的高位当页框号用，拼出来的地址就会是这类高位不对的选项；正确的顺序必须是查表得到页框号后再拼。

笔记本一句话： 第 28 题＋分页地址变换三步：按页大小切出页号和页内偏移，用页号查页表得页框号，页框号拼上原偏移就是物理地址；页不在内存时改进型 Clock 先找（0，0）页淘汰，本题淘汰页 3、页 2 装入 60H。

【图像】



请求分页地址变换三步：切出页号与偏移、查页表缺页置换、拼出物理地址 60A01H

【定位】 操作系统 · 第 3 章 · 内存管理 · 虚拟内存 · 请求分页地址变换与页面置换

【易混对比】

- 页号和页内偏移：页号是高位、查页表的索引；页内偏移是低位、原样保留拼进物理地址。一个负责"找页"，一个负责"页内定位"。
- 存在位 0 和存在位 1：存在位 1 才能直接用页框号；存在位 0 说明页不在内存，表里有这一行也不顶用，先走缺页加置换。
- 页框号和页号：页框号是主存里格子的编号（查表查出来的），页号是虚拟地址里的编号（查表的索引），别拿虚拟地址的高位直接当页框号用。

页内偏移原样保留，存在位是 0 就先缺页置换。

【坑位】

表里有页 2 这一行，不等于页 2 在内存：先看存在位，是 0 就得先缺页置换再定页框。

最容易踩的坑，是查表看到"页 2 对应页框 20H"就直接拿 20H 去拼，拼出 20A01H。存在位是 0，说明页 2 根本不在内存，页框 20H 是旧记录、不能用。正确做法是先触发缺页：进程只有 2 个固定页框（装着页 3、页 4），按改进型 Clock 淘汰页 3，页 2 装入腾出来的 60H，再用 60H 拼地址。另外，页内偏移是低 12 位，也就是低 3 位十六进制数 A01H，别切错位数。

【套路】

- 第一步，按页大小切地址：4 KB 是 2 的 12 次方，低 12 位是页内偏移（低 3 位十六进制数），剩下高位是页号。
- 第二步，用页号查页表，先看存在位：是 1 直接拿页框号；是 0 先走缺页加置换，定下新页框号再往下走。
- 第三步，页框号放高位、页内偏移原样放低位，拼成物理地址。

切、查、拼三步走，偏移原样留，存在位是 0 先置换。

【类比】

像快递分拣：地址前半是"小区号"（页号），拿去查对照表（页表）找到"楼栋号"（页框号）；后半"门牌号"（页内偏移）原样抄下来。查对照表发现这个小区号还没入住（存在位 0），先按规则腾一间房出来（置换），再把新楼栋号和门牌号拼成完整地址。

【口诀】 切查拼三步走，偏移原样留。

【考频】 「请求分页」在 2009–2022 年解析中出现 35 次 · 高频

第 29 题 · 二级页表基址寄存器内容（正确答案：B）

题干与选项：

在二级页表分页系统中，CPU 的页表基址寄存器中的内容是（）。

- A. 当前进程一级页表的起始虚拟地址
- B. 当前进程一级页表的起始物理地址
- C. 当前进程二级页表的起始虚拟地址
- D. 当前进程二级页表的起始物理地址

入手点： 地址变换是硬件拿着寄存器里的地址去内存里查表；硬件查表用的地址必须是直接能用的物理地址，而且必须先找到一级页表，才能顺藤摸瓜找到二级页表。

题干专有名词从零解释：

- 页表基址寄存器：CPU 里的一个专用寄存器，存放当前进程页表的起始地址，是硬件开始地址变换的起点。
- 二级页表：把页表再分两级管理的结构；一级页表（页目录）的每一项指向一张二级页表，二级页表的每一项才指向一个真正的页框。
- 一级页表：第一级页表，整个进程只有一张，存放在内存里。
- 二级页表：第二级页表，一个进程可以有多张，每张对应一级页表里的一项。
- 起始虚拟地址：某张表在虚拟地址空间里的开头位置，这个地址本身还需要经过地址变换才能找到对应的内存位置。
- 起始物理地址：某张表在主存里的真实开头位置，硬件可以直接按这个地址去内存里读。
- 进程切换：CPU 从一个进程换到另一个进程运行时，操作系统要把页表基址寄存器换成新进程的页表地址，否则新进程会查错表。

考查知识点： 二级页表地址变换的起点，页表基址寄存器的作用，虚拟地址与物理地址在查表起点上的区别。

正确选项为什么对： 硬件做地址变换时，第一步就是按页表基址寄存器里的地址去内存里读一级页表。如果寄存器里放的是虚拟地址，硬件为了读一级页表，先得把这个虚拟地址本身翻译成物理地址，而翻译又要查页表，查页表又要先知道页表在哪，就绕成了一个解不开的圈。所以起点必须是物理地址，硬件才能直接去内存里读。起点也必须是一级页表：二级页表有很多张，分布在不同的地方，只有一个寄存器装不下所有二级页表的开头；只有先读一级页表，才能按一级页表里的项找到本次访问对应的那张二级页表。所以寄存器里放的是当前进程一级页表的起始物理地址，答案是 B。进程切换时，操作系统把这个寄存器换成新进程的一级页表物理地址，新进程的地址变换就从它自己的表开始了。

错误选项为什么错＋举例：

- A（一级页表的起始虚拟地址）：一级页表找对了，但地址性质错了。举例：寄存器里写着一级页表的虚拟地址 0x1000，硬件想读它，先得问“0x1000 对应的物理地址在哪”，而回答这个问题又要查页表，查页表又要回到这个寄存器，死循环转不出来。
- C（二级页表的起始虚拟地址）：两处都错。举例：一个进程可能有几十张二级页表，一个寄存器只能装一个地址，装哪张都不对；而且虚拟地址同样要先翻译，硬件拿着它没法直接去内存里读表。
- D（二级页表的起始物理地址）：地址性质对了，但级别错了。举例：就算寄存器里装着某一张二级页表的物理地址，硬件也不知道本次访问该用哪一张二级页表；只有先查一级页表，才能按虚拟地址里的一级页号挑出对应的二级页表，跳过一级页表就等于跳过了挑表这一步。

笔记本一句话： 第 29 题＋页表基址寄存器装的是当前进程一级页表的起始物理地址：物理地址才能直接查，查一级才能找到对应的二级。

【图像】



二级页表查表链条：寄存器装一级页表起始物理地址，逐级找到页框

【定位】 操作系统 · 第 3 章 · 内存管理 · 分页存储 · 多级页表

【易混对比】

- 一级页表和二级页表：一级只有一张，是查表的总入口；二级有很多张，一个寄存器装不下，必须先查一级才能挑出对的那张。
- 物理地址和虚拟地址：硬件拿着寄存器里的地址直接去内存读表，不用翻译，所以必须是物理地址；虚拟地址还得再翻译一遍，拿它当起点会绕成死循环。
- 级别维度和地址性质维度：A 错在地址性质（一级对了，虚拟错了），D 错在级别（物理对了，二级错了），C 两个维度全错。做题时两个维度分开检查。

起点必须同时满足：物理地址（直接能用）加一级页表（总入口）。

【坑位】

别把两个维度混成一团：先判级别（一级还是二级），再判地址性质（物理还是虚拟）。

最容易踩的坑，是只看对一半就选：看到"一级页表"觉得对了，漏看后面跟的是"虚拟地址"（A）；或者看到"物理地址"觉得对了，漏看前面是"二级页表"（D）。两个维度要分开判：级别上必须是一级（二级太多张，装不下也挑不出），地址性质上必须是物理（硬件直接读，不翻译）。

【套路】

- 第一步，问硬件拿着它要不要翻译：直接去内存读表，不翻译 → 必须是物理地址，排除 A、C。
- 第二步，问一张表够不够当起点：二级页表有很多张，一个寄存器装不下 → 必须是一级页表，排除 C、D。
- 第三步，两步都通过的只剩 B；顺带记一句：进程切换时操作系统换掉这个寄存器的内容。

物理才能直接读，一级才能找到二级。

【类比】

像图书馆的总目录柜：管理员手里拿的必须是总目录柜在馆内的真实位置（物理地址），不能拿"三楼 B 区"这种还得再问路的说法（虚拟地址）；而且必须先看总目录（一级页表），才知道你要的那本书在哪个分目录柜（二级页表）里——分目录柜有几十个，一只手拿不下。

【口诀】 物理才能直接读，一级才能找到二。

【考频】 「页表基址寄存器」在 2009–2022 年解析中出现 15 次 · 高频

第 30 题 · 删除文件内核不必完成的工作（正确答案：A）

题干与选项：

目录 dir 下有文件 file1。为删除该文件，内核不必完成的工作是（）。

- A. 删除 file1 的快捷方式
- B. 释放 file1 的文件控制块
- C. 释放 file1 占用的磁盘空间
- D. 删除目录 dir 中与 file1 对应的目录项

入手点： 本题问的是“不必”，先把删除文件本身必须清理的三样东西（名字、元数据、数据块）列出来，再看哪个选项清理的是文件之外的另一个独立东西。

题干专有名词从零解释：

- 目录 dir：存放文件名与文件对应关系的文件夹，本题里 file1 就放在这个目录下。
- 文件 file1：要删除的目标文件，包括它的名字、元数据和数据内容。
- 快捷方式：指向某个文件的一个独立小文件或链接，双击它会打开它指向的目标文件；它本身不是目标文件，只是一张写着目标文件位置的纸条。
- 文件控制块（FCB）：操作系统为每个文件建立的管理信息块，记录文件名、大小、存放位置、权限等元数据。
- 磁盘空间：文件数据实际占用的磁盘块，文件删除后这些块应当还给系统，供别的空闲文件使用。
- 目录项：目录里的一条记录，记录一个文件名和它对应的文件控制块之间的对应关系；用户在目录里看到一个文件名，就是因为目录里有这样一条目录项。

考查知识点： 文件删除时内核要完成的工作，快捷方式与目标文件的关系。

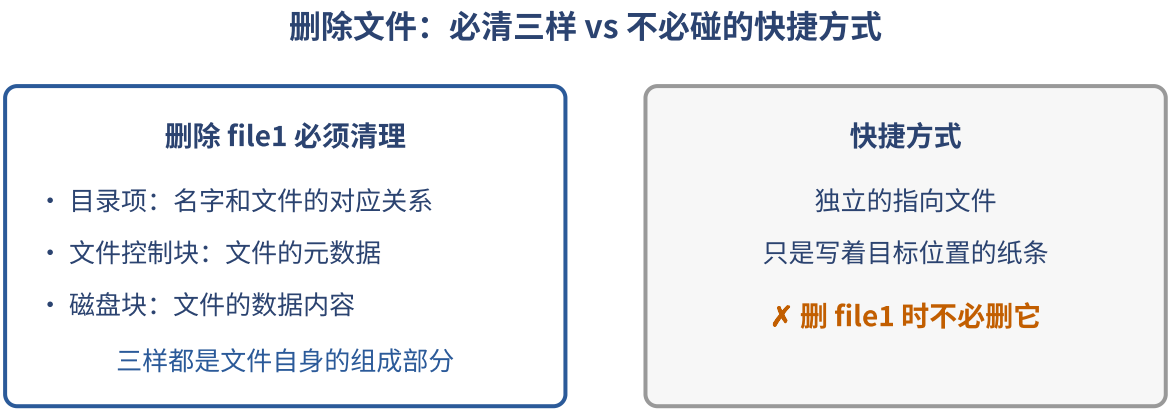
正确选项为什么对： A 是不必完成的工作。快捷方式是独立于 file1 的另一个文件，它只是指向 file1。删除 file1 时，内核要清理的是 file1 自身的名字、元数据和数据块，快捷方式并不属于 file1 的组成部分。举例：桌面上有一个指向某篇文档的快捷方式，删掉那篇文档本身，桌面上的快捷方式图标还留在那里，只不过双击它会因为找不到目标而打不开；要删掉这个快捷方式，需要用户另外单独删除它，内核在删除目标文件时不会顺手把它也删掉。所以删除 file1 的快捷方式不是内核删除 file1 时必须完成的工作。

错误选项为什么错＋举例：

- B（释放 file1 的文件控制块）：这是必须做的。举例：文件控制块是记录 file1 元数据的管理块，不释放它，这块管理信息就一直占着位置，系统以后也没法把这个管理块分配给新文件，等于管理信息泄露了。
- C（释放 file1 占用的磁盘空间）：这是必须做的。举例：file1 的数据占了 10 个磁盘块，删文件却不把这 10 个块还给空闲块管理结构，磁盘空间就白白少了 10 块，别的用户再存文件时明明有空块却用不上。
- D（删除目录 dir 中与 file1 对应的目录项）：这是必须做的。举例：目录项就是“file1 这个名字对应哪个文件”的记录，不删它，用户打开目录还是能看到 file1 这个名字，点进去却找不到对应的文件，目录和实际文件就对不上了。

笔记本一句话： 第 30 题＋删文件必清三样：目录项（名字）、文件控制块（元数据）、磁盘块（数据）；快捷方式是独立的指向文件，删目标时不必顺手删它。

【图像】



删除文件必清三样对照快捷方式：目录项、文件控制块、磁盘块必须清，快捷方式是独立文件

【定位】 操作系统 · 第 4 章 · 文件管理 · 文件的删除

【易混对比】

- 目录项和快捷方式：目录项是"文件名和文件的对应关系"，是文件名字本身的登记，删文件必删；快捷方式是独立于目标文件的另一个文件，只是写着目标位置的纸条，删目标不必删它。
- 文件自身的三样东西和文件之外的东西：目录项（名字）、文件控制块（元数据）、磁盘块（数据）是文件自身的组成部分，删文件必须清理；快捷方式是文件之外的独立东西，不归这次删除管。
- "删除文件"和"删除快捷方式"：删文件清的是文件自己；删快捷方式是另一个独立操作，要用户单独做，内核删文件时不会顺手做。

删文件清三样：名字、元数据、数据块；文件之外的东西不动。

【坑位】

题问的是"不必"：先列出必须清的三样，再看哪个选项清理的是文件之外的独立东西。

最容易踩的坑，是被"快捷方式好像和 file1 有关"带偏，觉得删文件就该把相关的都清掉。但快捷方式是一个独立的文件，它指向 file1，而不是 file1 的一部分。删 file1 时，桌面上的快捷方式图标还留在那儿（双击会因为找不到目标而打不开），要删它得用户另外单独删。做"不必 / 不……"的题，先把"必须"列全，剩下的那个就是答案。

【套路】

- 第一步，列出删除文件必须清理的三样：目录项（名字）、文件控制块（元数据）、磁盘块（数据）。
- 第二步，逐个看选项清理的是不是这三样：B、C、D 分别对上三样，都是必须的。
- 第三步，找出清理"文件之外独立东西"的选项：快捷方式是独立文件 → A 是不必的。

先列必须，再找例外，问"不必"就这么做。

【类比】

像搬家退房：必须清的是门牌登记（目录项）、租房合同（文件控制块）、屋里的东西（磁盘块）；朋友手机里存的你家地址（快捷方式）不用你去删——那是朋友自己的东西，你退房不影响它存在，只不过按那个地址找过来会发现人去楼空。

【口诀】 删文件清三样，快捷方式不用管。

【考频】 「文件删除」在 2009–2022 年解析中出现 8 次 · 中频

第 31 题 · 不死锁所需资源数至少（正确答案：C，n＋1）

题干与选项：

系统中有 n（n≥2）个进程，每个进程均需要某类临界资源 2 个。系统不死锁所需该类资源总数至少是（ ）。

- A. 2
- B. n
- C. n＋1
- D. 2n

入手点： “至少”要两面证：先构造一个资源数不够时必然死锁的场面，再证明多给一个资源后死锁场面就再也摆不出来；只证一面，答案就站不住。

题干专有名词从零解释：

- 临界资源：一次只允许一个进程使用的资源，例如一台打印机、一段独占的内存；两个进程不能同时用同一份临界资源。
- 进程：正在运行的程序的一次执行过程，本题有 n 个进程，每个进程跑完都需要拿到 2 个该类资源。
- 死锁：一组进程互相等待对方手里的资源、谁也走不下去的状态。
- 持有并等待：进程手里已经拿着一部分资源，同时还在等剩下的资源，这是死锁的典型姿势。
- 资源总数：系统里该类临界资源一共的份数，本题要求的是保证不死锁的最少份数。

考查知识点： 死锁的必要条件与资源数下界，最坏情况构造法。

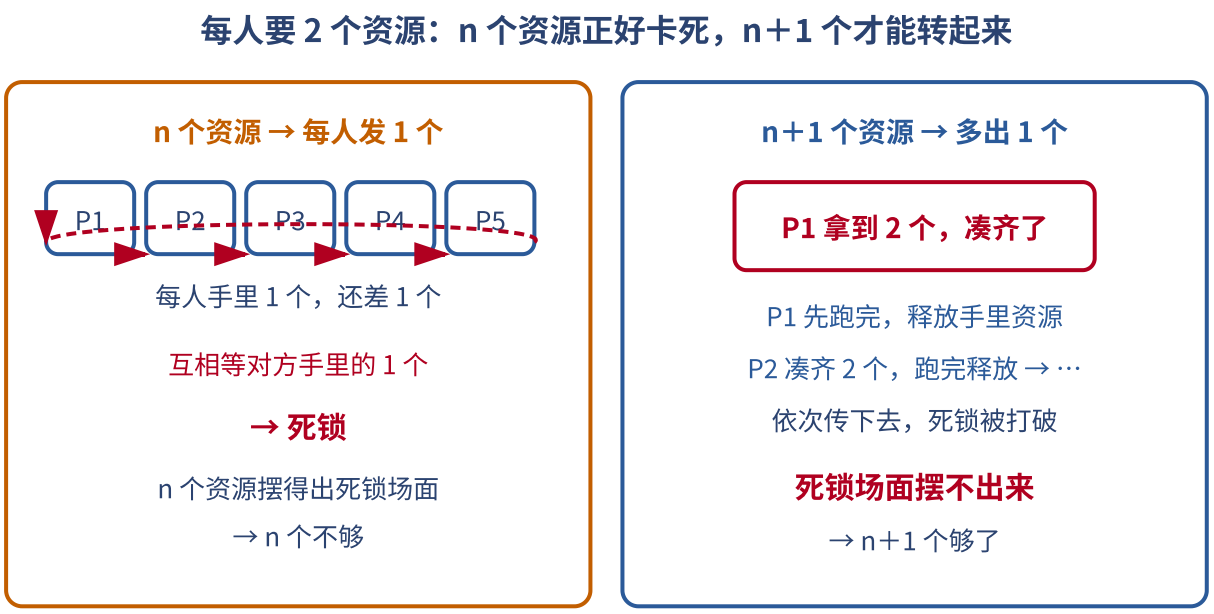
正确选项为什么对： 先看 n 个资源为什么不够：把 n 个资源每人发 1 个，n 个进程每人手里 1 个、每人还差 1 个才能跑完，所有进程都在等别人手里的那 1 个，谁也不肯先放手，这就是死锁。所以 n 个资源摆得出死锁场面，n 个不够。再看 n＋1 个资源为什么够：假设出现了死锁，那么每个被卡住的进程手里至少拿着 1 个资源（手里 1 个都没有的进程可以直接拿空闲资源，不会被卡住），n 个进程手里至少一共拿着 n 个资源；系统一共 n＋1 个资源，拿走至少 n 个之后还至少剩下 1 个空闲资源；被卡住的进程里只要有一个再拿到这 1 个，就凑够 2 个、可以跑完并释放它手里的资源，释放出来的资源又能让下一个进程凑够，依次传下去，死锁就被打破了。所以死锁场面在 n＋1 个资源下摆不出来，n＋1 个足够。n 个不够、n＋1 个足够，最少就是 n＋1，答案是 C。

错误选项为什么错＋举例：

- A（2 个）：n≥2 时，2 个资源连每人发 1 个都不够。举例：n = 5 个进程，2 个资源最多让 1 个进程凑够 2 个跑完，剩下的进程连 1 个都分不到，更不用说凑够 2 个；资源数这么少，死锁场面很容易摆出来。
- B（n 个）：正好是上面构造出的死锁场面。举例：n = 5 时有 5 个资源，5 个进程每人拿 1 个、每人等第 2 个，5 个人互相等，谁也跑不完，死锁。
- D（2n 个）：够是够，但不是最少。举例：n = 5 时 2n = 10 个资源，5 个进程可以每人同时拿 2 个、一起跑完，确实不死锁；但题目问的是“至少”，n＋1 = 6 个已经够了，多给的 4 个是浪费，不是下界。

笔记本一句话： 第 31 题＋每人要 2 个资源时，n 个正好够每人卡 1 个等第 2 个，n＋1 个多出的 1 个能让一个人先跑完、释放后依次传下去。

【图像】



每人要 2 个资源时 n 个资源卡死、n＋1 个资源转起来的对照图

【定位】 操作系统 · 第 2 章 · 死锁的必要条件与资源数下界

【易混对比】

- "够用"和"最少够用"是两回事：2n 个资源确实够用，但题目问的是"至少"，多给的资源是浪费，不是最下界。
- n 个资源和 n＋1 个资源差的不是"多 1 个更宽裕"：n 个正好能摆出"每人拿 1 个、每人等 1 个"的死锁场面，多出的那 1 个恰好能打破这个场面，差 1 个就是死锁与不死锁的分水岭。
- "每个进程需要 2 个"和"每个进程手里拿着 1 个"是两回事：前者是进程跑完的条件，后者是死锁场面里每个进程被卡住时的姿势，构造死锁场面时用的是后者。

n 个资源正好够每人卡 1 个等第 2 个，多出的 1 个才是打破死锁的钥匙。

【坑位】

看到"至少"二字，一定要两面证：既要摆出少 1 个就死锁的场面，也要证明多 1 个死锁就摆不出来。

最容易踩的坑是只证一面：有人算出 n＋1 够用就直接选，忘了题目问的是"最少"；也有人摆出了 n 个资源死锁的场面就直接选 B，忘了"摆得出死锁"只证明 n 个不够，还要证明 n＋1 够。两个方向缺一不可，少证一面，答案就站不住。

【套路】

- 第一步，摆死锁场面：把资源数设成 n，每人发 1 个，n 个进程每人手里 1 个、每人还差 1 个，互相等 → 死锁，证明 n 个不够。
- 第二步，反设死锁：假设在 n＋1 个资源下出现了死锁，被卡住的进程手里至少各拿 1 个，一共至少拿走 n 个。
- 第三步，找空闲资源：系统一共 n＋1 个，拿走至少 n 个，还剩至少 1 个空闲；随便一个被卡住的进程拿到它就凑齐 2 个。
- 第四步，凑齐的进程跑完释放手里资源，下一个进程接着凑齐，依次传下去，死锁被打破 → n＋1 够。

n 个不够、n＋1 个够，"至少"就是 n＋1。

【类比】

像 n 个人分 n 把钥匙开 n 扇门，每扇门要 2 把钥匙才能开。每人先拿 1 把，剩下的人都捏着 1 把等别人手里的第 2 把，谁也不肯先松手，全卡住了。这时候多拿来 1 把钥匙，随便给谁，他就能开门，门一开他手里的钥匙还回来，下一个人接着开，一扇一扇全开。多出来的这 1 把钥匙，就是让全场转起来的那一把。

【口诀】 每人卡一个，多一破死锁。

【考频】 「死锁」在 2009–2022 年解析中出现 15 次 · 高频

第 32 题 · 通过系统调用完成的操作（正确答案：C）

题干与选项：

下列操作中，通过系统调用完成的是（）。

- A. 页置换
- B. 进程调度
- C. 创建新进程
- D. 生成随机整数

入手点： 系统调用是用户程序请内核帮忙的正规入口；判断时只问一句：这件事是用户程序主动向内核要一项服务，还是内核自己在内部运转时顺手做的事，或者用户态自己就能算出来的事。

题干专有名词从零解释：

- 系统调用：用户程序请求操作系统内核提供服务的入口，例如读文件、创建进程；用户程序执行 trap 指令陷入内核，内核执行完服务再返回用户态。
- 页置换：内存页框满了、要调入新页时，按某种算法挑出一页淘汰出内存的操作。
- 进程调度：操作系统挑选下一个上 CPU 运行的进程的操作。
- 创建新进程：产生一个新进程的操作，包括分配进程控制块、分配内存和资源、登记到进程管理结构里。
- 生成随机整数：算出一个随机数（通常是伪随机数）的操作，例如调用随机数库函数得到一个数。
- 用户态：普通应用程序运行的模式，只能执行非特权指令，不能直接分配内核管理结构。
- 内核态：操作系统内核运行的模式，能执行特权指令、管理进程和内存。

考查知识点： 系统调用的概念，系统调用与操作系统内部活动、用户态库函数的区别。

正确选项为什么对： 创建新进程必须通过系统调用。创建进程要分配进程控制块、分配内存、登记资源，这些都是内核才能动的管理结构，用户程序在用户态没有权限直接改它们。举例：一个程序想启动一个子进程干活，它自己不能凭空变出一张进程控制块，只能执行创建进程的系统调用（例如 fork），请内核分配好新进程的管理结构并把它登记进系统，新进程才能存在。所以创建新进程是通过系统调用完成的操作。

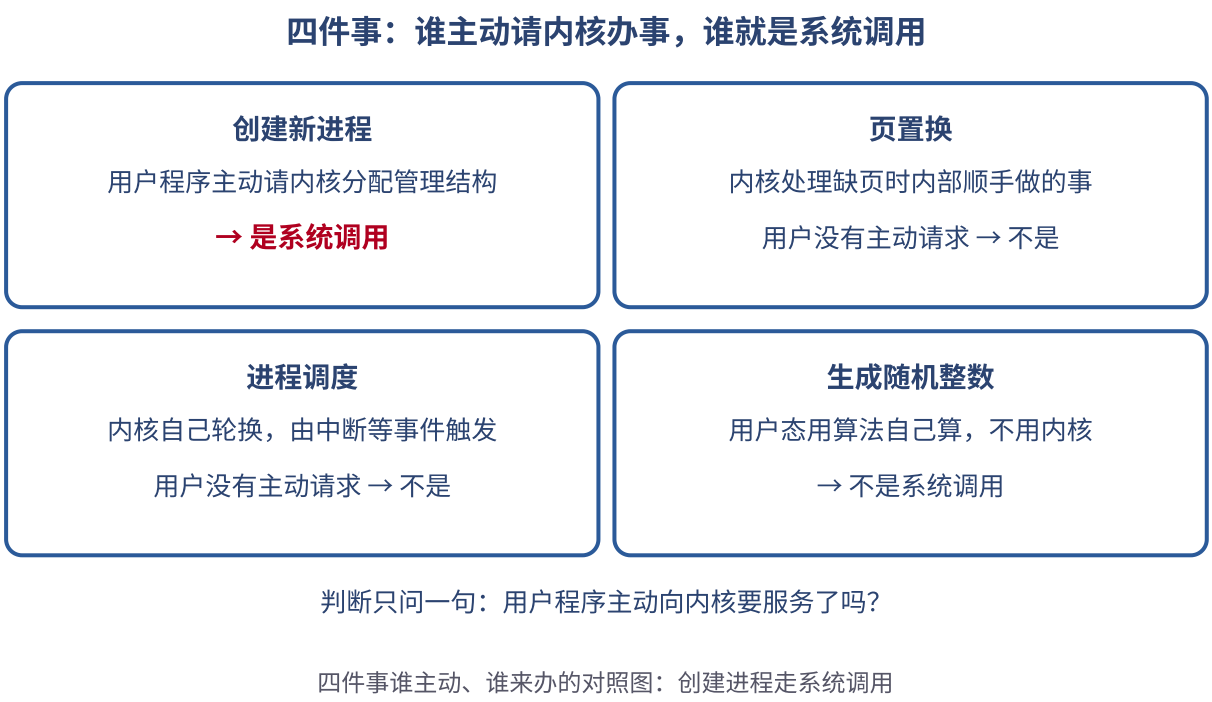
错误选项为什么错＋举例：

- A（页置换）：页置换是内核在处理缺页异常时的内部活动，不是用户程序主动请求的服务。举例：程序访问了一个不在内存的页，硬件触发缺页异常，内核接管后发现页框满了，就挑一页淘汰、把需要的页调进来；整个过程是用户程序访问内存时被动触发的，程序并没有发出“我要页置换”这样的请求。
- B（进程调度）：进程调度是内核在内部运转时做的挑选工作，由时钟中断、进程阻塞或进程结束等事件触发，不是用户程序通过系统调用要来的服务。举例：一个进程的时间片用完了，时钟中断处理程序调用调度程序换下一个进程上 CPU；这个换人动作是内核自己安排的，用户程序并没有调用一个“请调度”的系统调用。
- D（生成随机整数）：生成随机整数可以在用户态用算法算出来，不必陷入内核。举例：随机数库函数按一个种子数经过确定性的数学运算算出下一个数，整个计算过程只在用户程序自己的内存里进行，不需要分配内核资源、不需要特权指令，所以它只是一个普通的库函数调用，不是系统调用。

笔记本一句话： 第 32 题＋系统调用是用户请内核办事的入口：创建进程要内核分配管理结构，必须走系统调用；页置换和进程调度是内核内部活动，随机数用户态自己就能算。

补充一句：分不清时就看发起方，用户主动请内核分配或登记新资源的就是系统调用，内核自己轮换或换页的就是内部活动。

【图像】



四件事谁主动、谁来办的对照图：创建进程走系统调用

【定位】 操作系统 · 第 1 章 · 系统调用与内核内部活动、用户态库函数的区别

【易混对比】

- 系统调用和内核内部活动是两回事：系统调用是用户程序主动向内核要服务，有明确的"请求"动作；内核内部活动是内核自己运转时做的事，没有用户在请求。页置换和进程调度都属于后者。
- 创建新进程和生成随机整数也是两回事：创建进程要动内核才能分配的管理结构（如进程控制块），用户态没权限自己办；生成随机整数只是在用户自己的内存里算个数，用不着内核。
- "用户程序引发了内核活动"和"用户程序发起了系统调用"也是两回事：程序访问缺页时确实引发了内核做页置换，但那是被动触发的，不是程序主动请内核办事。

有没有用户主动"请内核办事"这个动作，是不是系统调用的分水岭。

【坑位】

别把"内核参与了"当成"走了系统调用"。

最容易踩的坑，是看到页置换、进程调度都有内核参与，就以为它们是通过系统调用完成的。内核参与和系统调用是两码事：系统调用是用户程序主动发起的请求，内核参与只是说这件事发生在内核里。页置换是程序访问内存时被动触发的，进程调度是内核自己轮换的，两者都没有"用户主动请"这个动作。判断时只问一句：这是用户程序主动向内核要的一项服务吗?

【类比】

像去银行办业务。创建新进程，好比开一个新账户：必须到柜台（内核）办手续，自己在家写张纸条不算数，这就是系统调用。页置换，好比银行内部整理档案柜：柜员（内核）自己腾地方，你没让它这么干，它是顺手做的。进程调度，好比银行叫号：下一个轮到谁，是银行自己安排的。生成随机整数，好比你在家掷骰子：自己就能玩，不用去银行。

【口诀】 用户请客走调用，内核自家事不算。

【考频】 「系统调用」在 2009–2022 年解析中出现 23 次 · 高频

第 33 题 · TCP/IP 传输层相邻下一层的功能（正确答案：B）

题干与选项：

TCP/IP 参考模型中，由传输层相邻的下一层实现的主要功能是（ ）。

- A. 对话管理
- B. 路由选择
- C. 端到端报文段传输
- D. 结点到结点流量控制

入手点： 先把 TCP/IP 的四层从上到下排好，找出传输层下面紧挨着的那一层，再看那一层的主业是什么，不要直接凭选项里的词猜。

题干专有名词从零解释：

- TCP/IP 参考模型：把网络功能从上到下分成四层，分别是应用层、传输层、网际层、网络接口层，上一层使用下一层提供的服务。
- 传输层：负责端到端的通信，代表协议是 TCP 和 UDP，数据单位叫报文段。
- 相邻的下一层：对传输层来说，就是紧挨在它下面的网际层，不是隔了一层的网络接口层。
- 网际层：负责把分组从源主机送到目的主机，代表协议是 IP，数据单位叫数据报。
- 路由选择：路由器根据目的地址决定下一跳往哪里走，从众多路径中选出一条转发路径。
- 对话管理：管理一次会话的建立、维持和结束，属于 OSI 模型中会话层的功能说法。
- 端到端：从源端主机到目的端主机的全程，不是指相邻两个结点之间的一段。
- 结点到结点：相邻两个结点之间的一段传输，例如一台主机到直连的路由器之间的一段。

考查知识点： TCP/IP 四层结构、网际层的主要功能、端到端与结点到结点的区分。

正确选项为什么对： 传输层下面紧挨着的是网际层。网际层的主业是把 IP 数据报从源主机送往目的主机，途中每个路由器都要做一次路由选择：查看目的 IP 地址，对照路由表，决定把数据报从哪个接口转发给下一个路由器。举例：主机 A 要把数据发给远方的主机 B，中间经过路由器 R1、R2，R1 收到数据报后查路由表决定发往 R2，R2 再决定发往 B，这一步一步的下一跳选择就是路由选择。所以 B 是网际层实现的主要功能。

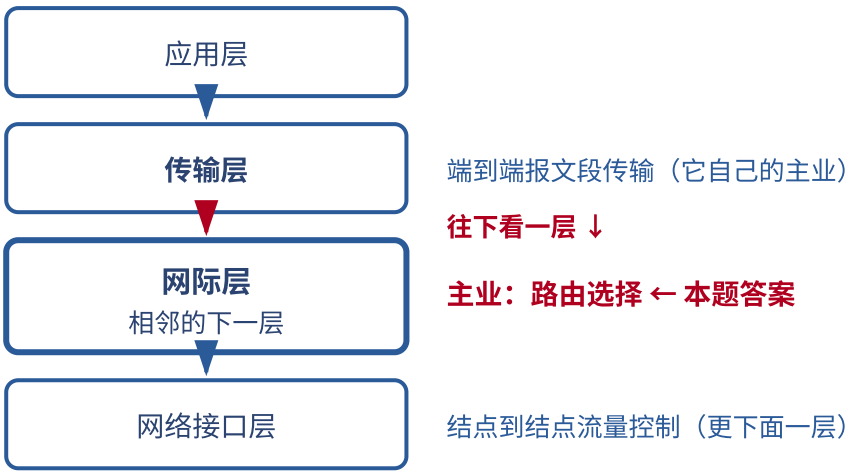
错误选项为什么错＋举例：

- A（对话管理）：对话管理是 OSI 会话层的说法，TCP/IP 模型没有单独的会话层，网际层也不管会话的建立和结束。举例：两个人打电话时约好先说什么、后说什么，那是会话层面的事；路由器只管把分组往下一跳转发，不管这次通话进行到哪一步。
- C（端到端报文段传输）：这是传输层自己的主业，不是它下一层的主业。举例：TCP 把应用层的数据切成报文段、编号、重传、按序交给目的进程，这些都是传输层在做；网际层只负责把装着报文段的数据报在网络中转发，不负责端到端的可靠交付。
- D（结点到结点流量控制）：结点到结点的控制属于更下面的网络接口层（数据链路层的功能），管的是相邻两个结点之间一段链路上的收发节奏，不是网际层的主业。举例：主机把一帧发给直连的路由器时，路由器来不及收就让主机慢一点，这是相邻两点之间的事；网际层的路由选择管的是全程走哪条路，两者层次不同。

笔记本一句话： 第 33 题＋传输层下面是网际层，主业是路由选择；端到端是传输层自己，结点到结点是更下面的链路层。

【图像】

TCP/IP 四层：传输层下面紧挨的是网际层



对话管理是 OSI 会话层的说法，TCP/IP 四层里没有这一层

TCP/IP 四层楼梯图：传输层下面紧挨着网际层，主业路由选择

【定位】 计算机网络 · 第 1 章 · TCP/IP 参考模型与网际层的主要功能

【易混对比】

- "传输层相邻的下一层"和"传输层自己"是两回事：端到端报文段传输是传输层自己的主业，不是它下一层的主业，看到"端到端"先别急着选。
- 网际层和网络接口层也是两回事：网际层管全程走哪条路（路由选择），网络接口层管相邻两点之间一段链路上的收发，"结点到结点"属于后者。
- 对话管理是 OSI 会话层的说法：TCP/IP 四层里没有单独的会话层，网际层更不管会话的建立和结束。

先排层、再找紧挨的下一层、最后看那一层的主业，不要凭选项里的词直接猜。

【坑位】

"端到端"这个词是本题最大的烟雾弹。

最容易踩的坑，是看到选项里有"端到端"就条件反射选它。端到端确实是传输层的关键词，但题目问的是传输层"相邻的下一层"实现的功能，也就是网际层的主业。传输层自己的主业（端到端报文段传输）不能拿来回答下一层的问题。做这类题先把四层从上到下排好：应用层、传输层、网际层、网络接口层，传输层下面紧挨的是网际层，网际层的主业是路由选择。

【套路】

- 第一步，把 TCP/IP 四层从上到下排好：应用层、传输层、网际层、网络接口层。
- 第二步，找到传输层下面紧挨着的那一层：是网际层，不是隔了一层的网络接口层。
- 第三步，看网际层的主业是什么：把 IP 数据报从源主机送到目的主机，途中每个路由器做路由选择。
- 第四步，把"端到端"（传输层自己）、"结点到结点"（更下面的链路层）、"对话管理"（OSI 会话层的说法）逐个排除。

传输层下面是网际层，网际层的主业是路由选择。

【类比】

像寄快递。传输层好比打包发货的人，负责把东西装箱、贴单、保证整单送到（端到端）；网际层好比快递分拣中心，负责看地址、决定包裹下一站走哪条线路（路由选择）；网络接口层好比开货车的司机，只管相邻两个站点之间这一段路（结点到结点）。问"打包发货的人下面一层是干什么的"，答案是分拣中心看地址选路，不是打包本身。

【口诀】 传输层下住网际，路由选择是主业。

【考频】 「网际层」在 2009–2022 年解析中出现 20 次 · 高频

第 34 题 · 差分曼彻斯特编码的位串（正确答案：A）

题干与选项：

若下图为一段差分曼彻斯特编码信号波形，则其编码的二进制位串是（ ）。

- A. 1011 1001
- B. 1101 0001
- C. 0010 1110
- D. 1011 0110

入手点： 差分曼彻斯特的解码不看电平本身是高还是低，只看两个位置有没有跳变：每个码元中间必有跳变，码元起始处有无跳变才区分 0 和 1。

题干专有名词从零解释：

- 码元：传送一位二进制数所占用的时间段，一个码元对应一位 0 或 1。
- 电平：信号在某一时刻的电压高低，通常画成高电平和低电平两条横线。
- 跳变：电平从高变到低，或者从低变到高，在波形图上是一段竖线。
- 曼彻斯特编码：每个码元中间都有一次跳变，中间的跳变既传送数据又提供时钟同步。
- 差分曼彻斯特编码：在曼彻斯特编码的基础上，用码元起始处有无跳变来区分 0 和 1，而不是用电平的高低或者中间跳变的方向来区分。
- 位串：按时间顺序排好的 0、1 序列，本题一共 8 个码元，所以位串一共 8 位。

考查知识点： 差分曼彻斯特编码的规则、码元中间跳变与起始处跳变的分工。

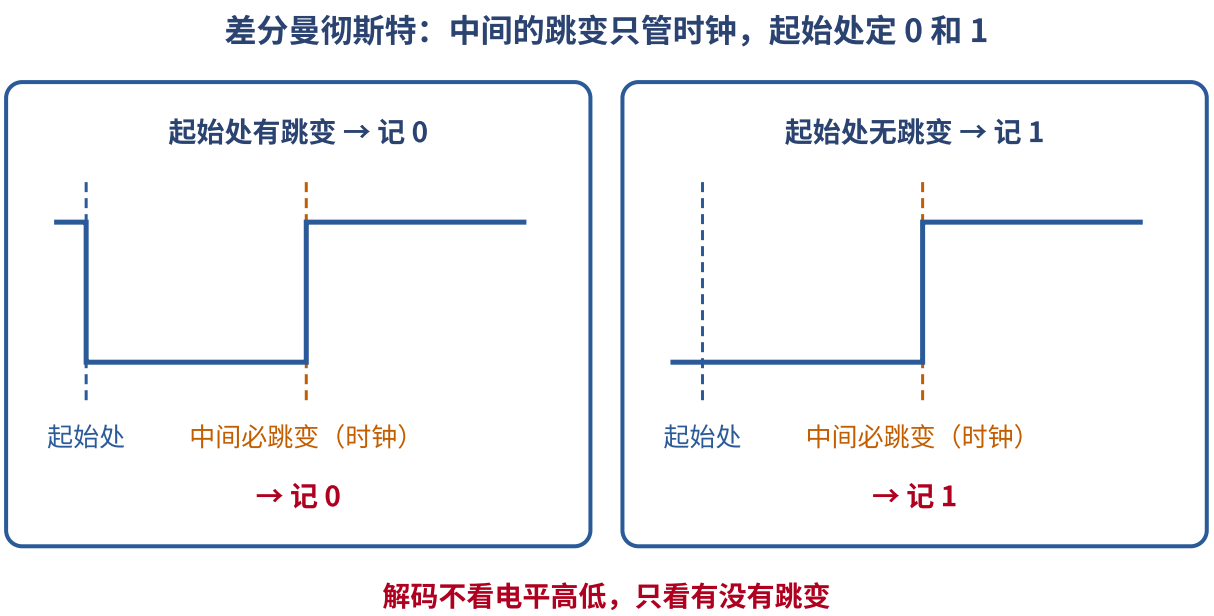
正确选项为什么对： 差分曼彻斯特的解码方法分三步。第一步，把波形按时间切成 8 个码元段，每段对应一位。第二步，检查每个码元的中间位置：中间必有一次跳变，这一步只用来确认码元的边界和时钟，不区分 0 和 1。第三步，看每个码元起始处有没有跳变：起始处有跳变表示 0，起始处没有跳变表示 1。把 8 个码元按这个规则逐个判读，得到的位串就是 1011 1001。原卷波形核对情况：在在线预览中，第 1、2 码元可以辨认（起始电平均为高、码元中部均有高→低跳变、码元边界均有低→高跳变）；第 3—8 码元在放大至 240%—330% 后虚线与实线跳变位置仍无法可靠区分，未能逐码元核对。[待我终审核对：差分曼彻斯特波形第 3—8 码元电平细节]。也就是说，规则和判读方法是完整的，结论以原卷解析答案表为准，第 3—8 码元起始处有无跳变的具体对照仍待更清晰的原卷图核对后落定。

错误选项为什么错＋举例：

- B（1101 0001）：只要有一位判错，整个位串就不是正确答案。举例：第 2 位上，A 是 0，B 是 1，两者正好相反，说明其中一个把该码元起始处有无跳变看反了；看反一位，答案就从 A 变成了 B。
- C（0010 1110）：这个位串与 A 有多位不同。举例：第 1 位上 A 是 1，C 是 0，起手第一位就不同；只要第一位判错，后面再对也救不回整个位串。
- D（1011 0110）：前 4 位与 A 相同，后 4 位不同。举例：前半段判对不代表全对，后半段里只要有一位把起始处有无跳变看反，位串就从 1011 1001 变成了 1011 0110，所以必须 8 位逐位判读，不能只看一半。

笔记本一句话： 第 34 题＋差分曼彻斯特：中间必跳变只管时钟，起始处有跳变是 0、无跳变是 1，8 位逐位判读。

【图像】



差分曼彻斯特解码规则图：中间必跳变只管时钟，起始处有跳变是 0、无跳变是 1

【定位】 计算机网络 · 第 2 章 · 差分曼彻斯特编码的解码规则

【易混对比】

- 中间的跳变和起始处的跳变分工不同：中间的跳变每个码元都有，只管提供时钟、确认码元边界，不区分 0 和 1；只有起始处有无跳变才区分 0 和 1。
- 电平是高还是低，和 0/1 没关系：解码只看"有没有跳变"这一个动作，不看跳变之前是高电平还是低电平，盯着电平高低判读会走偏。
- 起始处"有跳变"对应的是 0、"无跳变"对应的是 1：方向别记反——有动作的是 0，没动作的是 1。

解码只看两个位置：中间确认边界，起始处定 0/1。

【坑位】

千万别用电平的高低判读，也别把 0/1 的方向记反。

最容易踩的坑有两个。第一，盯着波形是高电平还是低电平去猜 0/1——差分曼彻斯特里电平高低本身不携带数据，看的是"有没有跳变"这个动作。第二，把规则记反：起始处有跳变是 0、无跳变是 1，有人凭感觉写成"有跳变是 1"。记住口诀里的顺序：有跳是零、无跳是一，一个码元一个码元按顺序判，8 个码元逐位判完再拼位串。

【套路】

- 第一步，把波形按时间切成 8 个码元段，每段对应一位。
- 第二步，检查每个码元中间位置的跳变：中间必有一次跳变，这一步只确认码元边界和时钟，不区分 0 和 1。
- 第三步，看每个码元起始处有没有跳变：有跳变记 0，无跳变记 1。
- 第四步，8 位逐位判读、按顺序拼成位串；只看一半就下结论，错一位就全错。

中间的跳变不管数，起始处有跳变是 0、无跳变是 1。

【类比】

像看交接班记录。每班（码元）中间都敲一下钟（中间必跳变），只为告诉你"这一班到这儿"，不记录干了什么；真正记录事情的是交接班那一刻：交接时换了人（起始处有跳变）记 0，没换人（起始处无跳变）记 1。你要统计的是"换没换人"，不是"钟敲得响不响"，更不是"上一班是白班还是夜班"（电平高低）。

【口诀】 中间跳变只管钟，起始有跳是零、无跳是一。

【考频】 「差分曼彻斯特」在 2009–2022 年解析中出现 7 次 · 中频

第 35 题 · 子网划分中不可能的子网（正确答案：B）

题干与选项：

某 IP 网络划分为 3 个子网，若其中一个子网是 192.168.9.128/26，则下列网络中不可能是另外两个子网之一的是（）。

- A. 192.168.9.0/25
- B. 192.168.9.0/26
- C. 192.168.9.192/26
- D. 192.168.9.192/27

入手点： 先把每个选项换算成地址范围，看它和已知子网重不重叠，再看“恰好 3 个子网”这个总数能不能凑出来；只看不重叠是不够的。

题干专有名词从零解释：

- IP 地址：点分十进制写成的 32 位地址，本题前三段都是 192.168.9，只看最后一段的变化。
- 斜线记法：/26 表示前 26 位是网络位和子网位，后 6 位是主机位；/25 表示后 7 位是主机位；/27 表示后 5 位是主机位。
- 块大小：一个子网包含的地址个数，等于 2 的主机位数次方。/26 的主机位是 6 位，块大小是 2^6 = 64；/25 的块大小是 2^7 = 128；/27 的块大小是 2^5 = 32。
- 子网地址：一个子网里最小的那个地址，也是该子网的编号，例如 192.168.9.128/26 的地址范围是 128 到 191。
- 子网划分：把一个较大的地址块切成若干个较小的子网，每个地址只能属于一个子网，子网之间不能重叠。
- 恰好 3 个子网：切完之后一共只有 3 块，不能多也不能少，这是本题最关键的限制条件。

考查知识点： 可变长子网掩码的地址块计算、子网之间不重叠、划分子网总数的约束。

正确选项为什么对： 先算已知子网的范围：192.168.9.128/26 的块大小是 64，范围是 128 到 191。再算 B 的范围：192.168.9.0/26 的块大小也是 64，范围是 0 到 63。B 和已知子网确实不重叠，但问题出在总数上。已知子网占了 128 到 191，B 占了 0 到 63，剩下的地址是两段互不相连的块：一段是 64 到 127（64 个地址），另一段是 192 到 255（64 个地址）。这两段不能合成一个子网：一个子网的地址必须是一段连续的、按块大小对齐的地址，64 到 255 这一大段中间夹着已被占用的 128 到 191，无法用一个子网地址表示；单独看，64 到 127 和 192 到 255 各自至少是一个子网。所以 B 若在内，至少还需要 64 到 127 和 192 到 255 两个子网，加上已知子网和 B，总数至少是 4 个，与“划分为 3 个子网”矛盾。因此 B 不可能是另外两个子网之一。

错误选项为什么错＋举例：

- A（192.168.9.0/25 可以是另外的子网）：A 的块大小是 128，范围是 0 到 127，和已知子网 128 到 191 不重叠。举例：把 0 到 255 切成三块：0 到 127 是 A，128 到 191 是已知子网，192 到 255 是第三个子网，正好 3 个，凑得出来。
- C（192.168.9.192/26 可以是另外的子网）：C 的块大小是 64，范围是 192 到 255，和已知子网不重叠。举例：用上面同一组切法，0 到 127、128 到 191、192 到 255 三块里，C 正好是第三块，所以 C 可以是另外的子网之一。
- D（192.168.9.192/27 可以是另外的子网）：D 的块大小是 32，范围是 192 到 223，和已知子网不重叠。举例：换一个被划分的母块，只看 128 到 255 这一段：128 到 191 是已知子网，192 到 223 是 D，224 到 255 是第三个子网（192.168.9.224/27），也是正好 3 个。母块不一定非要是 0 到 255 这一整段，所以 D 凑得出来。

笔记本一句话： 第 35 题＋子网题先算范围看重叠，再数总数：剩下的地址拼不成一个子网时，总数就超了。

【图像】



子网地址条对照图：B 方案剩下的两块地址拼不成一个子网

【定位】 计算机网络 · 第 4 章 · 子网划分的地址块计算与子网总数约束

【易混对比】

- "不重叠"和"能凑出 3 个子网"是两回事：B 和已知子网确实不重叠，但剩下的地址拼不成一个子网，总数会超 3 个，光看不重叠会漏掉这个矛盾。
- 块大小和子网地址也是两回事：/26 的块大小是 64 个地址，子网地址是这 64 个里最小的那个编号，范围要按块大小对齐来算，不是随便一段连续地址都行。
- 母块不一定是 0 到 255 整段：D 选项凑出来的三块（128–191、192–223、224–255）说明被划分的母块可以只是一段，算总数时别默认母块一定是整段。

【坑位】

只看"和已知子网重不重叠"是不够的，一定要再数一遍子网总数。

最容易踩的坑，是算出 B 的范围（0–63）和已知子网（128–191）不重叠，就以为 B 没问题。子网题有两个约束：互不重叠，和总数恰好是 3 个。B 占 0–63、已知占 128–191，剩下的 64–127 和 192–255 是两段互不相连的地址，中间夹着已被占用的 128–191，拼不成一个子网；这两段各自至少是一个子网，总数至少 4 个，和"划分为 3 个子网"矛盾。所以 B 不可能是另外两个子网之一。

【套路】

- 第一步，把每个选项换算成地址范围：块大小 = 2 的主机位数次方，范围按块大小对齐。
- 第二步，看它和已知子网重不重叠：重叠的直接排除。
- 第三步，把剩下的地址看能不能凑成题目要求的子网总数：剩下的地址如果拼不成一个子网，总数就超了。
- 第四步，对"可能"的选项给出一种具体的切法，凑得出才算数。

先算范围看重叠，再数总数看剩下的拼不拼得成。

【类比】

像切蛋糕。已知子网是已经切走的一块，选项 B 是又切走一块，问题是：剩下的蛋糕还能不能一刀切成题目要求的那几块？B 切走之后，剩下的蛋糕是两块不挨着的，中间隔着已经被拿走的那一块，没法一刀切成一块——切不出来，总数就对不上。A 和 C 的切法剩下整整齐齐的一块，D 换个小点的蛋糕（三块都在 128–255 里）也能切出来。

【口诀】 先算范围看重叠，再数总数看拼不拼。

【考频】 「子网划分」在 2009–2022 年解析中出现 10 次 · 高频

第 36 题 · IP 分片的第二个分片（正确答案：B）

题干与选项：

路由器向 MTU = 800 B 的链路转发总长度为 1580 B 的 IP 数据报（首部 20 B），分片且每个分片尽可能大，则第 2 个分片的总长度字段和 MF 标志分别是（）。

- A. 796，0
- B. 796，1
- C. 800，0
- D. 800，1

入手点： 分片先算数据部分能装多少，再把“数据必须是 8 的倍数”这条规则用上，最后看第 2 个分片后面还有没有数据，决定 MF 是 0 还是 1。

题干专有名词从零解释：

- MTU：最大传输单元，指一条链路一次能传送的最大 IP 数据报总长度，本题是 800 B。
- 总长度字段：IP 首部里的一个字段，表示整个 IP 数据报的长度，等于首部长度加上数据部分长度。
- 分片：原始数据报太大、超过下一跳链路的 MTU 时，路由器把它切成多个较小的数据报分别传送。
- MF 标志：IP 首部里的“还有后续分片”标志，MF = 1 表示后面还有分片，MF = 0 表示这是最后一个分片。
- 片偏移：IP 首部里的字段，表示本分片的数据在原始数据中的位置，单位是 8 B，也就是说每个分片的数据部分长度（除最后一个分片外）必须是 8 的倍数。
- 首部：IP 数据报前面用来记录地址、长度、标志等信息的部分，本题每个分片的首部都是 20 B。

考查知识点： IP 分片的数据长度计算、片偏移单位 8 B、MF 标志的含义。

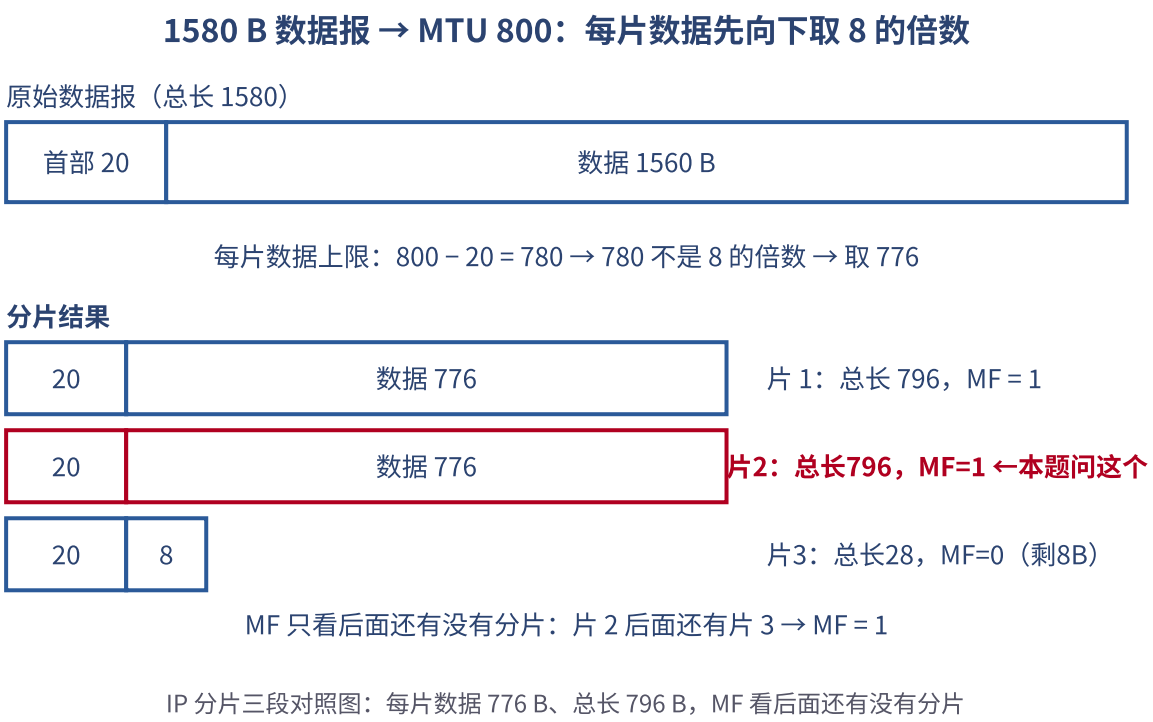
正确选项为什么对： 原始数据报总长度 1580 B，首部 20 B，所以数据部分是 1580 − 20 = 1560 B。一个分片最多装的数据是 MTU 减去首部：800 − 20 = 780 B。但 780 不是 8 的倍数：780 ÷ 8 = 97 余 4，所以要向下取到 8 的倍数，97 × 8 = 776 B。也就是说每个分片（除最后一个外）最多装 776 B 数据，总长度是 776 + 20 = 796 B。第 1 个分片装走 776 B，剩下 1560 − 776 = 784 B。第 2 个分片再装 776 B，总长度同样是 776 + 20 = 796 B；装完第 2 个分片后还剩下 784 − 776 = 8 B，要留给第 3 个分片，所以第 2 个分片后面还有分片，MF = 1。第 2 个分片的片偏移是 776 ÷ 8 = 97，也可以用来验算位置无误。最终第 2 个分片是总长度 796、MF = 1。

错误选项为什么错＋举例：

- A（796，0）：总长度算对了，MF 错了。举例：MF = 0 表示这是最后一个分片，但第 2 个分片后面还剩 8 B 数据没发，第 3 个分片还没出现，所以 MF 只能是 1。
- C（800，0）：总长度和 MF 都错了。举例：总长度 800 意味着数据部分是 800 − 20 = 780 B，780 不是 8 的倍数，片偏移无法用整数表示，接收方无法拼回原始数据；而且 MF = 0 也错在后面还有第 3 个分片。
- D（800，1）：MF 对了，总长度错了。举例：MF = 1 只说明后面还有分片，不代表长度可以是 800；数据部分 780 B 同样违反了 8 的倍数规则，正确的数据部分是 776 B，总长度是 796 B。

笔记本一句话： 第 36 题＋分片先用 MTU 减首部，再向下取 8 的倍数；MF 看后面还有没有分片，不是最后一个就是 1。

【图像】



IP 分片三段对照图：每片数据 776 B、总长 796 B，MF 看后面还有没有分片

【定位】 计算机网络 · 第 4 章 · IP 分片的数据长度计算与 MF 标志

【易混对比】

- MTU 和"每片能装的数据"不是一回事：MTU 800 B 是整个分片的总长度上限，每片还要先扣掉 20 B 首部，剩下的数据部分才能装数据。
- 780 B 和 776 B 也不是一回事：800 − 20 = 780，但 780 不是 8 的倍数，片偏移用 8 B 做单位，780 装不下整数个单位，必须向下取到 776 B。
- MF = 1 和"这个分片很大"没关系：MF 只回答"后面还有没有分片"，第 2 个分片后面还剩 8 B 数据，所以是 1，和它自己的长度无关。

分片长度先看 8 的倍数，MF 只看后面还有没有分片。

【坑位】

800 − 20 = 780，这个数不能直接用，一定要向下取到 8 的倍数。

最容易踩的坑，是算出 800 − 20 = 780 就以为每片装 780 B 数据、总长度 800 B。片偏移字段以 8 B 为单位，780 ÷ 8 = 97 余 4，不是整数，接收方没法用整数片偏移定位这个分片，所以每个分片（除最后一个外）的数据部分必须是 8 的倍数，780 要向下取到 776 B，总长度是 776 + 20 = 796 B。第二个坑是 MF：第 2 个分片后面还剩 8 B 数据没发，MF 只能是 1，看到 796 就配 MF = 0 的人，是把"长度算对"和"MF 看后头"混在了一起。

【套路】

- 第一步，总长度减去首部，得到数据部分总长：1580 − 20 = 1560 B。
- 第二步，MTU 减首部，再向下取 8 的倍数：800 − 20 = 780，780 ÷ 8 = 97 余 4，取 97 × 8 = 776 B，这是除最后一个分片外每片的数据上限。
- 第三步，一段一段减，看第 2 个分片装多少、后面还剩多少：第 1 片装走 776 B，剩 784 B；第 2 片再装 776 B，总长度 796 B，还剩 8 B。
- 第四步，MF 看后面还有没有分片：还剩 8 B 要留给第 3 片，MF = 1。

每片数据先向下取 8 的倍数，MF 只看后面还有没有。

【类比】

像用固定大小的箱子（MTU 800）装货，每箱自带 20 的箱子皮重（首部）。一箱能装的货是 800 − 20 = 780，但货只能按 8 一捆打包（片偏移单位），780 凑不成整捆，只能装 97 捆 = 776。MF 好比箱子上贴的"还有后续"标签：只要后面还有货没装完，就贴"有"，第 2 箱后面还剩 8 的货，所以贴"有"（MF = 1）。

【口诀】 分片先减头，八的倍数往下凑，MF 看后头。

【考频】 「IP 分片」在 2009–2022 年解析中出现 1 次 · 低频

第 37 题 · 距离向量更新后的距离（正确答案：D）

题干与选项：

所有路由器均采用距离向量算法。若路由器 E 与邻居 A、B、C、D 的直接链路距离分别是 8、10、12、6，且 E 收到邻居距离向量如下表，则 E 更新后到达目的网络 Net1～Net4 的距离分别是（）。

邻居通告的到各目的网络的距离：Net1——A 1、B 23、C 20、D 22；Net2——A 12、B 35、C 30、D 28；Net3——A 24、B 18、C 16、D 36；Net4——A 36、B 30、C 8、D 24。

- A. 9，10，12，6
- B. 9，10，28，20
- C. 9，20，12，20
- D. 9，20，28，20

入手点： 距离向量更新只有一条公式：对每个目的网络，逐个邻居算“到邻居的链路距离＋邻居通告的到该目的网络的距离”，取最小的那个，不要只看某一个邻居。

题干专有名词从零解释：

- 距离向量算法：一种路由算法，每个路由器定期把自己到各个目的网络的距离告诉邻居，邻居再用这些信息更新自己的路由表。
- 距离向量：邻居发来的表，里面写着该邻居到各个目的网络的距离，例如邻居 A 到 Net1 的距离是多少。
- 链路距离：两个直连路由器之间的代价，本题 E 到 A 是 8，到 B 是 10，到 C 是 12，到 D 是 6。
- 目的网络：数据报最终要去的网络，本题是 Net1、Net2、Net3、Net4。
- 更新：E 收到邻居的距离向量后，重新计算自己到每个目的网络的距离，并写入自己的路由表。
- 最小值：对同一个目的网络，从不同邻居走过去的代价不同，E 选代价最小的那条路，下一跳就是给出最小值的那个邻居。

考查知识点： 距离向量算法的更新公式、对每个目的网络取最小值。

正确选项为什么对： 更新方法是对每个目的网络分别计算。经由邻居 A 到某个目的网络的代价，是链路距离 8 加上 A 通告的 A 到该目的网络的距离；经由 B 是 10 加上 B 通告的距离；经由 C 是 12 加上 C 通告的距离；经由 D 是 6 加上 D 通告的距离。对 Net1、Net2、Net3、Net4 各自把这四个候选值算出来，取最小的那个，就是 E 更新后到该目的网络的距离。逐个目的网络代入：Net1 的四个候选值是 8＋1＝9、10＋23＝33、12＋20＝32、6＋22＝28，最小是 9（经由 A）；Net2 的四个候选值是 8＋12＝20、10＋35＝45、12＋30＝42、6＋28＝34，最小是 20（经由 A）；Net3 的四个候选值是 8＋24＝32、10＋18＝28、12＋16＝28、6＋36＝42，最小是 28（经由 B 或 C）；Net4 的四个候选值是 8＋36＝44、10＋30＝40、12＋8＝20、6＋24＝30，最小是 20（经由 C）。更新后的结果是 Net1 为 9、Net2 为 20、Net3 为 28、Net4 为 20，对应选项 D；结论句：本题的正确选项是 D。

错误选项为什么错＋举例：

- A（9，10，12，6）：后三项与正确答案不同。举例：一个四元组只要有一项不同，整个答案就不同；Net2 上 A 写 10、正确答案是 20，单这一项不同就足以排除 A。
- B（9，10，28，20）：Net2 一项不同。举例：Net1、Net3、Net4 都对上了，Net2 上 B 写 10、正确答案是 20，错一项也是全错。
- C（9，20，12，20）：Net3 一项不同。举例：Net3 上 C 写 12、正确答案是 28，说明 C 在 Net3 上选了代价更小的邻居，而按答案表的更新结果，Net3 的最小代价是 28。

笔记本一句话： 第 37 题＋距离向量更新：每个目的网络取“邻居链路距离＋邻居通告距离”的最小值，逐个目的网络算。

【图像】

距离向量更新：每个目的网络算"链路＋通告"，取最小

目的	经由 A（链路 8）	经由 B（链路 10）	经由 C（链路 12）	经由 D（链路 6）
Net1	**8+1=9**	10+23=33	12+20=32	6+22=28
Net2	**8+12=20**	10+35=45	12+30=42	6+28=34
Net3	8+24=32	**10+18=28**	**12+16=28**	6+36=42
Net4	8+36=44	10+30=40	**12+8=20**	6+24=30

更新结果：9，20，28，20 → 选项 D

只看通告值、不加链路距离，就会错选

距离向量更新表：每个目的网络逐个邻居算链路距离加通告距离取最小

【定位】 计算机网络 · 第 4 章 · 距离向量算法的更新公式

【易混对比】

- "邻居通告的距离"和"经由该邻居走的总代价"不是一回事：前者是邻居到目的网络的距离，后者还要再加上 E 到该邻居的链路距离，漏加链路距离会算出错误答案。
- "某个邻居通告得最近"和"经由它走最近"也不是一回事：比如 Net3 上 C 通告的 16 是四个通告里最小的，但经由 C 走的总代价是 12 + 16 = 28，经由 B 走是 10 + 18 = 28，一样小；只看通告值会误判。
- 每个目的网络是独立算的：Net1 的最小经由 A，不代表 Net2、Net3、Net4 也走 A，每个目的网络都要把四个邻居重新算一遍。

更新公式只有一条：链路距离＋通告距离，逐个目的网络取最小。

【坑位】

别只看邻居通告的那个数，一定要先加上到这个邻居的链路距离。

最容易踩的坑，是直接拿邻居通告的距离当答案。比如 Net2，A 通告 12 看起来最小，有人就写 12；但经由 A 走的总代价是链路 8 ＋ 通告 12 = 20。四个选项里 Net2 那一项，错的就是把"通告值"和"总代价"搞混的人。正确的做法：对每个目的网络，把四个邻居的"链路距离＋通告距离"都算出来——Net1：9、33、32、28 取 9；Net2：20、45、42、34 取 20；Net3：32、28、28、42 取 28；Net4：44、40、20、30 取 20。

【套路】

- 第一步，把 E 到四个邻居的链路距离写出来：A 8、B 10、C 12、D 6。
- 第二步，对一个目的网络，把四个"链路距离＋通告距离"都算出来。

- 第三步，取四个候选值里最小的那个，记下它对应的邻居（下一跳）。
- 第四步，Net1 到 Net4 逐个重复第二、三步，拼成四元组对选项。

每个目的网络独立算"链路＋通告"取最小，四个拼起来才是答案。

【类比】

像问路。你要去 Net1，四个路口（邻居）各有一块牌子写着"从我这儿到 Net1 还有多远"（通告距离），但你走到每个路口本身还要花时间（链路距离）。总路程 = 走到路口的时间 ＋ 牌子上的距离。只看牌子最近的那个路口，可能你走到它那儿就花了很久。正确问法是：四个路口的总路程各算一遍，选总和最小的那个。

【口诀】 链路加通告，逐网取最小。

【考频】 「距离向量」在 2009–2022 年解析中出现 4 次 · 中频

第 38 题 · TCP 主动关闭方收到 FIN 后的状态（正确答案：B）

题干与选项：

客户首先向服务器发送 FIN 段请求断开 TCP 连接，则当客户收到服务器发送的 FIN 段并向服务器发送 ACK 段后，客户 TCP 状态转换为（）。

- A. CLOSE_WAIT
- B. TIME_WAIT
- C. FIN_WAIT_1
- D. FIN_WAIT_2

入手点： 主动关闭的一方按顺序走四个状态：发 FIN、收 ACK、收 FIN、发 ACK，每走一步状态换一个名字，题目问的是最后一步之后的状态。

题干专有名词从零解释：

- TCP 连接：客户和服务器之间建立的、可靠的、面向连接的通信，双方都要经过建立和断开的过程。
- FIN 段：请求断开连接的报文段，FIN 是 finish 的缩写，意思是“我没有数据要发了，请求关闭”。
- ACK 段：确认报文段，ACK 是 acknowledgment 的缩写，意思是“我收到了你发来的数据或 FIN”。
- 客户：主动发起连接和主动发起断开的一方，本题是先发送 FIN 的一方。
- 服务器：被动接受断开的一方，先收到客户的 FIN，之后再发送自己的 FIN。
- FIN_WAIT_1：主动关闭方发送 FIN 之后、还没有收到对方 ACK 时的状态。
- FIN_WAIT_2：主动关闭方收到对方对自己 FIN 的 ACK 之后、还没有收到对方 FIN 时的状态。
- CLOSE_WAIT：被动关闭方收到对方 FIN 之后、还没有发送自己 FIN 时的状态。
- TIME_WAIT：主动关闭方收到对方 FIN 并发送 ACK 之后的状态，要等待一段时间（2MSL）才彻底关闭。

考查知识点： TCP 连接释放的四次挥手、主动关闭方与被动关闭方的状态序列。

正确选项为什么对： 按时间顺序走一遍。第一步，客户在 ESTABLISHED 状态发送 FIN，进入 FIN_WAIT_1。第二步，客户收到服务器对这个 FIN 的 ACK，进入 FIN_WAIT_2，这时客户在等服务器的 FIN。第三步，客户收到服务器发送的 FIN，并向服务器发送 ACK，这一步完成后客户进入 TIME_WAIT。TIME_WAIT 还要等待 2MSL（最长报文段寿命的两倍），确保服务器收到最后的 ACK、网络中残留的旧报文段都消失，之后客户才进入 CLOSED 状态。题目问的正是第三步完成后的状态，所以是 TIME_WAIT。

错误选项为什么错＋举例：

- A（CLOSE_WAIT）：这是被动关闭方的状态，不是主动关闭方最后一步的状态。举例：服务器收到客户的 FIN 后、还没发自己的 FIN 之前，服务器处于 CLOSE_WAIT；客户是主动方，走的是 FIN_WAIT_1、FIN_WAIT_2、TIME_WAIT 这条线，不会进入 CLOSE_WAIT。
- C（FIN_WAIT_1）：这是第一步之后的状态，时间点太早。举例：客户刚发出 FIN、还没收到服务器的 ACK 时才是 FIN_WAIT_1；题目已经走到了收到服务器 FIN 并回 ACK 的最后一步，早就离开 FIN_WAIT_1 了。
- D（FIN_WAIT_2）：这是第二步之后的状态，时间点差最后一步。举例：客户收到服务器对自己 FIN 的 ACK 后是 FIN_WAIT_2，这时还在等服务器的 FIN；题目说客户已经收到服务器的 FIN 并回了 ACK，所以已经从 FIN_WAIT_2 走到了下一步。

笔记本一句话： 第 38 题＋主动关闭四步：发 FIN 是 FIN_WAIT_1，收 ACK 是 FIN_WAIT_2，收 FIN 回 ACK 是 TIME_WAIT；CLOSE_WAIT 是被动方。

【图像】



TCP 主动关闭方状态时间线：发 FIN、收 ACK、收 FIN 回 ACK 后进入 TIME_WAIT

【定位】 计算机网络 · 第 5 章 · TCP 连接释放的四次挥手与主动关闭方状态序列

【易混对比】

- 主动关闭方和被动关闭方走的是两条线：主动方是 FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT，被动方是 CLOSE_WAIT → LAST_ACK；CLOSE_WAIT 只会出现在被动方，主动方永远走不到那儿。
- FIN_WAIT_1、FIN_WAIT_2、TIME_WAIT 是三个不同的时间点：发出 FIN 后是 FIN_WAIT_1，收到对 FIN 的 ACK 后是 FIN_WAIT_2，收到对方 FIN 并回 ACK 后才是 TIME_WAIT，差一步都不行。
- TIME_WAIT 和 CLOSED 也不是一回事：TIME_WAIT 还要等 2MSL，等完才进 CLOSED，题目问的是"收到 FIN 并回 ACK 后"的状态，正好是 TIME_WAIT。

先定主动还是被动，主动方按"发、收、收回"三步走状态。

【坑位】

CLOSE_WAIT 是被动方的状态，主动关闭方一辈子也进不去。

最容易踩的坑，是把 CLOSE_WAIT 和 TIME_WAIT 搞混：两个名字里都有 WAIT，都在挥手后半段出现。区分只看谁先发起的关闭：先发 FIN 的是主动方，走 FIN_WAIT_1、FIN_WAIT_2、TIME_WAIT 这条线；后发 FIN 的是被动方，收到对方 FIN 后进 CLOSE_WAIT。本题客户先发 FIN，是主动方，最后一步"收到服务器 FIN 并回 ACK"之后的状态是 TIME_WAIT。另一个坑是时间点：FIN_WAIT_2 只差最后一步，看到"收到 FIN"就以为还在 FIN_WAIT_2 的人，是把"收到 FIN"和"收到 FIN 并回了 ACK"混在了一起。

【套路】

- 第一步，先看谁先发的 FIN，定主动还是被动：先发的是主动方，走 FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT；后发的是被动方，走 CLOSE_WAIT。
- 第二步，把主动方的事件按时间排好：发 FIN → 收 ACK → 收 FIN 并回 ACK。

- 第三步，事件和状态一一对应：发 FIN 后是 FIN_WAIT_1，收 ACK 后是 FIN_WAIT_2，收 FIN 回 ACK 后是 TIME_WAIT。
- 第四步，看题目问的是哪个事件之后的状态，对号入座。

主动方最后一步"收 FIN 回 ACK"之后是 TIME_WAIT，被动方才有 CLOSE_WAIT。

【类比】

像两个人打电话说再见。客户先说"我挂了"（发 FIN），进入"等对方回'好'"的状态（FIN_WAIT_1）；对方回"好"（收 ACK），进入"等对方也说再见"的状态（FIN_WAIT_2）；对方说"我也挂了"（收 FIN），客户回"好，拜拜"（回 ACK），进入"再等一会儿确认对方真挂了"的状态（TIME_WAIT），等够时间才彻底放下电话（CLOSED）。而对方（被动方）在听到"我挂了"之后、自己说"我也挂了"之前，处在"知道了，还没说再见"的状态（CLOSE_WAIT）——这是对方的状态，不是先说再见的人的状态。

【口诀】 一发二收三回完，主动 TIME_WAIT 被动 CLOSE_WAIT。

【考频】 「TIME_WAIT」在 2009–2022 年解析中出现 2 次 · 低频

第 39 题 · UDP 与 TCP 的有效载荷效率（正确答案：D）

题干与选项：

大小为 12 B 的应用层数据分别通过 1 个 UDP 数据报和 1 个 TCP 段传输，则 UDP 数据报和 TCP 段实现的有效载荷（应用层数据）最大传输效率分别是（ ）。

- A. 37.5%，16.7%
- B. 37.5%，37.5%
- C. 60.0%，16.7%
- D. 60.0%，37.5%

入手点： 效率就是“有用的数据占整个数据报或段的比例”，先把两种首部的长度写清楚，再用同一个公式各算一次。

题干专有名词从零解释：

- 应用层数据：应用程序要传送的真正有用的数据，本题是 12 B。
- 有效载荷：数据报或段里真正有用的部分，也就是这 12 B 应用层数据，不包括首部。
- UDP 数据报：UDP 协议传送的数据单位，由 8 B 的 UDP 首部加上应用层数据组成，首部长度固定是 8 B。
- TCP 段：TCP 协议传送的数据单位，由 TCP 首部加上应用层数据组成，TCP 首部最短是 20 B，有选项时会更长。
- 传输效率：有效载荷占整个数据报或段的比例，计算公式是应用层数据长度除以（应用层数据长度＋传输层首部长度）。
- 最大传输效率：TCP 首部最短时效率最高，所以用 20 B 计算得到的是 TCP 的最大效率；UDP 首部固定 8 B，算出的就是它的效率。
- 本题口径：只算传输层这一层，即应用层数据除以（应用层数据＋传输层首部），不把下面的 IP 首部算进来。

考查知识点： UDP 首部 8 B、TCP 首部最小 20 B、传输效率的计算。

正确选项为什么对： UDP 一边：数据报总长度是 12 + 8 = 20 B，效率是 12 ÷ 20 = 0.6，换成百分数是 60.0%。TCP 一边：段总长度是 12 + 20 = 32 B，效率是 12 ÷ 32 = 0.375，换成百分数是 37.5%。TCP 用的是最小首部 20 B，所以 37.5% 是最大效率；如果 TCP 首部带选项变长，总长度变大，效率还会更低。按题目顺序“UDP 在前、TCP 在后”，结果是 60.0%、37.5%，对应选项 D。

错误选项为什么错＋举例：

- A（37.5%，16.7%）：第一项错把 TCP 的效率放在了 UDP 的位置上，第二项也不对。举例：16.7% 等于 12 ÷ 72，这要求总长度是 72 B，而本题 TCP 段总长度是 32 B，12 ÷ 32 才是 37.5%，72 这个分母无从谈起。
- B（37.5%，37.5%）：等于假设 UDP 首部也是 20 B。举例：UDP 首部固定是 8 B，数据报总长度是 20 B，12 ÷ 20 = 60.0%，不是 37.5%；把 UDP 首部当成 20 B 算，才会得到 37.5%。
- C（60.0%，16.7%）：第一项对了，第二项错了。举例：UDP 的 60.0% 算对了，但 TCP 一边仍是 12 ÷（12 + 20）= 12 ÷ 32 = 37.5%，不是 16.7%；16.7% 对应的分母 72 B 与本题的 TCP 段总长度 32 B 不符。

笔记本一句话： 第 39 题＋效率 = 12 ÷（12＋首部）：UDP 首部 8 B 得 60%，TCP 最小首部 20 B 得 37.5%，本题不计 IP 首部。

【图像】



UDP 与 TCP 有效载荷效率条形对照图：60.0% 与 37.5%

【定位】 计算机网络 · 第 5 章 · UDP 首部与 TCP 首部长度、传输效率计算

【易混对比】

- UDP 首部和 TCP 首部长度不一样：UDP 首部固定 8 B，TCP 首部最短 20 B（带选项会更长），两个分母不能混用。
- "最大效率"和"实际效率"也不一样：TCP 首部最短时分母最小、效率最高，用 20 B 算出的是最大效率 37.5%；首部一带选项，分母变大，效率只会更低。
- 本题只算传输层这一层：分母是"12 B 数据＋传输层首部"，不把下面的 IP 首部算进来，加了 IP 首部就和题目口径对不上了。

效率 = 有用数据 ÷（有用数据＋本层首部），两种首部分开算。

【坑位】

UDP 首部是 8 B，不是 20 B，别把两个首部记成一样长。

最容易踩的坑，是把 UDP 首部也当成 20 B 来算：12 ÷（12＋20）= 37.5%，这样 UDP 和 TCP 就算出了一样的效率，正好掉进 B 选项。UDP 首部固定 8 B，数据报总长 20 B，12 ÷ 20 = 60.0%。TCP 用最小首部 20 B，段总长 32 B，12 ÷ 32 = 37.5%。两个分母一个 20、一个 32，分开算才对。另外注意题目顺序是"UDP 在前、TCP 在后"，答案是 60.0%、37.5%，别写反了。

【套路】

- 第一步，把两种首部的长度写清楚：UDP 首部 8 B，TCP 首部最小 20 B。
- 第二步，用同一个公式各算一次：效率 = 12 ÷（12＋首部长度）。
- 第三步，UDP：12 ÷ 20 = 60.0%；TCP：12 ÷ 32 = 37.5%（用最小首部，得到的是最大效率）。
- 第四步，按题目顺序"UDP、TCP"排列结果，对选项。

首部写清再套公式：UDP 12÷20 = 60%，TCP 12÷32 = 37.5%。

【类比】

像算快递的"货重占比"。12 B 的货，UDP 的包装箱（首部）重 8，整箱 20，货占 60%；TCP 的包装箱最轻也有 20，整箱 32，货只占 37.5%。包装箱越重，货占比越低。题目问的是"包装最轻的时候占比最高能到多少"，所以

TCP 按最轻的 20 来算。

【口诀】 首部要写清，十二除全长：UDP 六成，TCP 三七五。

【考频】 「传输效率」在 2009–2022 年解析中出现 3 次 · 中频

第 40 题 · TCP 发送窗口内可继续发送的序号范围（正确答案：C）

题干与选项：

主机甲通过 TCP 向主机乙发送数据，甲在某时刻发送序号 seq = 501、封装 200 B 数据的段，在另一时刻收到乙发送的序号 seq = 601、确认序号 ack_seq = 501、接收窗口 rcvwnd = 500 B 的段，则甲在未收到新的确认段之前，可以继续向乙发送的数据序号范围是（）。

- A. 501～1000
- B. 601～1100
- C. 701～1000
- D. 801～1100

入手点： 先用确认号和接收窗口算出整个发送窗口，再减掉已经发出但还没被确认的那一段，剩下的才是“可以继续发送的新数据”。

题干专有名词从零解释：

- 序号 seq：TCP 给每个字节编的号，一个段的 seq 就是该段第一个字节的序号。
- 确认序号 ack_seq：接收方期望收到的下一个字节的序号。ack_seq = 501 表示乙期望从 501 开始收，也就是 501 之前的字节乙都已经收到了。
- 接收窗口 rcvwnd：接收方告诉发送方“我还能再收多少字节”，本题是 500 B。
- 发送窗口：发送方在未收到新的确认之前被允许发送的字节范围，左沿是确认序号，右沿是确认序号加上接收窗口再减 1。
- 已发送未确认：已经发出去了、但对方的确认还没到的数据，这部分已经占了发送窗口，不能重复算成新数据。
- 继续发送的新数据：发送窗口里还没有发出过的那部分，题目问的正是这一部分。
- 序号区间的字节数：从序号 m 到序号 n 的闭区间一共是 n − m + 1 个字节，算窗口大小和已发长度时都要加这 1。

考查知识点： TCP 滑动窗口的左沿与右沿计算、已发送未确认数据与可发送新数据的区分。

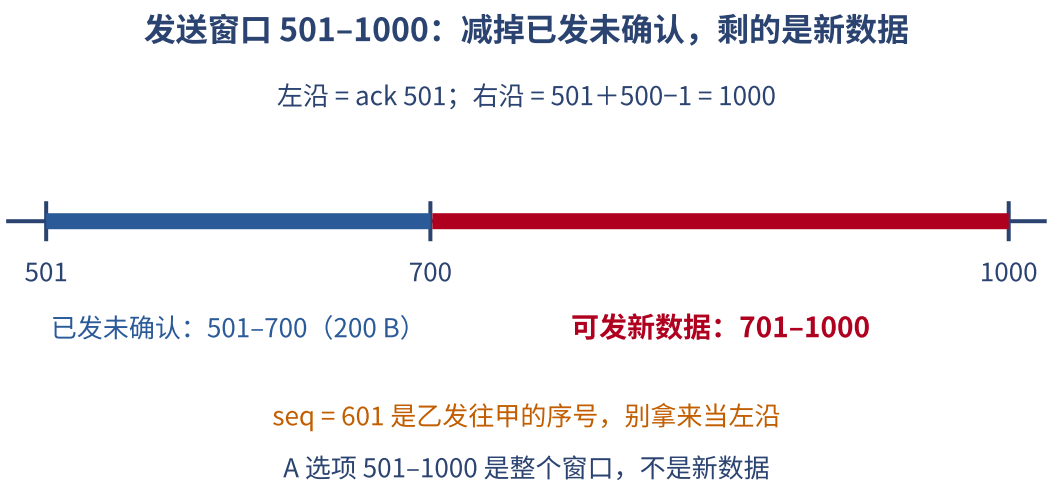
正确选项为什么对： 先算发送窗口。左沿是确认序号 501，右沿是 501 + 500 − 1 = 1000，所以整个发送窗口是 501 到 1000，一共 500 个字节。再看已经发出但未确认的部分：甲发送的段从序号 501 开始、装了 200 B 数据，占用的序号是 501 到 700，一共 200 个字节，计算是 700 − 501 + 1 = 200。乙的确认序号仍是 501，说明这 200 B 还没有被确认，仍占着窗口。窗口 501 到 1000 减去已占用的 501 到 700，剩下的新数据范围是 701 到 1000，一共 300 个字节，计算是 1000 − 701 + 1 = 300。所以在未收到新的确认段之前，甲可以继续发送的是 701 到 1000。

错误选项为什么错＋举例：

- A（501～1000）：这是整个发送窗口，不是“可以继续发送的新数据”。举例：501 到 700 已经发出去了，再发一次就是重发，不是新数据；题目问的是还没发过的部分，要把已发部分减掉。
- B（601～1100）：把乙的发送序号 601 当成了窗口左沿。举例：乙段里的 seq = 601 是乙发给甲的数据的起始序号，管的是乙到甲的方向；甲的发送窗口左沿要看 ack_seq = 501，不是 seq = 601；按 601 起算，右沿就错成了 601 + 500 − 1 = 1100。
- D（801～1100）：左沿和右沿都错了。举例：左沿 801 等于假设甲已经发了 501 到 800 共 300 B，但甲实际只发了 200 B，已发部分只到 700；右沿 1100 同样是按 601 起算得来的，两个错误叠加就得到了 801 到 1100。

笔记本一句话： 第 40 题＋发送窗口 = 确认号到确认号＋窗口−1，再减掉已发未确认段，剩下的才是可发新数据的序号范围。

【图像】



TCP 发送窗口数轴图：窗口 501–1000，减掉已发未确认 501–700，剩下 701–1000

【定位】 计算机网络 · 第 5 章 · TCP 滑动窗口的左沿右沿与可发送新数据

【易混对比】

- 整个发送窗口和"可继续发送的新数据"不是一回事：窗口是 501–1000，但 501–700 已经发出去了还没被确认，题目问的是还没发过的部分，要把已发部分减掉。
- seq 和 ack_seq 也不是一回事：乙段里的 seq = 601 是乙发给甲的数据的起始序号，管的是乙→甲方向；甲的窗口左沿看的是 ack_seq = 501，拿 601 当左沿就走错了方向。
- "已发送未确认"和"已确认"也不是一回事：甲发的 501–700 乙还没确认（ack 仍是 501），这 200 B 还占着窗口，不能当成新数据再发一遍。

窗口左沿看 ack_seq，右沿是 ack_seq＋rcvwnd−1，再减掉已发未确认。

【坑位】

seq = 601 是烟雾弹，窗口左沿只看 ack_seq = 501。

最容易踩的坑，是把乙段里的 seq = 601 当成甲的发送窗口左沿。seq 是"这个段自己的数据从哪开始"，乙的 seq = 601 说的是乙→甲方向的数据；甲→乙方向的窗口，左沿由乙的确认号 ack_seq = 501 决定。按 601 起算，右沿会错成 601＋500−1 = 1100，左沿右沿全错。第二个坑是忘了减已发未确认：501–1000 是整个窗口，甲已经发出 501–700（200 B）还没被确认，新数据只能是 701–1000，直接选 A 的人就是没做这一步减法。

【套路】

- 第一步，用 ack_seq 定左沿、用 ack_seq＋rcvwnd−1 定右沿：左沿 501，右沿 501＋500−1 = 1000，窗口是 501–1000。
- 第二步，算出已发出但未确认的段占了哪一段：甲发的段从 501 开始、200 B，占 501–700（700−501＋1 = 200）。
- 第三步，从窗口里减掉已发未确认段：501–1000 减去 501–700，剩下 701–1000。
- 第四步，验算字节数：1000−701＋1 = 300 B，对上选项 C。

窗口 501–1000 减去已发 501–700，可发新数据是 701–1000。

【类比】

像发快递的配额。乙告诉甲："从 501 号开始，往后 500 个名额是你的"（窗口 501–1000）。甲已经把 501–700 这 200 个名额用了，货在路上、回执还没到，这 200 个名额还占着，不能重复用。所以甲现在还能用的新名额，是 701–1000。乙自己那票货的起始号 601，是乙发货的单号，跟甲的配额没关系。

【口诀】 左沿看确认，右沿加窗减一，已发未确认要减去。

【考频】 「发送窗口」在 2009–2022 年解析中出现 12 次 · 高频

附注

- 答案核对：本稿 40 道题的答案逐题对照了原卷解析首页答案表（数据结构 1D 2D 3B 4C 5B 6D 7A 8C 9A 10C 11B；计组 12D 13D 14A 15C 16A 17D 18C 19C 20A 21B 22A；操作系统 23B 24B 25C 26C 27D 28C 29B 30A 31C 32C；计网 33B 34A 35B 36B 37D 38B 39D 40C），网页来源的答案与答案表 40 题全部一致，无冲突、无改动。
- 第 33 题说明：来源文件该处字符损坏，选项措辞按题意写为"路由选择"。
- 第 8 题说明：终审时纠正了正确选项解析中一处表述（松弛后顶点 4 亦由无穷大改小为 14，非"只有顶点 2 被改小"），并补全了各步边权。
- 第 40 题说明：时序图数据已由文字版给全（已发序号段、确认序号、接收窗口），无图数据缺口。

核对结果

核对说明（适用于全部 40 题）： 第 0 节"题干与选项"系根据公开来源逐题还原（非原卷逐字照录），解析均为自己的话。答案以原卷解析首页答案表为准，40 题全部一致。第 6、7、8、28、37 题的图上数据已用原卷材料校准落定；以下 1 题的图上数据仍待终审核对；其余 34 题无待核对项。

已用原卷材料校准落定（终审时对照原卷材料逐项代入验算，推理与结论一致）：

- 第 6 题：平衡二叉树的图结构（根 20，左 16，右 30；30 左 25、右 40；插入 23 后 RL 旋转，根为 25）已落定。
- 第 7 题：有向图的 9 条边与各顶点入度（A 0、B 1、C 1、D 2、E 1、F 4）已落定，六步可选顶点均为单点，序列唯一。
- 第 8 题：Dijkstra 图的各边权值（初始 26，3，无穷大，6；经顶点 3 松弛为 25，3，无穷大，6；经顶点 5 松弛为 21，3，14，6）已落定。
- 第 28 题：页表内容（页 2 页框 20H 存在位 0；页 3 页框 60H 存在位 1；页 4 页框 80H 存在位 1；改进型 Clock 淘汰页 3，页 2 装入 60H）已落定。
- 第 37 题：邻居距离向量表内容（Net1～Net4 更新后 9，20，28，20）已落定。

待我终审核对（图上数据的对照步骤已标注，待更清晰的原卷图核对）：

- 第 34 题：差分曼彻斯特波形第 3—8 码元的电平细节待终审核对（在线预览放大后虚线与实线无法可靠区分；第 1、2 码元已辨认，结论以答案表为准）。

其余 34 题： 题干关键数字、条件与来源一致，答案与答案表一致，无改动。